CS102 - Week 1 Notes Outline and Study Guide

Table of Contents

What is inside

• Week 1 overview and master framework: data structures, algorithms, indexed access, arrays, searching, and efficiency

• Lesson 1 - Course Orientation and What to Expect

• Lesson 2 - What Is a Data Structure?

• Lesson 3 - Indexed Data Structures

• Lesson 4 - Understanding Arrays

• Lesson 5 - Accessing Elements in Arrays

• Lesson 6 - Updating and Replacing Values

• Lesson 7 - Lists vs Arrays and Dynamic Collections

• Lesson 8 - Adding and Removing Data

• Lesson 9 - Searching Through Collections

• Lesson 10 - Introduction to Algorithms

• Lesson 11 - Linear Search

• Lesson 12 - Comparing Search Strategies

• Lesson 13 - Organizing Information Efficiently

• Lesson 14 - Week 1 Integration and Review

• Glossary, concept connections, common mistakes, review guide, cheat sheet, and exam reference

Week 1 at a Glance

Week 1 is where programming starts to move from writing code that runs to designing systems that can organize and process information well. The central theme is organization.Data structures answer, "How should this information be arranged?" Algorithms answer, "What steps should I follow to solve the problem?"

The lessons build a foundation for indexed access, arrays, collections, searching, and efficiency. You will practice thinking about data before implementation, because the structure you choose can make a solution easier to understand, easier to maintain, and faster as the amount of information grows.

• Data structures organize information so programs can access, update, search, and manage it.

• Algorithms are step-by-step procedures for solving problems.

• Indexed access lets a program retrieve information by position.

• Arrays and collections group related data instead of scattering it across separate variables.

• Sequential access checks items one at a time; direct access jumps straight to a known location.

• A strong programmer asks: what information do I have, how should it be organized, and what steps should process it?

Memory hook: Structure first. Steps second. Code third.

Made with Goodnotes

Lesson 1 - Course Orientation and What to Expect

Big Idea

CS102 shifts the focus from simply writing Python syntax to thinking about how software systems organize information. The point is not just to make code run; the point is to design code that stays understandable, maintainable, and efficient as problems get bigger.

Full Explanation

Up to this point, programming has mostly involved learning language features, writing functions, and building smaller programs. CS102 adds another layer: deciding how information should be stored, retrieved, updated, searched, and organized. That matters because two programs can produce the same result but still differ in quality. One may be repetitive, slow, fragile, or hard to update. Another may be clean, organized, and easier to scale. Data structures and algorithms are the tools that help programmers make those design choices on purpose.
The course is less about memorizing definitions and more about building judgment. Each time you meet a new structure or algorithm, the question is not just "What is it called?" The better question is: "What problem does this help solve, and what trade-offs come with it?"

Why This Lesson Matters

Beginners often think stronger programming means learning more syntax. This lesson argues that strong programming also means learning how to design solutions before typing code.
• Think about information first.

• Think about organization second.

• Think about implementation third.

Real-World Connection

Imagine organizing thousands of photos from a horse show. If every image lands in one folder with random names, finding a specific rider becomes painful. Organizing by event, class, rider, date, or client creates a structure that makes searching easier. That organization system is the real-world version of a data structure. The way you search through it is similar to an algorithm.

Term	Plain-English Meaning	Why It Matters
Data	Information a program can store, process, or use.	It is the raw material programs work with.
Data Structure	A method for organizing data efficiently.	It determines how easy data is to access or change.
Algorithm	A sequence of steps used to solve a problem.	It describes the process, not just the storage.
Efficiency	How much work a task requires.	Less work usually means faster, cleaner programs.
Scalability	How well a solution performs as data grows.	A good small solution should not collapse at large size.

Why Beginners Usually Miss This

• They treat the course as vocabulary memorization instead of design practice.

• They focus on code syntax and forget that organization affects performance.

• They assume there is one perfect solution instead of weighing trade-offs.

Helpful tip: When a lesson feels simple, ask what thinking habit it is trying to train.

Notes

_____

_____

_____

_____

Lesson 2 - What Is a Data Structure?

Made with Goodnotes

Big Idea

Every program works with information. A data structure is the way that information is organized so it can be accessed, updated, searched, and managed efficiently. The information itself matters, but the organization is what makes it usable.

Full Teaching Explanation

A data structure is easier to understand when you compare organized and unorganized information. A pile of random papers may contain everything you need, but finding one document takes forever. Put those same papers into labeled folders and the information becomes useful because the structure supports retrieval.
Programming works the same way. A music app stores playlists, a bank stores customer records, a navigation app stores locations, and an online store stores products and inventory. Each system needs a different structure because each system has different goals.
This is the big lesson: there is rarely one perfect way to store data. Good programmers choose a structure based on what the program needs to do most often - search, add, remove, update, preserve order, or group related information.

Unorganized Information	Organized Information	Real-Life Example	Purpose
Hard to find data	Easy to find data	Playlist	Organizes songs
Slow searching	Faster searching	Grocery receipt	Organizes purchases
Confusing structure	Predictable structure	Calendar	Organizes events
Difficult to maintain	Easier to update	Contact list	Organizes people
Increased mistakes	Reduced mistakes	Filing cabinet	Organizes documents
		Study guide	Organizes learning material

Real-World Examples

- Grocery receipt: item names, quantities, prices, discounts, taxes, and total cost are arranged predictably so questions can be answeredquickly.
- Music playlist: songs are stored in a specific order so you can play next, skip back, rearrange, add, or remove songs.
- Study notes: lessons, definitions, examples, reviews, and cheat sheets are grouped so studying is not random scavenger hunting.

Programmer Thinking

Before writing code, ask: How should this information be organized? If I am building Spotify, I need playlists. If I am building a banking app, I need customer records. If I am building a school system, I need student information. The structure affects everything that comes afterward.

Lesson 2 - Code Pattern and Study Check

Code Example - Grouping Related Data

```
grocery_items = [
    "milk",
    "bread",
    "eggs",
    "apples"
]
print(grocery_items)
grocery_items.append("orange juice")
```

What This Code Does

The list stores several grocery items together in one organized collection. Instead of creating separate variables like `i t e m 1 ,i t e m 2` , and `i t e m 3` , the list keeps related information grouped.
The append line shows why structure matters: adding another item becomes simple because the collection already has a place for related data.

Made with Goodnotes

Terms and Definitions	Common Mistakes and Helpful Tips
• Data: information that can be stored and used by a program. • Data structure: a method of organizing data so it can be usedefficiently. • Organization: arranging information in a meaningful way. • Collection: a group of related pieces of information storedtogether. • Efficiency: how much work is required to perform a task.	Mistake: Thinking data structures only exist in code. Reality: People use organizational structures constantly. Mistake: Memorizing definitions without understanding purpose. Reality: The purpose matters more than the label. Tip: For every data structure, ask what it organizes, why it is useful, what problem it solves, and when you would choose it.

Review Checkpoint

- What is a data structure?
- Why does organization matter?
- How is a playlist an example of a data structure?
- Why might two programs choose different structures for the same information?
- What role does efficiency play in data organization?

Key Takeaways

- Data structures organize information.
- Organization affects efficiency.
- Different problems require different structures.
- Real-world organizational systems are data structures too.
- Choosing the right structure makes programming easier.

Additional Notes

_____

_____

_____

_____

_____

_____

Lesson 3 - Indexed Data Structures

Big Idea

Indexed data structures make direct access possible. Instead of checking every item one at a time, an index acts like an address that tells the program exactly where a value is stored.

Full Teaching Explanation

The main problem this lesson solves is retrieval: how do we find the information we need quickly? Without an index, a program may have to use sequential access, checking items one by one until it finds the target.
An index gives the program a known location. If you know the item is in locker 327, you do not open lockers 1 through 326 first. You go directly to locker 327. That same idea applies to many programming collections.
This matters more as data grows. Searching through 5 items is not a huge deal. Searching through 5,000 or 5,000,000 items can become a serious performance problem. Indexed access reduces that work when the position is known.

Locker Hallway Mental Model	Sequential Access	Indexed Access

Made with Goodnotes

Sequential access: start at locker 1 and check each locker until you reach the right one.
Indexed access: if you already know the locker number, you walk straight to that locker.
The index works like an address. It points to the location instead of forcing a search.

Checks items one at a time	Jumps directly to a location
May require many checks	Usually one lookup
Gets slower as data grows	Stays fast when position is known
Like searching a pile	Like using an address

Code Example - Indexed Access

```
items = ["A", "B", "C", "D", "E", "F", "G",
"H"] print(items[7]) # Output:
# H
```

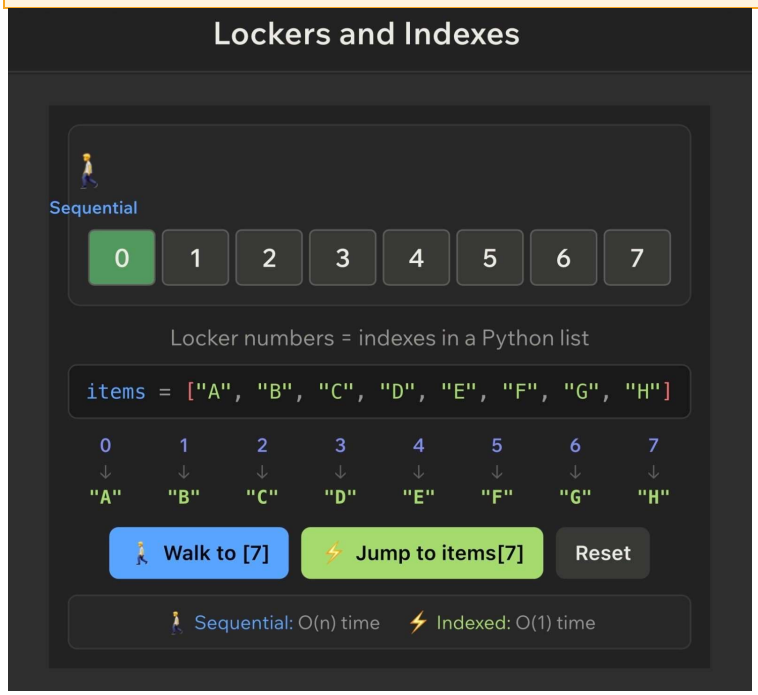
What Happened?

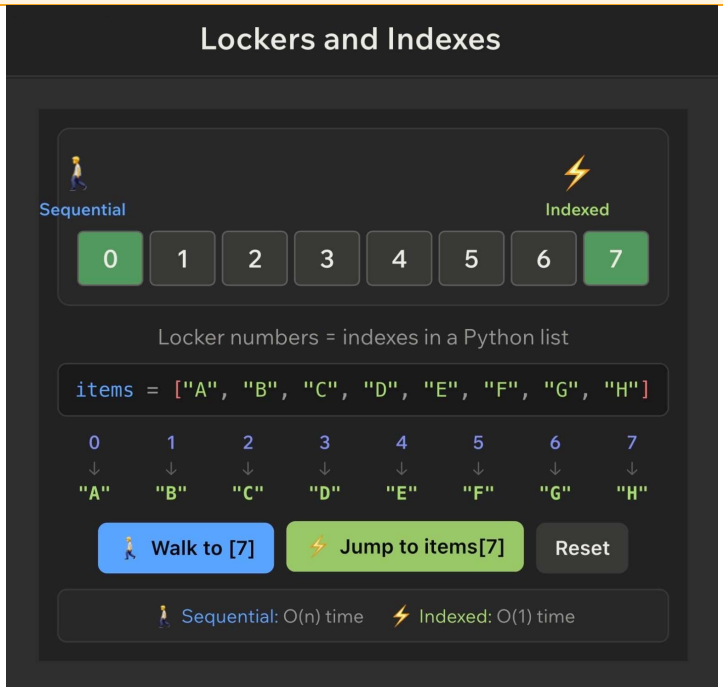
The program did not search for H. The expression items[7] told Python exactly which position to retrieve. The index served as an address.

Zero-Based Indexing

Python positions start at 0, not 1. This is one of the most common beginner mistakes.

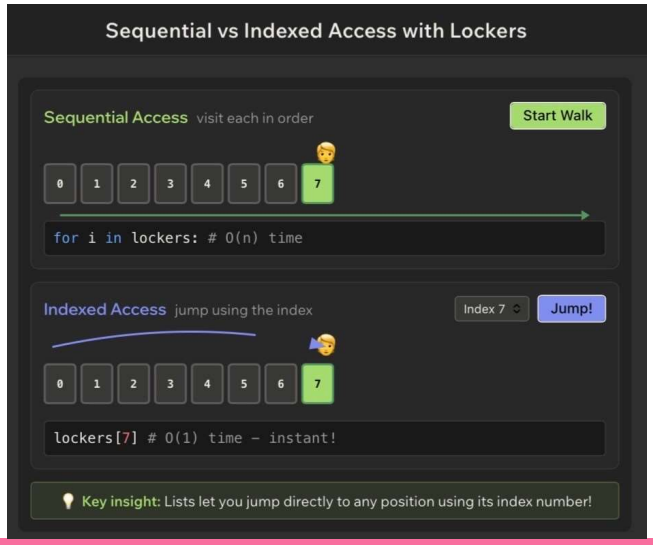
Human counting: 1 2 3 4 5
Python indexing: 0 1 2 3 4
Example: in colors = ["red", "blue", "green"], colors[0] returns red and colors[2] returns green.

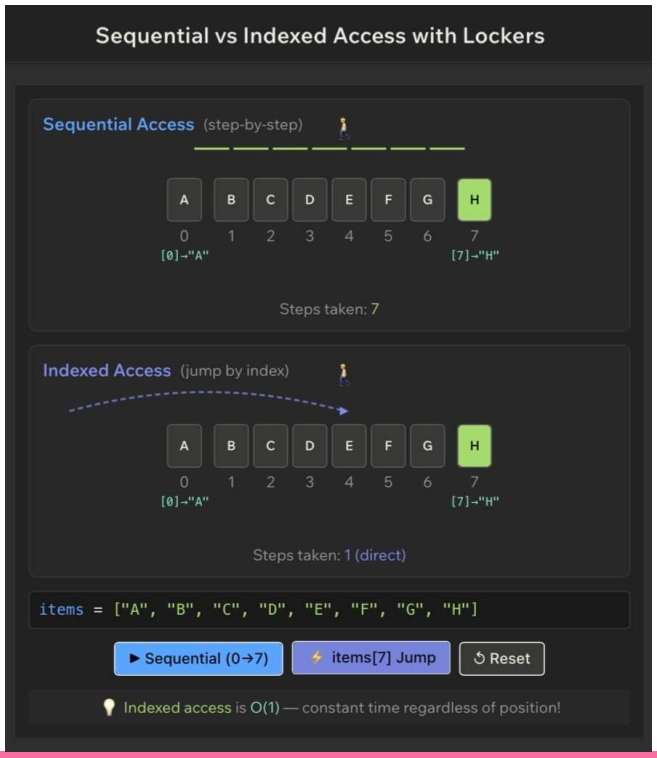




Made with Goodnotes

Lesson 3 - Study Check





Key Takeaways

- Indexes act like addresses.
- Indexed access allows direct retrieval.
- Sequential access examines items one at a time.
- Python uses zero-based indexing.
- Direct access becomes more valuable as collections grow.
- Understanding indexes is foundational for arrays and lists.

Notes

_____

_____

_____

_____

_____

_____

Terms and Definitions

- Index: a numerical position within a collection, such as the 3 initems[3].
- Indexed access: using an index to retrieve information directly.
- Sequential access: checking items one at a time until the desireditem is found.
- Position: the location of an item inside a collection.

Common Mistakes and Helpful Tips

Mistake: assuming the first position is 1.
Reality: the first position is 0.
Mistake: confusing the value with the index. colors[1] does not mean "blue is 1"; it means retrieve the value stored at position 1.
Mistake: accessing an index that does not exist, like colors[5] in a three-item collection.
Tip: draw indexes above values whenever you feel unsure.

Made with Goodnotes

- Zero-based indexing: a numbering system where positions beginat 0 instead of 1.

Index	0	1	2	3
Value	A	B	C	D

Review Checkpoint

- What is an index?
- How does indexed access differ from sequential access?
- Why are indexes useful?
- What is zero-based indexing?
- What value would print from items = ["A", "B", "C", "D"]; print(items[2])?
- Why does direct access become more valuable as collections grow?

Made with Goodnotes

Lesson 4 - Understanding Arrays

Big Idea

Arrays store information in an organized sequence of positions. Each position has an index, which means a program can retrieve a value directly when it knows where that value lives.

Full Teaching Explanation

Lesson 4 connects two earlier ideas: organization and indexing. An array is not just a loose pile of values. It is a structure where values live in predictable positions.
A movie theater is the perfect mental model. If your ticket says Row B, Seat 3, you do not inspect every seat until you find yours. The row and seat number tell you exactly where to go. In arrays, the index plays that same role.
Arrays are especially useful when order matters, positions matter, fast access is needed, or similar kinds of data are stored together. Student grades, product prices, daily temperatures, survey responses, game scores, and roster names are all natural array-style examples.

Movie Theater Analogy

Theater seat number = array index

Person sitting there = value

Theater layout = array structure

Finding Seat 4 = accessing index 4

Why Arrays Matter

- They preserve order.
- They store related values together.
- They support direct access by index.
- They appear everywhere: e-commerce, education, gaming, andweather systems.

Index	0	1	2	3	4
Value	A	B	C	D	E

Code Example - Accessing an Array Position

```
letters = ["A", "B", "C", "D",
"E"] print(letters[3]) # Output:
# D
```

Why This Works

The array stores information in specific positions. D is stored at index 3, so Python retrieves the value at position 3. It does not search for D; it goes directly to the location.

Lesson 4 - Arrays in Practice

Code Example - Student Scores

```
scores = [85, 92, 78, 100,
88] print(scores[1]) #
Output:
# 92
```

Programmer Thinking

When I look at an array, I do not just think, "This is a list of values." I think, "This is a collection of positions. Each position contains a value, and the index tells me where that value lives."

Terms and Definitions

Common Mistakes and Helpful Tips

Made with Goodnotes

- Array: a collection of values stored in an ordered sequence.
- Element: an individual value stored inside an array.
- Index: the position of an element.
- Ordered collection: a collection where positions matter and orderis preserved.
- Direct access: retrieving a value immediately by using its index.

Mistake: thinking arrays store information randomly.
Reality: arrays store values in a specific order.
Mistake: forgetting indexes begin at zero.
Reality: the first element is array[0].
Mistake: confusing element number with index number. The first element is at index 0, not index 1.
Tip: draw the index row above the value row.

Review Checkpoint

- What is an array?
- How is a movie theater similar to an array?
- What is an element?
- What is an index?
- Why are arrays considered ordered collections?
- What value is returned by items = ["A", "B", "C", "D"]; print(items[3])?
- Why is direct access useful?

Key Takeaways

- Arrays store information in ordered positions.
- Each position has an index.
- Indexes allow direct access.
- Arrays preserve order.
- Elements are individual values stored in the array.
- Arrays are foundational data structures used throughout programming.

Notes

_____

_____

_____

_____

_____

Lesson 5 - Accessing Elements in Arrays

Big Idea

Once information is stored in an array, programmers need a way to retrieve specific values. Accessing elements means using an index

Full Teaching Explanation

An array is useful because the information inside it can be accessed. If someone gives you hotel room 214, you know exactly where to g
array, the index works the same way.
The access pattern is simple: identify the position needed, use the index, then retrieve the value. This is fast and predictable as long as
That last part matters. If you ask for an index outside the valid range, Python cannot retrieve a value because there is no value stored t

Made with Goodnotes

Index	0	1	2	3
Value	red	blue	green	yellow

Array Access Pattern

Step 1: Identify the position needed.

Step 2: Use the index.

Step 3: Retrieve the value.

Example: colors[2] retrieves green.

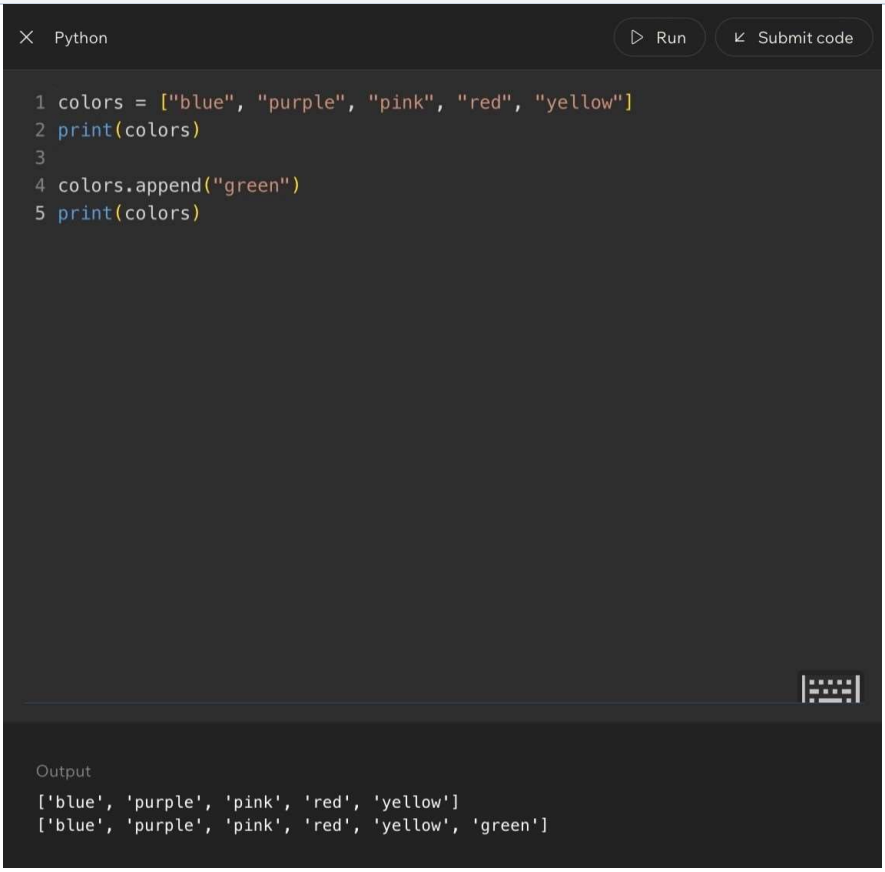
Code Example - Basic Access

```
colors = ["red", "blue", "green", "yellow"] print(colors[2]) # Output:
# green
```

Code Example - First and Last Elements

```
animals = ["dog", "cat", "bird", "fish"] print(animals[0]) print(animals[3]) # Output:
# dog
# fish
```

Code Example:



Lesson 5 - Adding, Inserting, Extending, and Index Errors

Operation	What It Does	Mental Model
append(value)	Adds one element to the end.	Put this new item at the next available position.
insert(index, value)	Places a value at a specific position.	Put it here, then shift later items right.

Made with Goodnotes

extend(other)	Adds every element from another collection.	Combine these collections into one.

append() Example

```
colors = ["red", "blue"]
colors.append("green")
print(colors)
# ['red', 'blue', 'green']
```

insert() Example

```
nums = [10, 3 , 40]
nums.insert(1, 20)
print(nums)
# [10, 2 , 3 , 40]
```

extend() Example

```
nums = [1, 2, 3]
more_nums = [4, 5, 6]
nums.extend(more_nums)
print(nums)
# [1, 2, 3, 4, 5, 6]
```

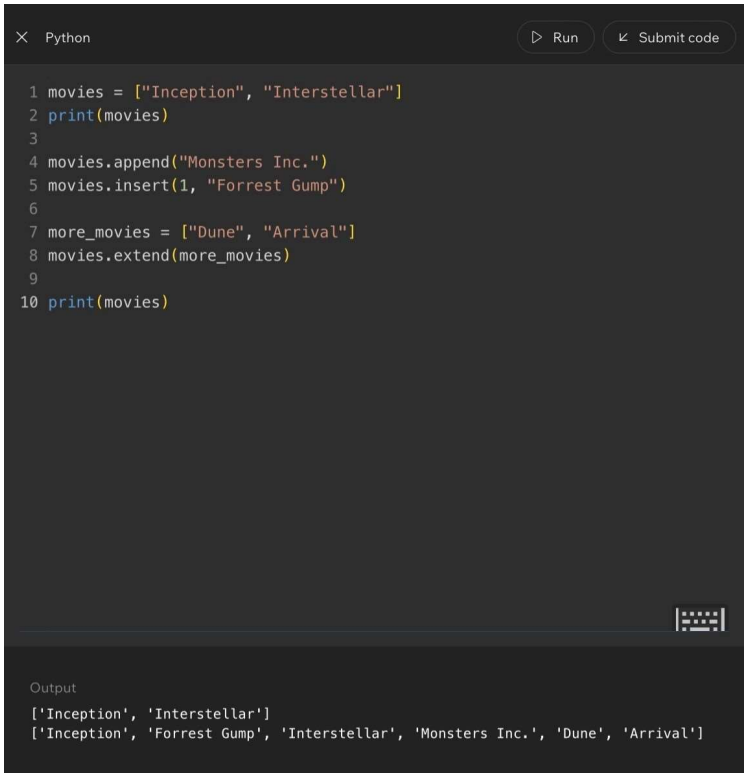
Invalid Indexes and Errors

A valid index must exist inside the collection. If colors has indexes 0, 1, and 2, then colors[3], colors[4], and colors[5] are invalid.

Example: colors = ["red", "blue", "green"]; print(colors[5]) causes IndexError because position 5 does not exist.

Code Example:





Lesson 5 - Study Check

Terms and Definitions | **Common Mistakes and Helpful Tips**

Made with Goodnotes

- Access: retrieving information from a collection.
- append(): adds an element to the end of a collection.
- insert(): places an element at a specific position.
- extend(): combines elements from another collection.
- IndexError: an error caused by requesting an invalid position.
- Valid index: an index that exists within the collection.

Mistake: assuming indexes start at 1.
Reality: indexes start at 0.
Mistake: using an index larger than the collection size.
Reality: only existing positions can be accessed.
Mistake: confusing append() and insert(). append() adds to the end; insert() places a value at a specific location.
Tip: when debugging index problems, write indexes above the values.

Review Checkpoint

- What does an index represent?
- How does append() work?
- How does insert() differ from append()?
- What does extend() do?
- What causes an IndexError?
- What prints after nums = [10, 20]; nums.append(30); print(nums)?
- What prints after nums = [10, 30]; nums.insert(1, 20); print(nums)?

Key Takeaways

- Arrays use indexes for direct access.
- append() adds to the end.
- insert() adds at a specific position and shifts later values.
- extend() combines collections.
- Invalid positions cause IndexError.
- Understanding indexes is critical for working with collections.

Notes

_____

_____

_____

_____

_____

_____

_____

Lesson 6 - Updating and Replacing Values in Arrays

Big Idea

Arrays are not just for storing and reading data. Real programs constantly update information: grades change, inventory counts drop, user profiles get edited, and seat assignments become occupied. Updating means replacing the value stored at an existing index.

Full Teaching Explanation

Made with Goodnotes

The key idea is that the location stays the same, but the value changes. In a gradebook, Student 3 may stay in the same row, but their score can change from 78 to 82 after a correction. Arrays work the same way: the index remains the address, and the value stored there is replaced.
This is different from append() or insert(). append() adds something new to the end. insert() places something new at a specific position and shifts other values. Updating does not create a new position. It overwrites the value already stored at that index.
This matters because software would become outdated quickly if it could only display old values. Banking balances, product inventory, grades, social media profiles, and game stats all depend on updates.

Gradebook Mental Model

Before update: Student 3 has 78.

After correction: Student 3 has 82.

The student did not move. Only the score stored in that position changed.

	Index 0	Index 1	Index 2	Index 3
Before	85	90	78	88
After	85	90	82	88

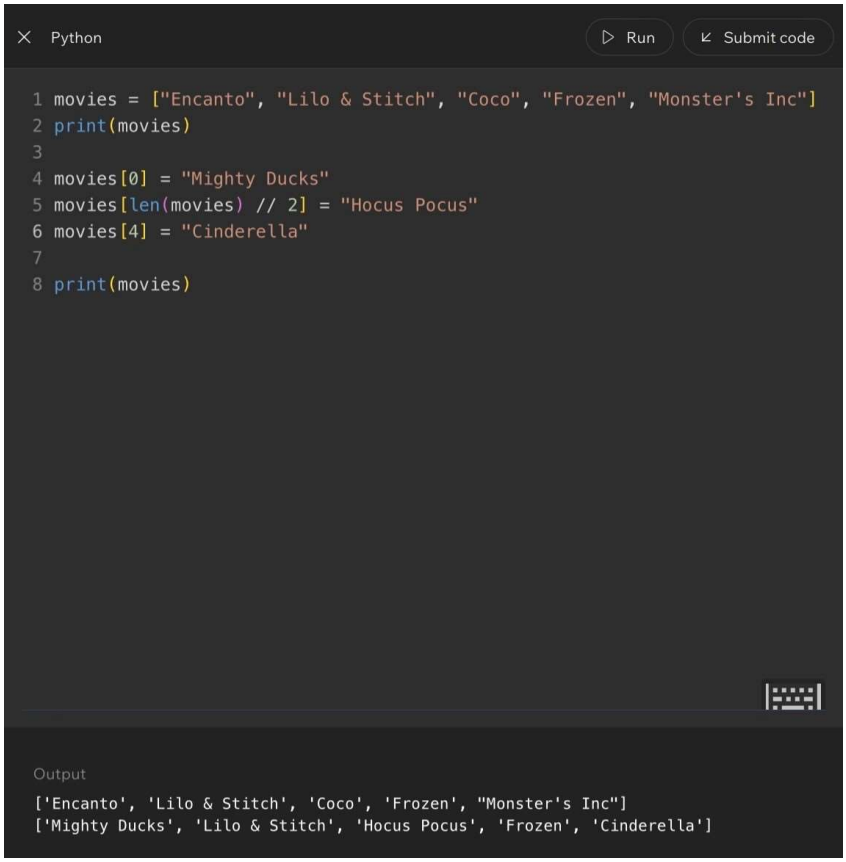
Code Example - Updating a Value

```
scores = [85, 90, 78,
88] scores[2] = 82
print(scores) # Output:
# [85, 90, 82, 88]
```

What Happened?

scores[2] = 82 means: go to index 2 and replace the current value with 82. Python does not create a new position; it overwrites the existing value.

Code Example:



Lesson 6 - First, Middle, Last, and Update Thinking

Code Example - Updating a Name

Made with Goodnotes

```
students = ["Sophia", "CJ",
"Caleb"] students[1] =
"Christopher" print(students) #
Output:
# ['Sophia', 'Christopher', 'Caleb']
```

Common Positions

First element: always index 0.

Last element: depends on the length. In a 4-item array, the last index is 3.
Middle element: for [10, 20, 30, 40, 50], the middle value is 30 at index 2.

Position	Index	Value
First	0	10
Middle	2	30
Last	4	50

Programmer Thinking

When I look at an array, I ask: am I trying to add something, replace something, or remove something? If I am replacing information, I do not need append() or insert(). I already know where the information belongs. I simply update the value at that location.

Code Example - Updating Inventory

```
inventory = [12, 8,
15] inventory[1] = 6
print(inventory) #
Output:
# [12, 6, 15]
```

System	What Changes
Banking	Account balances change after deposits and withdrawals.
E-commerce	Inventory counts change when products are sold.
Education	Grades change when assignments are entered.
Social Media	Profile information changes when users edit accounts.

Made with Goodnotes

Code Example:

Python Run Submit code

```
1 scores = [50, 60, 70, 80, 90]
2 print(scores)
3
4 scores[0] = 100
5 scores[len(scores) // 2] = 0
6 scores[len(scores) - 1] = 999
7
8 print(scores)
```

Output

```
[50, 60, 70, 80, 90]
[100, 60, 0, 80, 999]
```

Lesson 6 - Study Check

Terms and Definitions

- Update: changing an existing value.
- Replace: substituting one value for another.
- Assignment: storing a value in a variable or location, such asscores[2] = 82.
- Existing element: a value already stored in a collection.
- Position: the location of a value within an array.

Common Mistakes and Helpful Tips

Mistake: using append() when you mean to replace a value.
Reality: append() adds a new element; updating changes an existing one.
Mistake: forgetting which index needs to be updated.
Reality: draw the indexes first.
Mistake: confusing value with position.
Reality: the index identifies the location; the value is the information stored there.
Tip: before updating, ask: which element changes, what index contains it, and what should the new value be?

Review Checkpoint

- What does it mean to update an array?
- How is updating different from appending?
- What happens when you assign a new value to an existing index?
- Where is the first element located?
- How can you identify the middle element?
- What prints after nums = [10, 20, 30]; nums[1] = 99; print(nums)?
- Why are updates important in real-world software?

Key Takeaways

Made with Goodnotes

- Arrays can store changing information.
- Updating replaces an existing value.
- The index stays the same while the stored value changes.
- First, middle, and last positions are common access patterns.
- Updating data is one of the most common operations in software development.

Notes

_____

_____

_____

_____

_____

Lesson 7 - Dynamic Collections and Why Arrays Aren't Always Enough

Big Idea

Arrays are powerful because they store values in predictable positions, but real-world applications rarely stay the same size forever. Dynamic collections provide flexibility when data needs to grow, shrink, or reorganize over time.

Full Teaching Explanation

Traditional arrays were designed with fixed sizes. Once created, they could not easily grow or shrink. If more space was needed, programmers often had to create a new array and copy the existing information into it.
Modern languages give us flexible structures that can expand automatically, but the underlying idea still matters. Every data structure makes tradeoffs. Some prioritize speed. Some prioritize flexibility. Some prioritize memory efficiency.
The structure that works well for ten items may become a poor choice for ten million items. This lesson trains the habit of asking whether a collection will stay roughly the same size or keep changing.

Bookshelf Mental Model

A shelf that holds exactly 20 books works fine until you buy 10 more books.
Option 1: buy a larger shelf and move everything.

Option 2: add another shelf somewhere else.

Neither is perfect; both involve work. Data structures face similar growth decisions.

Fixed Collection	Dynamic Collection
Capacity is limited	Can grow or shrink
May require rebuilding	Adapts as data changes
Predictable size	More flexible
Good when size is known	Good when size changes

Code Example - Growing a Collection

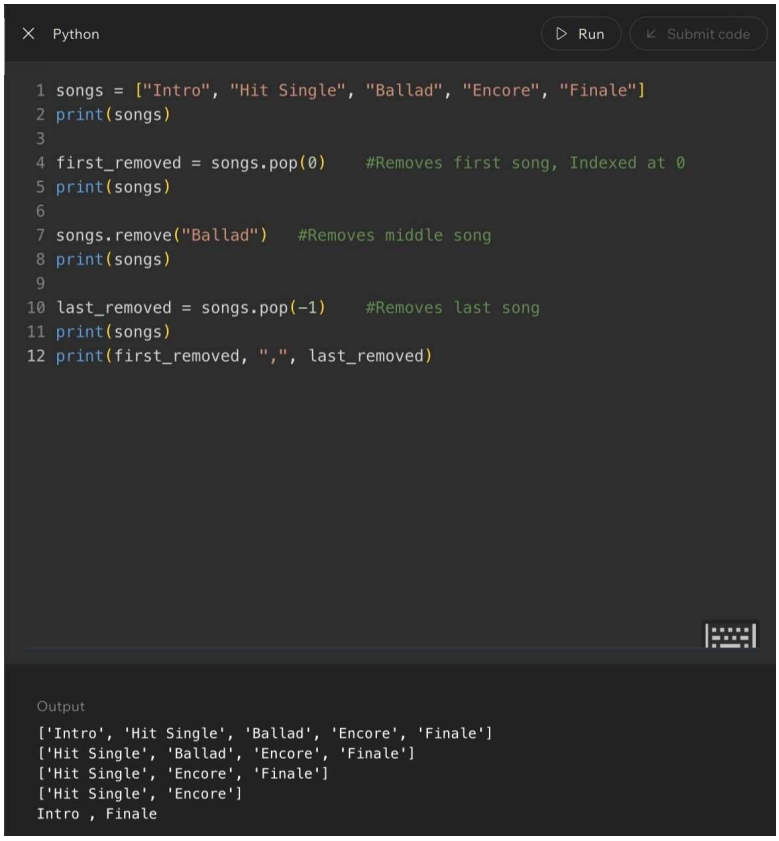
```
colors = ["red", "blue",
"green"] colors.append("yellow")
colors.append("purple")
print(colors) # Output:
# ['red', 'blue', 'green', 'yellow', 'purple']
```

Why This Matters

Spotify playlists, Amazon product catalogs, friend lists, and customer databases all grow over time. If every new item required rebuilding the whole collection, performance would suffer dramatically.

Made with Goodnotes

Code Example:



Lesson 7 - Study Check

Programmer Thinking

When designing a system, ask: will this collection stay roughly the same size, or will it grow constantly? If growth is expected, flexibility becomes an important design consideration.

Terms and Definitions

- Dynamic collection: a collection capable of growing and shrinkingas needed.
- Fixed size: a structure whose capacity cannot easily change.
- Capacity: the amount of information a structure can hold.
- Resizing: expanding or shrinking a collection.
- Tradeoff: improving one characteristic may require sacrificinganother.

Common Mistakes and Helpful Tips

Mistake: assuming bigger is always better.
Reality: different structures solve different problems.

Mistake: thinking all collections grow the same way.
Reality: growth strategies vary between structures.
Tip: whenever learning a new structure, ask how data is stored, how data is retrieved, how it handles growth, and what strengths/weaknesses it has.

Review Checkpoint

- Why might a fixed-size structure become problematic?
- What is a dynamic collection?
- What does capacity mean?
- Why are tradeoffs important?
- How do growing collections differ from fixed collections?

Key Takeaways

Made with Goodnotes

• Arrays are useful but not perfect.

• Some structures must adapt to growth.

• Dynamic collections provide flexibility.

• Every data structure involves tradeoffs.

• Choosing the right structure depends on the problem.

Notes

_____

_____

_____

_____

_____

_____

_____

Lesson 8 - Adding and Removing Data Efficiently

Big Idea

Adding and removing data changes a collection's organization. Adding to the end is often simple, but inserting into the middle or removing from the middle can force other elements to shift positions.

Full Teaching Explanation

Imagine a line at a movie theater: A B C D E. If someone needs to be inserted between B and C, the line becomes A B X C D E. Everyone after B shifts one position. Indexed collections often behave the same way.
Removing information creates a similar challenge because the collection must stay organized after the removal. The empty space needs to be closed, which usually means later values shift left.
This movement may feel small with five items, but it becomes much more expensive in collections with millions of elements. Understanding what moves helps explain why some operations are faster than others.

Operation	What Happens	Efficiency Concern
Add to end	New item goes after the last element.	Usually simple.
Insert in middle	Later elements shift right.	Can require moving many elements.
Remove from middle	Later elements shift left.	Indexes may change after removal.

Before Insert

Index: 0 1 2 3
Value: A B C D

Insert X at Index 2

Index: 0 1 2 3 4
Value: A B X C D

C and D moved right.

Code Example - Insert

```
letters = ["A", "B", "C",
"D"] letters.insert(2, "X")
print(letters) # Output:
# ['A', 'B', 'X', 'C', 'D']
```

Lesson 8 - Remove, Shift, and Study Check

Code Example - Remove

Made with Goodnotes

```
letters = ["A", "B", "C",
"D"] letters.remove("C")
print(letters) # Output:
# ['A', 'B', 'D']
```

Programmer Thinking

When I insert something into the middle of a collection, I immediately ask: what else has to move? That question helps explain why some operations are more expensive than others.

Terms and Definitions

- Insert: adding a value at a specific position.• Remove: deleting a value from a collection.
- Shift: moving elements to new positions.
- Modification: any change made to a collection.
- Reorganization: adjusting positions after changes occur.

Tip: draw the collection before and after modifications to visualize movement.

Mistake: assuming insert only affects one position. Reality: many elements may need to move.

Mistake: forgetting indexes change after removals. Reality: positions often shift after data is removed.

Key Takeaways

- Adding and removing data affects organization.
- Inserts often cause shifting.
- Removals often change indexes.
- Collection size influences efficiency.
- Understanding movement helps explain performance differences.

Notes

_____

_____

_____

_____

_____

_____

Review Checkpoint

- What happens when an element is inserted in the middle?
- Why might inserts require shifting?
- What happens when elements are removed?
- Why do indexes change after removals?
- How does collection size affect modification costs?

Made with Goodnotes

Lesson 9 - Searching Through Collections

Big Idea

Searching is how programmers locate specific information inside a collection. Once data is stored, the next question is: how do we find the one item we need quickly and accurately?

Full Explanation

Organization and searching work together. A messy pile of information is harder to search than a structured collection. Search algorithms give programs a systematic process for checking items, identifying matches, and deciding what to do when the target is found or missing.

In this lesson, the important shift is from “I have data” to “I need a repeatable way to find something inside that data.” The structure affects how much work the search requires.

Real-World Examples

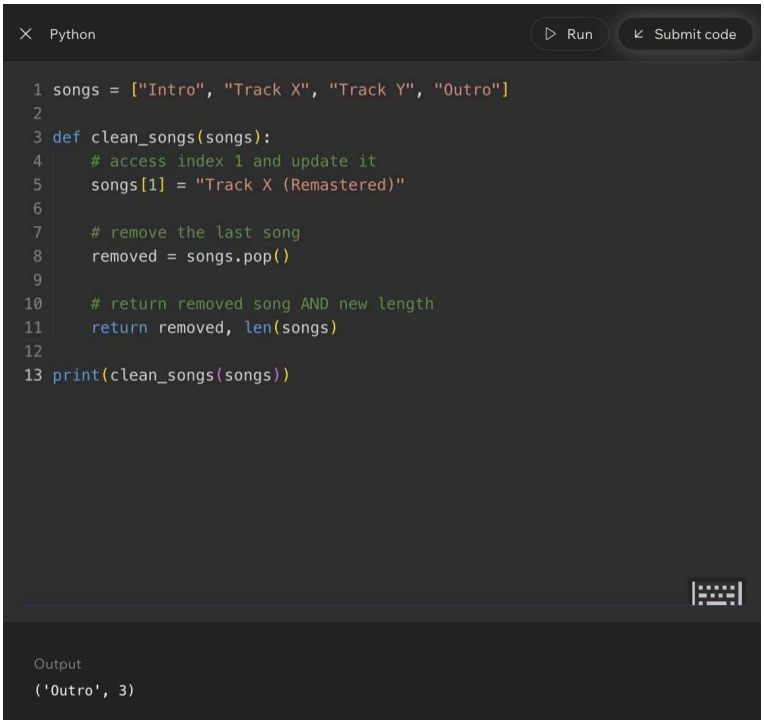
- Class roster: If the roster is random, you may read one name at a time. If it is alphabetized, you can search more efficiently.
- Playlist: Looking through 5,000 songs manually is slow. Typing the song title lets the app locate it much faster.
- Photography files: Finding one rider image is easier when photos are organized by event, class, and rider instead of dumpedinto one folder.

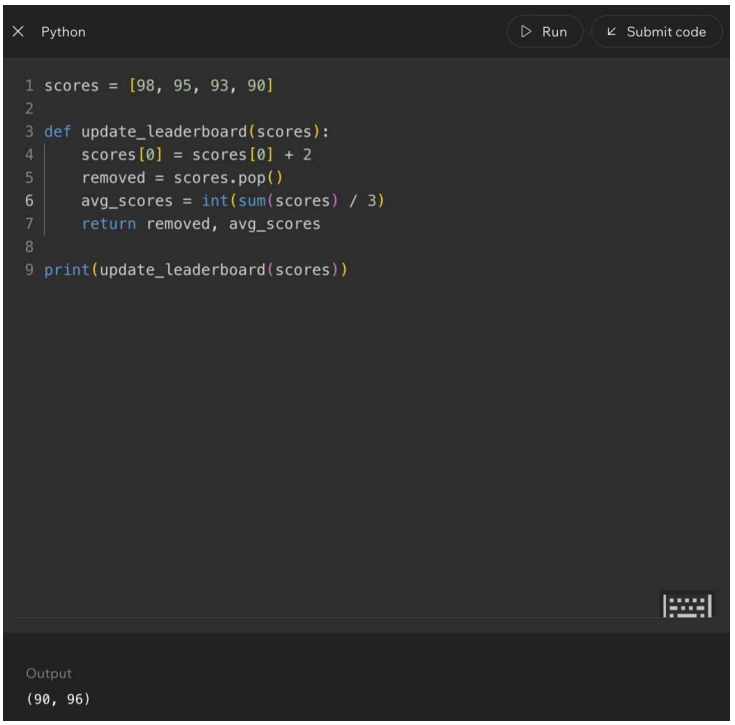
Made with Goodnotes

Search Pattern

Step	Question
1. Start	Choose where the search begins.
2. Check item	Compare the current item to the target.
3. Match?	If yes, stop. If no, continue.
4. Finish	Return success or report that the item was not found.

Code Example:





Lesson 9 - Searching Through Collections

Code Example - Simple Search

```
colors = ["red", "blue", "green", "yellow"]
target = "green"

for color in colors:
    if color == target:
        print("Found!")
```

Plain English: Look at each color. If the current color equals the target, print that it was found.

Terms and Definitions

Search: The process of locating specific information.
Search Algorithm: A set of steps used to find information.
Target: The value being searched for, such as `target = "green"` .
Match: When the current item equals the target.
Collection: A group of related data stored together.

Made with Goodnotes

Common Mistakes and Helpful Tips

- Mistake: Thinking searching only exists in programming. Reality: people search constantly in everyday life.
- Mistake: Ignoring how the information is organized. Reality: organization often determines search efficiency.
- Tip: Ask what you are searching for, how the data is organized, how many items exist, and what happens if the item is notfound.

Key Takeaways

- Searching locates information inside collections.
- Search algorithms provide systematic steps.
- Organization affects search efficiency.
- Data structures and algorithms work together.

Notes

Lesson 10 - Introduction to Algorithms

Big Idea

Algorithms are step-by-step procedures used to solve problems. Data structures organize information; algorithms process that information.

Full Explanation

An algorithm does not have to be code. A recipe, GPS directions, a morning routine, or instructions for assembling furniture are all algorithms because they are ordered, repeatable steps designed to reach a goal.

In computer science, algorithms tell software what actions to perform: search, sort, calculate, process, recommend, or decide. Good algorithms reduce confusion and unnecessary work.

Everyday Algorithm Examples

Example	Algorithm Thinking
Recipe	Follow ordered steps to create food.
GPS directions	Take each turn in sequence to reach a destination.
Morning routine	Repeat a familiar process each day.
Search feature	Follow steps to locate a target value.

Problem -> Algorithm -> Solution

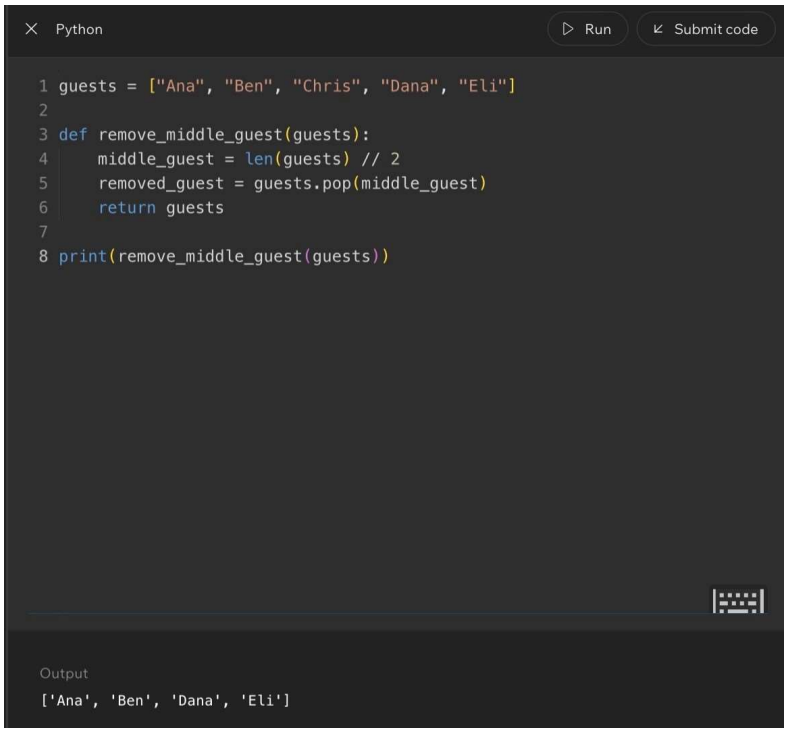
Problem: Need to find a book.

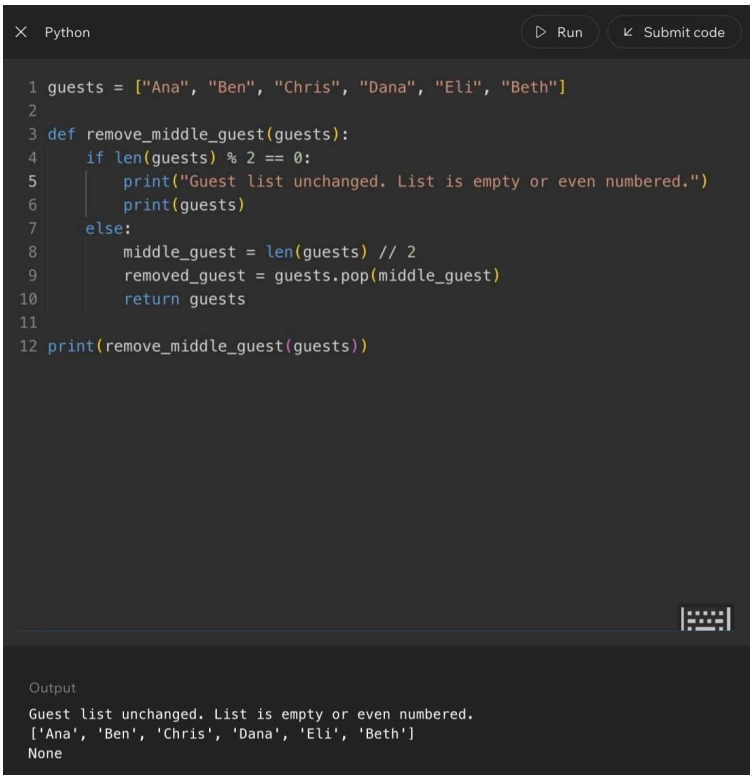
Algorithm: Go to shelf -> check title -> match? -> continue if necessary.

Solution: Book found, or search ends with no match.

Made with Goodnotes

Code Example:





Lesson 10 - Introduction to Algorithms

Code Example - Repeating a Process

```
numbers = [4, 8, 15, 16, 23, 42]

for number in numbers:
    print(number)
```

What this algorithm does: Starts with the first number, prints it, moves to the next number, and repeats until the collection is finished.

Think Like a Programmer

When you hear “algorithm,” do not jump straight to complicated math. Think: a repeatable set of instructions. Explain it in plain English first, then translate it into code.

Terms and Definitions

Algorithm: A sequence of steps used to solve a problem.

Procedure: A set of actions performed in order.

Step: One individual action inside an algorithm.

Process: A group of actions working toward a goal.

Logic: The reasoning behind decisions and actions.

Common Mistakes and Helpful Tips

- Mistake: Thinking algorithms are always code. Reality: algorithms exist before code is written.
- Mistake: Assuming algorithms must be complicated. Reality: even simple instructions can be algorithms.
- Tip: If you can explain the steps to another person, you are more likely to understand the code version.

Made with Goodnotes

Key Takeaways

- Algorithms solve problems using ordered steps.
- Algorithms exist inside and outside programming.
- Data structures organize information; algorithms process it.
- A clear plain-English algorithm makes the code easier to write.

Notes

Made with Goodnotes

Lesson 11 - Linear Search

Big Idea

Linear Search is one of the simplest search algorithms. It checks items one at a time until it finds the target or reaches the end of the collection.

Full Explanation

Linear Search combines collections and algorithms. The collection holds the data. The algorithm tells the program how to move through that data. It begins at the first element, compares it to the target, and continues forward until there is a match or nothing is left to check.

It is simple and reliable because it works even when the collection is not sorted. The tradeoff is that large collections may require many comparisons.

Real-World Examples

- Looking for a name on an unsorted roster: read one name at a time until Sophia appears.
- Looking through mail: check each envelope until the electric bill appears.
- Searching a drawer: check each shirt until you find the one you wanted.

Linear Search Process

Collection	Target	Checks
A B C D E F G	F	A -> B -> C -> D -> E -> F -> Found
A B C D E F G	G	A -> B -> C -> D -> E -> F -> G -> Found
A B C D E F G	Z	Check every item -> Not found

Made with Goodnotes

Lesson 11 - Linear Search

Code Example - Basic Linear Search

```
colors = ["red", "blue", "green", "yellow"]
target = "green"

for color in colors:
    if color == target:
        print("Found!")
```

Plain English: Check red, then blue, then green. Green matches the target, so the search succeeds.

Code Example - Search Failure

```
numbers = [10, 20, 30, 40, 50]
target = 100
found = False

for number in numbers:
    if number == target:
        found = True

print(found)
```

Output: False. The algorithm checked every element and never found the target.

Worst-Case Scenario

The worst case happens when the target is last or does not exist. In that situation, Linear Search may have to check every element in the collection.

Made with Goodnotes

Lesson 11 - Linear Search

Out-Loud Programmer Thinking

Linear Search means: start at the beginning and check everything until you find what you need. It is not fancy, and it is not always fast, but it is reliable and works on unsorted data.

Terms and Definitions

Linear Search: A search algorithm that checks elements one at a time.
Target: The value being searched for.
Match: When the current element equals the target.
Search Failure: When the target does not exist in the collection.
Sequential Access: Examining items one after another.

Common Mistakes and Helpful Tips

- Mistake: Assuming the search always finds something. Reality: the target may not exist.
- Mistake: Forgetting to stop after finding a match. Reality: many searches can end immediately after success.
- Mistake: Assuming Linear Search is always fast. Reality: large collections may require many comparisons.
- Tip: Physically trace the search path on paper. Watching each comparison makes the algorithm easier to understand.

Review Checkpoint

1. What is Linear Search?
2. How does it work?
3. What happens if the target is not found?
4. Why is Linear Search considered sequential?
5. What is the worst-case scenario for Linear Search?

Key Takeaways

- Linear Search examines items one at a time.
- It starts at the beginning of the collection.
- It stops when the target is found or the collection ends.
- It works on unsorted collections.
- Simplicity is one of its biggest strengths.

Notes

Made with Goodnotes

Lesson 12 - Comparing Search Strategies

Big Idea

Not all search methods require the same amount of work. Two algorithms can both produce the correct answer, but one may take dramatically longer than the other.

Full Explanation

This lesson separates two ideas that beginners often combine: correctness and efficiency. A correct algorithm gives the right answer. An efficient algorithm gives the right answer while doing less unnecessary work.

As collections grow, the difference matters more. A search strategy that feels fine with 10 items may become frustratingly slow with 10 million items. Strong programmers compare solutions instead of stopping at the first one that works.

Real-World Example - Dictionary Search

- Student A starts at page 1 and reads forward until the word appears.
- Student B opens near the middle, decides whether the word comes before or after that point, and narrows the search.
- Both can be correct. Student B usually does much less work.

Made with Goodnotes

Correctness vs. Efficiency

Question	What It Tells You
Does it work?	Correctness: the solution produces the right answer.
How much work does it require?	Efficiency: the solution reaches the answer with less wasted effort.
What happens when the data grows?	Scalability: the solution still performs well with larger collections.

Code Example:

Python

Run Submit code

```python
garage = [
    [  # floor 0
        ["empty", "empty", "empty"],  # row 0
        ["empty", "empty", "empty"]   # row 1
    ],
    [  # floor 1
        ["empty", "empty", "empty"],  # row 0
        ["empty", "empty", "empty"]   # row 1
    ]
]

print("Garage Floor 1:")
print("Row 0:", garage[1][0])
print("Row 1:", garage[1][1])

garage[1][0][2] = "ABC123"
print("Garage Floor 1 (Updated):")
print("Row 0:", garage[1][0])
print("Row 1:", garage[1][1])

print("Garage Floor 0:")
print("Row 0:", garage[0][0])
print("Row 1:", garage[0][1])

garage[0][1][0] = "XYZ999"
print("Garage Floor 0 (Updated):")
print("Row 0:", garage[0][0])
print("Row 1:", garage[0][1])
```

Output

```
Garage Floor 1:
Row 0: ['empty', 'empty', 'empty']
Row 1: ['empty', 'empty', 'empty']
Garage Floor 1 (Updated):
Row 0: ['empty', 'empty', 'ABC123']
Row 1: ['empty', 'empty', 'empty']
Garage Floor 0:
Row 0: ['empty', 'empty', 'empty']
Row 1: ['empty', 'empty', 'empty']
Garage Floor 0 (Updated):
Row 0: ['empty', 'empty', 'empty']
Row 1: ['XYZ999', 'empty', 'empty']
```

Lesson 12 - Comparing Search Strategies

Code Example - Correct, But Still Sequential

```
numbers = [1, 2, 3, 4, 5]
target = 5

for number in numbers:
    if number == target:
        print("Found")
```

Plain English: This works, but it still checks items one at a time. With a tiny list, that is fine. With a huge list, the amount of work matters.

Made with Goodnotes

Why This Matters as Data Grows

Collection Size	What Happens
10 items	Linear search usually feels fast.
1,000 items	The extra work becomes noticeable.
1,000,000 items	A poor strategy may require enormous work.

Terms and Definitions

Efficiency: The amount of work required to solve a problem.
Correctness: Producing the right answer.
Comparison: Evaluating different approaches.
Search Strategy: The method used to locate information.
Scalability: How well performance holds up as data grows.

Common Mistakes and Helpful Tips

- Mistake: Thinking the first working solution is automatically the best solution.
- Mistake: Ignoring efficiency because the small example runs quickly.
- Tip: Ask whether it works, whether it can be improved, and what happens if the data becomes much larger.

Key Takeaways

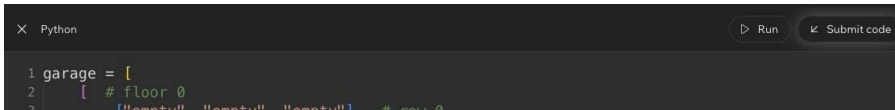
- Correctness and efficiency are different concepts.
- Two algorithms may produce the same answer.
- Some algorithms require more work than others.
- Efficiency becomes more important as data grows.
- Comparing solutions is a key software engineering skill.

Notes

Made with Goodnotes

Made with Goodnotes

Code Example:



Lesson 13 - Organizing Information Efficiently

Big Idea

This lesson ties together the week's major theme: information should be organized so it can be found, updated, processed, and maintained efficiently.

Full Explanation

Collections, indexes, arrays, searching, and algorithms all connect through one central question: how should information be organized so it can be used efficiently? Good organization reduces work. Poor organization creates extra work. Software often manages users, products, transactions, messages, images, documents, and locations. The better that information is organized, the easier it becomes to retrieve and process it.

Real-World Connection - Photography Files

Imagine storing every photo you have ever taken in one giant folder. After a few years, that folder could contain tens of thousands of images.

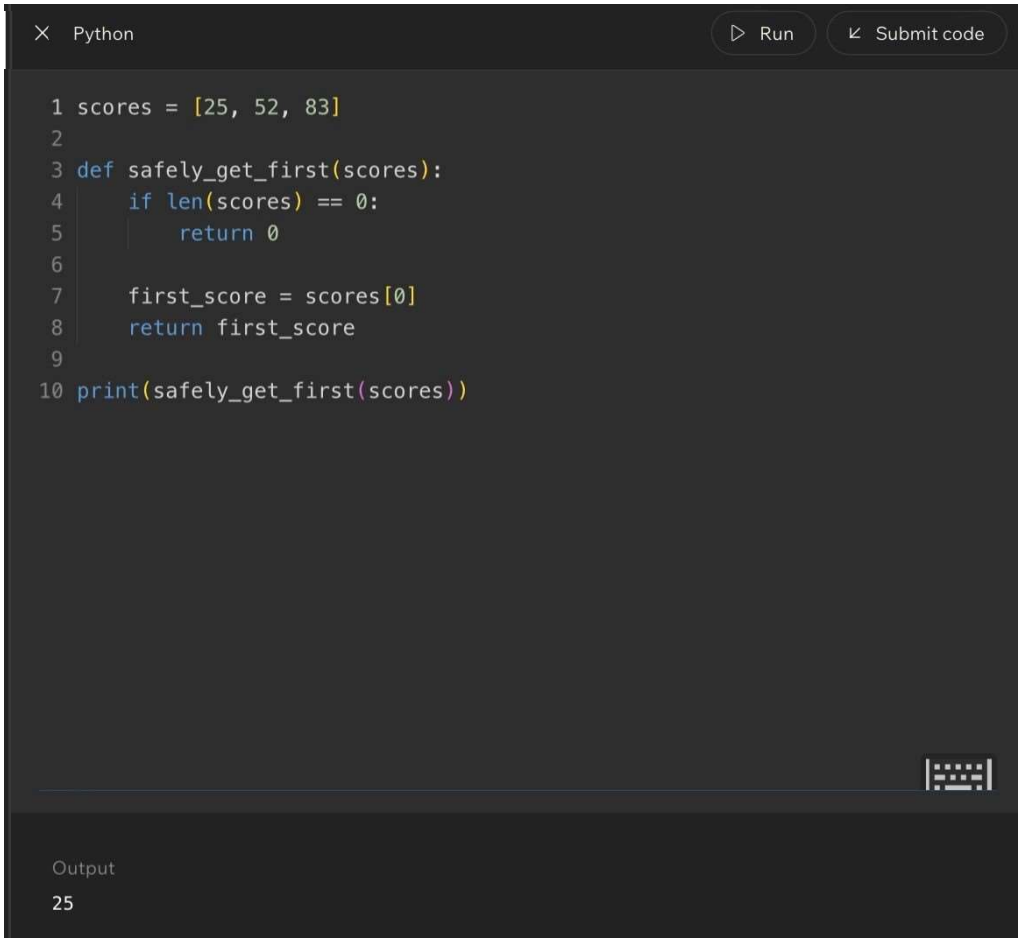
- Poor organization: every photo, receipt, video, and document mixed together.
- Better organization: folders by year, event, client, class, rider, or location.
- The files are the same. The structure makes them usable.

Made with Goodnotes

Connecting the Core Ideas

Concept	Question It Answers
Data structures	How should I organize information?
Indexes	How do I quickly locate something when I know its position?
Searching	How do I find something when I do not know its exact position?
Algorithms	What steps should I follow to solve the problem?

Code Example:



Lesson 13 - Organizing Information Efficiently

Code Example - Concepts Working Together

```
students = ["Sophia", "CJ", "Caleb", "Emma", "Lucas"]
target = "Caleb"

for student in students:
    if student == target:
        print("Student Found")
```

What this combines: a collection, stored data, searching, and an algorithm. Week 1 concepts are not separate; they work together.

Made with Goodnotes

Why Organization Matters More as Data Grows

Collection Size	Organization Impact
Small: 10 items	Almost any organization can work.
Medium: 1,000 items	Good organization starts becoming important.
Large: 1,000,000 items	Good organization becomes critical.

Out-Loud Programmer Thinking

When building software, ask: What information am I storing? How will users need to access it? What structure makes that easiest? Those questions often matter more than the individual lines of code.

Terms and Definitions

Organization: Arranging information in a meaningful way.
Retrieval: Obtaining stored information.
Structure: The arrangement used to store information.
Scalability: The ability to continue performing well as data grows.
Maintainability: How easy a system is to understand and update.

Common Mistakes and Helpful Tips

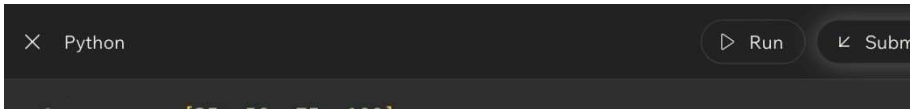
- Mistake: Thinking organization is just housekeeping. Reality: organization affects performance and usability.
- Mistake: Focusing only on code. Reality: data organization often matters more than individual lines of code.
- Tip: Ask what problem the structure solves, what makes it useful, what tradeoffs it involves, and when you would choose it.

Notes

Made with Goodnotes

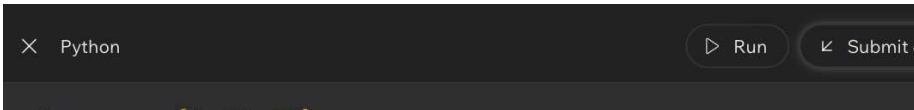
Made with Goodnotes

Code Example:



Made with Goodnotes

Code Example:



Lesson 14 - Week 1 Integration and Review

Big Idea

Week 1 introduced the foundation for CS102: information must be organized and processed efficiently. This lesson connects the whole week into one mental model.

Full Explanation

The week began with data structures and organization, then moved into indexes, arrays, access, updates, additions, removals, searching, and algorithms. The final pieces were efficiency and comparison: not every correct solution requires the same amount of work.

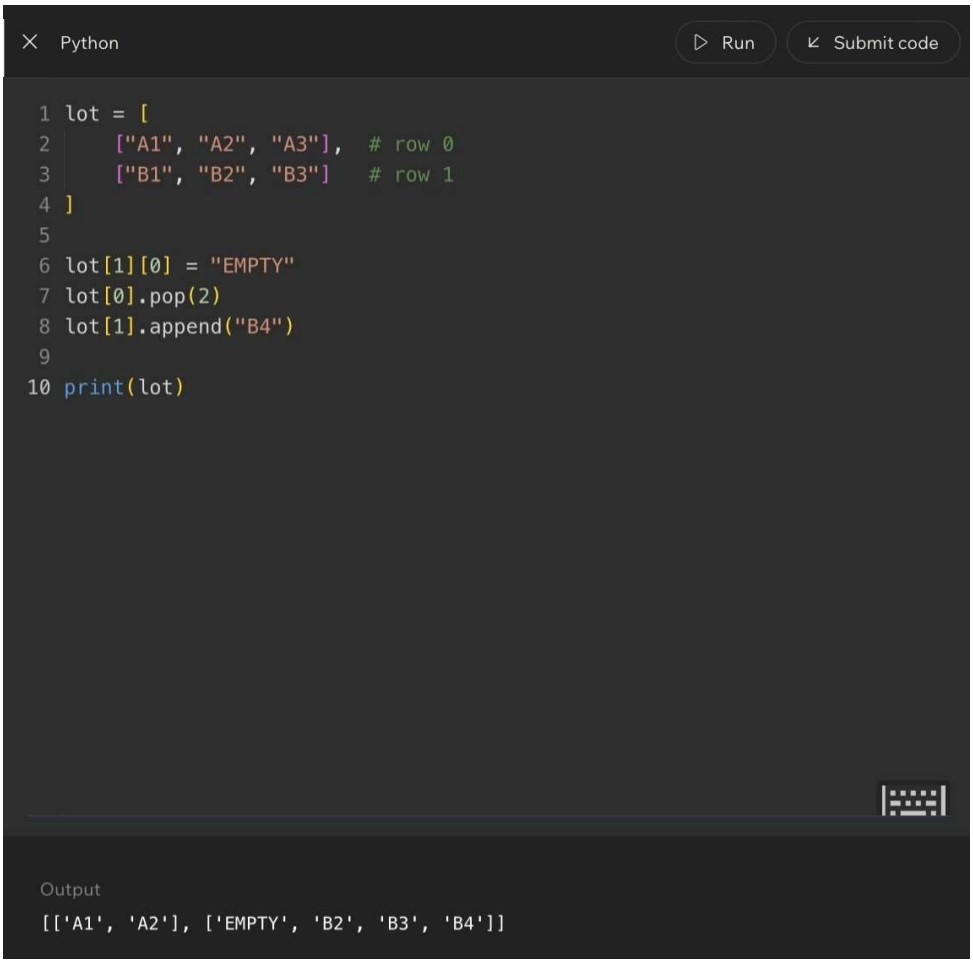
Together, these topics form the base for understanding more advanced data structures and algorithms later in the course.

Made with Goodnotes

Week 1 Concept Map

Flow	Meaning
Data Structures	Organize information.
Collections	Store related data together.
Indexes	Point to positions.
Arrays	Store values in ordered positions.
Access / Update / Modify	Use and change stored information.
Searching	Find information when location is unknown.
Algorithms	Follow repeatable steps.
Efficiency	Measure how much work is required.

Code Example:



Lesson 14 - Week 1 Integration and Review

Made with Goodnotes

How the Lessons Connect

Lesson	Main Focus
1	Course orientation and software engineering mindset
2	Data structures and organization
3	Indexed access
4	Arrays
5	Accessing elements
6	Updating values
7	Dynamic collections and tradeoffs
8	Adding and removing data
9	Searching
10	Algorithms
11	Linear Search
12	Comparing search strategies
13	Efficient organization
14	Integration and review

Real-World Example - Running a Business

- Store information: customers, products, orders, and employees.
- Organize information: databases, files, and records.
- Find information: customer accounts, inventory, and transactions.
- Process information: purchases, refunds, reports, and analytics.

Everything from Week 1 contributes to those tasks.

Out-Loud Programmer Thinking

Why do data structures exist? Because organization matters.
Why do algorithms exist? Because problems need repeatable solutions.
Why compare solutions? Because some solutions require less work than others.

Made with Goodnotes

Code Example:

Python Run Submit code

```python
storage = [
    [   # floor 0
        ["F0B0-item1", "F0B0-item2"],   # box 0
        ["F0B1-item1"]                  # box 1
    ],
    [   # floor 1
        ["F1B0-item1", "F1B0-item2"]    # box 0
    ]
]

storage[0][0].append("F0B0-item3")
storage[1].append(["F1B1-item1"])
storage[0][1].pop(0)

print(storage)


```

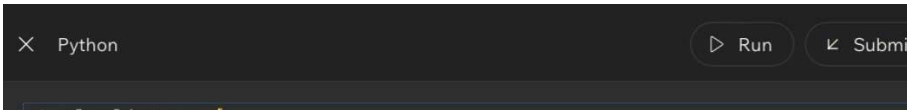
Output

```
[[['F0B0-item1', 'F0B0-item2', 'F0B0-item3'], []], [['F1B0-item1', 'F1B0-item2'], ['F1B1-item1']]]
```


Made with Goodnotes

Made with Goodnotes

Code Example:



Lesson 14 - Week 1 Integration and Review

Week 1 Master Summary

Concept	What to Remember
Data Structures	Organize information.
Indexes	Provide direct access.
Arrays	Store ordered collections.
Searching	Locates information.
Algorithms	Provide repeatable procedures.
Efficiency	Measures the work required.
Scalability	Describes how performance changes as data grows.

Common Week 1 Misconceptions

- Data structures are just lists. Reality: lists are only one example.
- Algorithms must be complicated. Reality: even simple instructions can be algorithms.
- If a solution works, it must be good. Reality: efficiency matters too.
- Indexes begin at 1. Reality: Python uses zero-based indexing.

Final Review Questions

1. What is a data structure?
2. What is an algorithm?
3. Why are indexes useful?
4. What is direct access?
5. How does Linear Search work?
6. What is efficiency?
7. Why does organization matter?
8. How do data structures and algorithms work together?
9. What is scalability?
10. What was the most important concept from Week 1?

Key Takeaways

- Data structures organize information.
- Algorithms process information.
- Organization improves efficiency.
- Searching locates data.
- Indexes provide direct access.
- Good software depends on thoughtful design decisions.
- Week 1 provides the foundation for the rest of CS102.

Notes

Made with Goodnotes

Made with Goodnotes

Expanded Glossary

How to Use This Glossary

Connect every term to three things: what it means, why it matters, and what beginner mistake it prevents.

Core Terms - Part 1

Term	Meaning	Example / Common Confusion
Algorithm	Step-by-step procedure used to solve a problem or complete a task.	Recipe, GPS directions, or a search procedure. Not always code.
Array	Ordered collection of elements stored in indexed positions.	Positions matter as much as values.
Collection	Group of related data stored together.	Students, scores, products, colors, photos.
Data	Information that can be stored, processed, and used.	Names, prices, dates, scores, temperatures.
Data Structure	Method of organizing information so it can be used efficiently.	The structure is the organization system, not the data itself.
Direct Access	Retrieving information immediately using its index.	colors[2] jumps directly to the value at position 2.
Efficiency	Amount of work required to complete a task.	Finding a song instantly vs. checking thousands manually.
Element	Individual value stored inside a collection.	Each color inside a colors list is an element.

Tiny Comparison

Concept	Plain-English Role
Data	The information.
Data structure	How the information is organized.
Algorithm	The process that works with the information.
Efficiency	How much work that process requires.

Glossary Notes

Expanded Glossary

Core Terms - Part 2

Term	Meaning	Example / Common Confusion
Index	Numerical position of an element inside a collection.	Indexes are positions, not values.
Indexed Access	Using an index to retrieve information directly.	students[0] retrieves the first stored student.
Linear Search	Search algorithm that checks elements one at a time until a match is found.	Works even when data is not organized.

Made with Goodnotes

Match	When the target value and current value are the same.	Searching for 'green' and finding 'green'.
Scalability	How well a system performs as data grows.	A solution for 10 items may struggle with 10 million.
Search	Process of locating information.	Finding a contact, product, message, or record.
Sequential Access	Checking information one item at a time.	Searching every locker until the correct one appears.
Target	The value being searched for.	target = 'green'
Zero-Based Indexing	Numbering system where positions begin at zero.	Humans count 1,2,3. Python positions start 0,1,2.

Quick Self-Test

Can you explain the difference between a value and an index? Can you explain why Linear Search still works on unsorted data? Can you explain why scalability matters before a program gets huge?

Memory Hook

Index = address. Value = information at that address. Linear Search = check one item at a time. Scalability = what happens when the data grows.

Glossary Notes

Concept Connections + Common Mistakes

Week 1 Concept Chain

Concept	How It Connects
Data	The raw information: names, scores, photos, messages.
Data structures	Organize the information so it can be used.
Collections	Hold groups of related data.
Indexes	Give positions so information can be accessed directly.
Arrays	Store ordered, indexed values.
Access + updates	Retrieve or change stored information.
Searching	Find information when the exact location is unknown.
Algorithms	Provide repeatable steps for solving the problem.
Efficiency + scalability	Measure work now and performance as data grows.

Common Mistakes by Topic

Topic	Mistake	Reality
Data structures	Thinking they are just vocabulary.	They are organization systems.
Indexes	Starting at 1.	Python starts at 0.
Arrays	Confusing values and positions.	Indexes are positions; elements are values.

Made with Goodnotes

Searching	Assuming the target exists.	Searches can fail.
Algorithms	Thinking they must be complicated.	Simple steps can be algorithms.
Efficiency	Ignoring work required.	Correct solutions can require very different work.

If You Remember Nothing Else

Data structures organize information. Indexes provide addresses. Arrays store ordered collections. Algorithms solve problems. Efficiency matters because working is not the same as working well.

Connection Notes

_____
_____
_____
_____
_____
_____
_____
_____
_____
_____
_____
_____

Review Question Breakdown + Exam Traps

Question Types and Answer Strategies

Question Type	Example	Strategy
Definition	What is a data structure?	Define it and explain its purpose.
Application	Why is a playlist a data structure?	Explain how information is organized.
Index	What does colors[1] return?	Label positions first, then answer.
Search	How does Linear Search work?	Explain step-by-step checking.
Comparison	Sequential vs indexed access	Focus on how retrieval happens.

Common Exam Traps and Fixes

Trap	What Usually Goes Wrong	How to Avoid It
Values vs indexes	colors[1] returns the second value, not the first.	Index = address. Value = data stored there.
Zero-based indexing	The third item is items[2], not items[3].	Write index labels above values.
Search failure	The target may not exist.	Always consider what happens if not found.
append vs insert	append adds to end; insert adds at a specific position.	Ask whether the new value belongs at the end or in the middle.
Correct vs efficient	Two answers can both work, but one may require much more work.	Ask: how many checks or steps?

Index Trap Example

```
colors = ["red", "blue", "green"]

Index:   0      1       2
Value: red   blue   green
colors[1] -> blue
```

Always label indexes before answering code-output questions.

Review Notes

Made with Goodnotes

Cheat Sheet + Exam-Day Quick Reference

Week 1 Cheat Sheet

Term	Fast Meaning	Example / Tes
Data Structures	Organize information	Playlist, contact list, array
Algorithms	Solve problems	Recipe, GPS directions, Linear Search
Arrays	Store ordered data	Index: 0 1 2 / Value: A B C
Elements	Individual values	'red' inside a colors list
Indexes	Positions	colors[0] is the first value
Direct Access	Jump directly to information	items[3]
Sequential Access	Check one item at a time	Linear Search
Efficiency	Amount of work required	How many checks?
Scalability	Performance as data grows	Will this still work with huge data?

Exam-Day Quick Reference

Term	Meaning
Data Structure	Organization system
Algorithm	Step-by-step procedure
Array	Ordered collection
Element	Stored value
Index	Position
Search	Find information
Linear Search	Check items one at a time
Efficiency	Work required
Scalability	Growth performance

Final Memory Hook

Data Structures = how information is organized.
Algorithms = how problems are solved.
Indexes = addresses.
Arrays = organized collections.
Searching = finding information.
Efficiency = work required.
Scalability = what happens when data grows.

Everything comes back to one question: How can information be organized and processed efficiently?

Quick Reference Notes

Practice Exercises + Visual Diagrams

Made with Goodnotes

Practice Set - Lessons 2 through 8

Lesson	Question	Answer / Check
Lesson 2	Which is a data structure: playlist, filing cabinet, contact list, or all of the above?	All of the above.
Lesson 3	letters = ['A','B','C','D']; what does letters[2] return?	C
Lesson 4	Label indexes for colors = ['red','blue','green','yellow'].	0 red, 1 blue, 2 green, 3 yellow
Lesson 5	nums = [10,20,30]; print(nums[1])	20
Lesson 5	colors.append('green') after ['red','blue']	['red', 'blue', 'green']
Lesson 6	scores = [90,85,80]; make the second score 95.	scores[1] = 95
Lessons 7-8	letters = ['A','B','D']; insert 'C' between B and D.	letters.insert(2, 'C')

Trace the Search

Collection	Target	Order of Comparisons
['red','blue','green','yellow']	yellow	red -> blue -> green -> yellow

Visual Diagram Collection

```
Sequential vs Indexed Access
Sequential: 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> 7
Indexed: jump directly to 7

Array Visualization
Index:  0   1   2   3
Value:  A   B   C   D

Linear Search
Searching for F in A B C D E F
Path: A -> B -> C -> D -> E -> F
```

Use these diagrams when you need to explain a concept quickly in plain English.

Practice Notes

Comprehensive Review + Programmer Thinking

One-Sentence Review by Lesson

Lesson	Core Idea
1	CS is about organizing information and solving problems efficiently.
2	Data structures organize information.
3	Indexes provide direct access.
4	Arrays store ordered collections.
5	Indexes allow retrieval of stored values.
6	Values can be updated without changing positions.

Made with Goodnotes

7	Collections often need to grow and adapt.
8	Adding and removing data affects organization.
9	Searching locates information.
10	Algorithms provide repeatable solutions.
11	Linear Search checks one item at a time.
12	Correctness and efficiency are different.
13	Organization affects performance.
14	All concepts work together.

Beginner Thinking vs Programmer Thinking

Topic	Beginner Thinking	Programmer Thinking
Data Structures	Where do I store it?	How should I organize it so future tasks are easier?
Arrays	An array is a list.	An array is a collection of indexed positions.
Searching	Did I find it?	How much work did it take to find it?
Algorithms	What's the answer?	What process consistently produces the answer?
Efficiency	It works.	Will it still work well with a million records?

Week 1 Mindset Shift

Organizing information properly often matters more than writing clever code. A programmer thinks about structure, future operations, efficiency, and growth before getting attached to the first working solution.

Comprehensive Review Notes

_____
_____
_____
_____
_____
_____
_____

Debugging Corner + Final Reflection

Debugging Corner

Issue	Example	Fix / Lesson
IndexError	colors = ['red','blue']; print(colors[5])	Position 5 does not exist. Check collection size and valid indexes.
Wrong index	Expected green, but colors[1] returns blue.	The programmer forgot zero-based indexing. Draw Index: 0 1 2 above values.
Wrong update	scores[0] = 95 changed the first score, not the second.	Label positions before replacing values.
Search failure	target = 'purple' is not in ['red','blue','green'].	Not every search succeeds. Always consider failure cases.

Debugging Checklist

1. Check indexes.
2. Check values.
3. Check collection size.
4. Trace the algorithm manually.
5. Ask what you expected, then compare that to what actuallyhappened.

Week 1 Concept Map

WEEK 1 -> DATA -> DATA STRUCTURES -> COLLECTIONS -> INDEXES -> ARRAYS -> ACCESS / UPDATE / MODIFY -> SEARCHING
-> ALGORITHMS -> EFFICIENCY -> SCALABILITY

Final Week 1 Reflection

Week 1 teaches that software development is fundamentally about organizing information and solving problems efficiently. Everything else in CS102 builds on that idea.

Made with Goodnotes

Final Notes

Made with Goodnotes

CS102 - Week 2 Notes Outline and Study Guide

Table of Contents

What is inside

- Week 2 overview and master framework: LIFO, stacks, push, pop, undo behavior, FIFO, queues, enqueue, dequeue, deque, queue challenges, and choosing between stacks and queues.
- Lesson 1 - What Is LIFO?
- Lesson 2 - Introduce the Stack
- Lesson 3 - Stack Operations: Push and Pop
- Lesson 4 - Building a Stack Using Lists
- Lesson 5 - Applying Stacks to Undo Behavior
- Lesson 6 - Challenge: Stacks
- Lesson 7 - What Is FIFO?
- Lesson 8 - Introduce the Queue
- Lesson 9 - Queue Operations: Enqueue and Dequeue
- Lesson 10 - Implementing Queues with deque
- Lesson 11 - Challenge: Queues
- Lesson 12 - Integrating Stacks and Queues
- Review logic, glossary, common mistakes, debugging tips, and syntax-heavy cheat sheet

Week 2 at a Glance

Week 2 builds directly from Week 1's data-structure foundation. In Week 1, the focus was how information is organized and accessed. In Week 2, the focus becomes the rule that controls which item leaves first. That rule changes the behavior of the whole structure.

The two major patterns are stacks and queues. A stack follows LIFO - Last In, First Out. A queue follows FIFO - First In, First Out. These sound like small vocabulary words, but they describe two very different ways software manages waiting items, histories, actions, and tasks.

Stacks are newest-first structures. They match plate piles, folded towels, browser back buttons, undo histories, and any situation where the most recent action should be handled first. Queues are oldest-first structures. They match checkout lines, ticket systems, print jobs, task schedulers, and any situation where fairness or arrival order matters.

Memory hook: stack = newest item leaves first. queue = oldest waiting item leaves first. If the latest action should be undone, think stack. If the earliest waiting item should be handled next, think queue.

Made with Goodnotes

Lesson 1 - What Is LIFO?

Big Idea

LIFO means Last In, First Out. The last item added to a collection is the first item removed. This lesson uses physical examples, like plates, books, apps, and folded towels, to make the rule feel obvious before turning it into code.

Full Teaching Explanation

The core idea of LIFO is order of removal. Imagine stacking clean plates in a cabinet. Every new plate goes on top. Later, when you need a plate, your hand naturally reaches for the top plate first. That top plate is the most recently added plate, so it is also the first one removed. That is Last In, First Out.

This matters because a computer science stack works the same way. The structure does not let you casually pull from the bottom or middle without disturbing the whole stack. The accessible item is the newest item at the top. LIFO is less about the object itself and more about the rule the collection follows.

A line of people is the opposite. The first person who got in line is usually the first person served. That is not LIFO because the oldest item leaves first, not the newest. Week 2 uses this contrast repeatedly: stacks remove from the newest end; queues remove from the oldest end.

The goal of this lesson is to build the mental picture first. Before you think about Python lists, append(), or pop(), you should be able to look at a situation and ask: does the newest item leave first? If yes, it probably behaves like LIFO.

Examples and Mental Models

Situation	Is it LIFO?	Why
Stack of plates	Yes	Newest plate is on top and leaves first.
Pile of books	Yes	Top book was placed last and is read first.
Folded towels	Yes	Clean towels are added on top; the top towel is taken first.
People waiting in line	No	First person in line is served first, so this is FIFO.
Closing recent apps	Yes	Most recent app or action is usually handled first.

Out Loud Programmer Thinking

When I see a collection, I ask which end matters. If the newest item is the next one removed, I am thinking in LIFO terms. If the oldest waiting item is next, I am thinking in FIFO terms.

A useful sentence is: "The last thing I put on is the first thing I take off." That sentence explains plate stacks, towel stacks, browser back buttons, and undo buttons.

Terms, Definitions, and Why They Are Called That

- LIFO: Last In, First Out. The newest item leaves first.
- Last In: the item most recently added to the structure.
- First Out: the item removed before the others.
- Top: the accessible end of a stack. New items are added at the top and removed from the top.
- Stack behavior: newest-first behavior. The structure behaves like a physical stack of objects.

Common Mistakes and Helpful Tips

Made with Goodnotes

- Mistake: thinking LIFO means the most important item leaves first. LIFO is about order added, not importance.
- Mistake: calling a waiting line LIFO. A normal line is FIFO because the first person in line is served first.
- Mistake: focusing on the example instead of the rule. Plates, books, towels, and apps are only examples. The rule is newest item out first.
- Helpful tip: draw arrows. Add items upward, then remove from the top. The picture usually makes the answer obvious.

Lesson 1 - What Is LIFO? (continued)

Notes

Made with Goodnotes

Lesson 2 - Introduce the Stack

Big Idea

A stack is a collection that follows LIFO behavior. Items are arranged so the last item added becomes the next item available at the top. A stack is not just any list of items; it is a collection with a specific access rule.

Full Teaching Explanation

A stack is the formal data-structure version of the plate pile. The top is the active end. When you add something, it goes on top. When you remove something, it comes from the top. The bottom and middle items are still in the stack, but they are not the next available item.

This is why the word stack is so visual. It describes items arranged on top of one another. In a real stack of dishes, the top item blocks access to the items below it. In a software stack, the newest item is treated as the accessible item. You can think of the stack as having a single doorway: items enter and leave through the same doorway at the top.

Stacks become useful when the program needs to remember a history of actions in reverse order. The newest action is usually the one you undo first. The most recent page might be the one a browser goes back from first. The most recent function call is the one that finishes first. All of those examples match the stack rule.

The important part is not just the definition. The important part is recognizing when a problem naturally wants stack behavior. If the task says the most recent thing should be handled first, that is a stack clue.

Stack Anatomy

Part	Meaning	Example
Top	Only directly accessible end.	Top pancake or top plate.
Bottom	Oldest item in the stack.	First plate placed down.
Push/Add	Put a new item at the top.	Add a clean towel to the stack.
Pop/Remove	Take the top item away.	Remove the top plate first.

Code Example - Thinking of a Stack as Data

Python lists can simulate stack behavior because they can add to the end and remove from the end. The end of the list acts like the top of the stack.

```
stack = []
stack.append("plate A")
stack.append("plate B")
stack.append("plate C") print(stack)

top_plate = stack.pop()
print(top_plate) print(stack)
```

Line-by-line explanation:

- stack = [] creates an empty list that will act like a stack.
- append() adds each new plate to the end of the list, which we treat as the top.
- pop() removes and returns the most recently added item.

Real English translation: "Build a stack of plates, then take the top plate off."

Lesson 2 - Introduce the Stack (continued)

Made with Goodnotes

Terms, Definitions, and Why They Are Called That

- Stack: a collection where the last item added is the next item removed. It is called a stack because it behaves like objects stacked ontop of one another.
- Top: the newest and next available position.
- Push: the operation that adds an item to the top. In Python list practice, append() plays this role.
- Pop: the operation that removes and returns the top item.
- LIFO: the ordering rule that makes a stack a stack.

Common Mistakes and Helpful Tips

- Mistake: thinking a stack is just any collection. A stack is defined by its removal rule.
- Mistake: trying to remove from the bottom and still call it stack behavior.
- Mistake: forgetting that the top changes every time you push or pop.
- Helpful tip: before writing code, label which side is the top. In Python list examples, we usually treat the right side/end as the top.

Notes

Made with Goodnotes



Lesson 3 - Stack Operations: Push and Pop

Big Idea

Stacks become useful when you can change them. The two core stack operations are push and pop. Push adds an item to the top. Pop removes and returns the current top item. Together, they turn LIFO from a definition into actual behavior.

Full Teaching Explanation

This lesson connects the stack concept to Python operations. In many languages, adding to a stack is called push. Removing from a stack is called pop. Python lists do not use a method named push, but list.append() behaves like push when the end of the list is treated as the stack top.

The most important detail is that pop() does two things at once. It removes the top item, and it returns that removed item. Beginners often remember the removing part but forget the returning part. That returned value matters because the program often needs to know what was just removed.

For example, if the stack contains ["A", "B", "C"], the top is "C" because it was added last. Calling pop() returns "C" and changes the stack to ["A", "B"]. If you call pop() again, it returns "B" and leaves ["A"]. This is the stack shrinking in exact reverse order.

When tracing stack code, do not only stare at the final print statement. Write the stack after each append and each pop. Stack questions are usually state questions: what does the stack contain right now?

Code Example 1 - Push Three Items, Pop One

Made with Goodnotes

This is the basic stack pattern used throughout the week.

```
stack = []

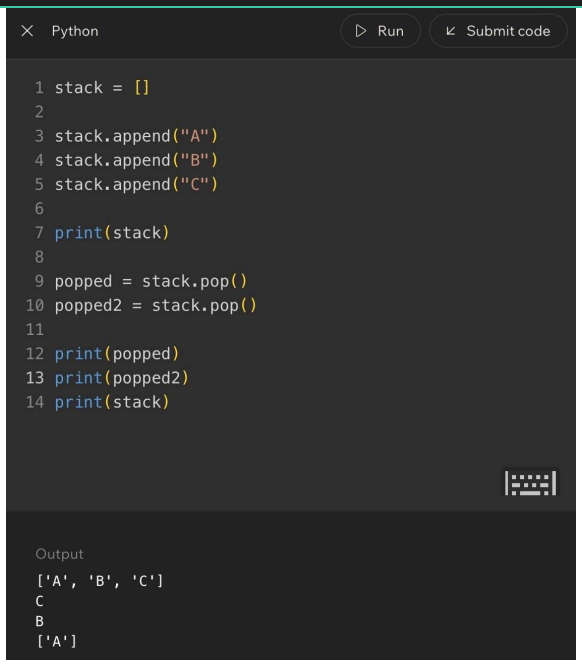
stack.append("A")
stack.append("B")
stack.append("C")
print(stack)

popped = stack.pop()
print(popped)
print(stack)
```

Line-by-line explanation:

- append("A"), append("B"), and append("C") push values onto the stack.
- After the pushes, the stack is ["A", "B", "C"]. The top is "C".• popped = stack.pop() removes "C" and stores it in popped.
- print(popped) displays the removed item.
- print(stack) displays what is left: ["A", "B"].

Real English translation: "Add A, B, and C. Take off the top item. The top item is C."



Lesson 3 - Stack Operations: Push and Pop (continued)

Code Example 2 - Pop Twice

Made with Goodnotes

After one pop, the item underneath becomes the new top.

```
stack = []
stack.append("A")
stack.append("B")
stack.append("C")

popped = stack.pop()
popped2 = stack.pop()

print(popped)
print(popped2)
print(stack)
```

Output logic:

- The first pop returns "C".
- The second pop returns "B" because after C leaves, B becomes the top.
- The stack ends as ["A"].

Out loud programmer thinking: "The top keeps changing after each pop."

Stack State Trace

Operation	Returned Value	Stack After
Start	-	[]
append("A")	-	["A"]
append("B")	-	["A", "B"]
append("C")	-	["A", "B", "C"]
pop()	"C"	["A", "B"]
pop()	"B"	["A"]

Terms, Common Mistakes, and Helpful Tips

- Push: add an item to the top of the stack. In Python list examples, append() does this.
- Pop: remove and return the item at the top. In Python lists, pop() does this from the end by default.
- Returned value: the value pop() gives back after removing it.
- Mistake: thinking pop() only deletes. It also returns the removed item.
- Mistake: losing track of the new top after a pop.
- Tip: write the stack after every operation when solving quiz questions.

Notes

Lesson 4 - Building a Stack Using Lists

Made with Goodnotes

Big Idea

Python lists can be used to build stack behavior when you consistently use append() to push items and pop() to remove the most recent item. The list is the storage tool; the stack is the behavior pattern you enforce.

Full Teaching Explanation

This lesson turns the stack idea into a repeatable coding pattern. Start with an empty list. Use append() to add items. Use pop() to undo or remove the newest item. After every operation, the list represents the current stack state.

The key is consistency. A list can do many things besides stack behavior. It can insert in the middle, remove by value, sort, slice, and more. But when you are using it as a stack, you intentionally restrict yourself to stack-style operations. You push to the end and pop from the end.

This is an important programming habit: sometimes the data structure is about how you choose to use the tool. Python does not stop you from misusing the list, so you have to preserve the stack rule yourself.

Another important idea is empty-stack safety. If you call pop() on an empty list, Python raises an error because there is no top item to remove. Before popping, you should check whether the stack has anything in it. That guard turns fragile code into safer code.

Code Example 1 - Build and Shrink a Stack

This example shows the stack growing with append() and shrinking with pop().

```
stack = []

stack.append("box1")
print(stack)

stack.append("box2")
print(stack)

stack.append("box3")
print(stack)

top_box = stack.pop()
print(top_box)
print(stack)
```

Real English translation:

- Start with no boxes.
- Add box1, then box2, then box3.
- The top box is box3 because it was added last.
- pop() removes box3 and returns it into top_box.
- The remaining stack is ["box1", "box2"].

Code Example 2 - Checking Before Popping

Made with Goodnotes

Before popping, check whether the stack contains at least one item.

```
stack = []

if len(stack) > 0:
    item =
stack.pop()
print(item) else:
print("empty")
```

Line-by-line explanation:

- len(stack) > 0 checks whether there is an item to remove.
- If the stack has something, pop() is safe.
- If the stack is empty, the else branch runs instead.

Real English translation: "Only remove from the stack if there is actually something there."

Lesson 4 - Building a Stack Using Lists (continued)

Trace Table - Mini Challenge

Operation	What It Does	Why It Helps
append("item1") Adds item1 to the top. Shows stack growth. append("item2") Adds item2 above item1. Newest item becomes top.		
pop()	Removes item2.	Newest item leaves first.
len(stack) > 0	Checks for items.	Prevents empty pop errors.

Terms, Common Mistakes, and Helpful Tips

- Empty stack: a stack with no items left to remove.
- Guard condition: a condition that protects code from unsafe behavior.
- State: the current contents of the stack at a specific moment.
- Mistake: calling pop() repeatedly without checking whether anything is left.
- Mistake: assuming the original stack stays the same after pop(). pop() changes the stack.
- Tip: after each stack operation, write the new stack state in brackets.

Notes

Made with Goodnotes

Code Example:

Python Run Submit code

```python
stack = []

stack.append("box1")
print(stack)

stack.append("box2")
print(stack)

stack.append("box3")
print(stack)

top_box = stack.pop()

print(top_box)
print(stack)
```

Output

```
['box1']
['box1', 'box2']
['box1', 'box2', 'box3']
box3
['box1', 'box2']
```

Made with Goodnotes

Code Example:

Python

Run Submit code

```python
stack = []

stack.append("item1")
stack.append("item2")
stack.append("item3")


stack.pop()
print(stack)

stack.pop()
print(stack)

if len(stack) > 0:
    stack.pop()
    print(stack)

if len(stack) == 0:
    print("Done, stack is empty")
```

Output

```
['item1', 'item2']
['item1']
[]
Done, stack is empty
```

Lesson 5 - Applying Stacks to Undo Behavior

Big Idea

Undo behavior is one of the clearest real-world uses of a stack. Each action is pushed onto a history stack. When the user presses undo, the most recent action is popped first.

Full Teaching Explanation

Made with Goodnotes

Imagine drawing three shapes in order: a red line, a blue circle, and a green square. If you press undo once, the app removes the green square first because that was the most recent action. If you press undo again, it removes the blue circle. The actions are undone in reverse order.

That reverse-order behavior is exactly LIFO. The first action may still matter, but it is buried under newer actions. The newest action is the one sitting on top of the stack. Undo systems use this idea because users usually expect the last thing they did to be the first thing removed.

In code, you can model an undo history with a list. Each user action is appended to the stack. When undo happens, pop() removes the latest action and returns it so the program knows what to reverse. The remaining list represents the actions that still exist.

The deeper lesson is that stacks are not abstract trivia. They model real software memory. Whenever a program needs to remember a sequence and walk backward through it, a stack is often the right mental model.

Code Example 1 - Undo Type Actions

The list stores action history. Each append() records a new action. Each pop() undoes the latest action.

```
actions = []

actions.append("open file")
actions.append("type hello")
actions.append("type world")
actions.append("press enter")
actions.append("type !")

last_action = actions.pop()
print(last_action)
print(actions)
```

Line-by-line explanation:

- actions = [] creates the undo history.
- Each append() records what the user just did.
- The final action, "type !", is the top of the stack.
- actions.pop() removes and returns that latest action.

Real English translation: "Remember everything the user did, then undo the newest action first."

Undo State Evolution

Action	Stack State	Meaning
Open file	["open file"]	One action recorded.
Type hello	["open file", "type hello"]	New action added on top.
Type world	[..., "type world"]	History keeps growing.
Undo once	Removes "type world"	Most recent action is reversed.
Undo again	Removes "type hello"	Next most recent action is reversed.

Lesson 5 - Applying Stacks to Undo Behavior (continued)

Code Example 2 - Save the Undo Action

Made with Goodnotes

Usually, the program needs to know which action was undone.

```
actions = ["draw line", "draw circle", "draw square"]

undo_1 = actions.pop()
undo_2 = actions.pop()

print(undo_1)
print(undo_2)
print(actions)
```

Output logic:

- undo_1 is "draw square" because it was added last.
- undo_2 is "draw circle" because after square is removed, circle becomes the top.
- The remaining stack is ["draw line"].

This is the same LIFO pattern from plates, books, and towels, but now it powers an actual software feature.

Real-World Meaning

Stacks appear anywhere software needs to move backward through recent history. Undo buttons, browser back buttons, editor action histories, and some navigation flows all depend on the idea that the latest action is the first one reversed.

This is why the stack rule matters. It does not just describe storage. It describes what users expect. If undo removed the oldest action first, the feature would feel broken even if the code technically ran.

Terms, Common Mistakes, and Helpful Tips

- Undo stack: a stack that stores actions so the newest action can be reversed first.
- Action history: the ordered list of actions a user performed.
- Current state: what remains after some actions are undone.
- Mistake: undoing the first action instead of the latest action.
- Mistake: popping from an empty undo stack.
- Tip: when you see "undo," think "store actions as they happen, then pop the newest one."

Key Takeaways

- LIFO means the last item added is the first item removed.
- A stack is a data structure that enforces LIFO behavior.
- push adds to the top; pop removes and returns the top.
- Python lists can model stacks with append() and pop().
- Undo behavior is a stack because the most recent action is undone first.
- Always trace stack state after each operation, especially after pop().

Notes

Made with Goodnotes

Code Example:

Python Run Submit code

```python
actions = []

actions.append("open file")
actions.append("type 'hello'")
actions.append("type ' world'")
actions.append("press enter")
actions.append("type '!'")

actions.pop()
actions.pop()

print(actions)
```

Output

```
['open file', "type 'hello'", "type ' world'"]
```

Made with Goodnotes

Lesson 6 - Challenge: Stacks

Big Idea

This lesson is where the stack idea stops being just a definition and becomes a tracing skill. You practice reading a sequence of push and pop operations, predicting which value is removed, and describing what remains in the stack afterward.

The important habit is not guessing. The important habit is tracking the stack state after every line. In stack problems, order matters because the last item pushed is the first item available to pop.

Full Teaching Explanation

A stack is easiest to understand when you treat it like a physical pile. Every new item goes onto the top. When something is removed, it comes off the top. That means the newest item is the most accessible item. The bottom items are still there, but you cannot reach them until the items above them are removed first.

In Python, a list can simulate a stack. The right side of the list acts like the top of the stack. When you use append(), Python adds the new item to the end of the list. When you use pop(), Python removes and returns the last item from the list. That gives you LIFO behavior: Last In, First Out.

This lesson uses challenge rounds because stacks are less about memorizing words and more about being able to trace motion. Each append changes the stack by adding a new top. Each pop changes the stack by removing the current top. If the popped value is stored in a variable, that variable remembers the removed value while the stack itself has already changed.

Stack State Mental Model

Question	What to Ask Yourself
What is on top?	Look at the most recently appended item, usually the rightmost item in a Python list.
What does pop() remove?	It removes and returns the current top item.
What remains?	Everything underneath the removed item stays in the stack.
Did a variable store the pop?	The variable stores the removed value; the stack no longer contains it.

Lesson 6 - Challenge: Stacks (Code Walkthrough)

Code Example 1 - One Pop Removes the Top

Made with Goodnotes

```
stack = []
stack.append("plate A")
stack.append("plate B")
stack.append("plate C")
removed = stack.pop()

print(removed)
print(stack)
```

Expected output:

```
plate C
['plate A', 'plate B']
```

Line-by-line explanation:

- stack = [] starts with an empty stack.
- append('plate A'), append('plate B'), and append('plate C') push three plates onto the stack in that order.
- Because plate C was added last, it is on top.
- removed = stack.pop() removes plate C and stores that removed value in removed.
- After the pop, the stack still contains plate A and plate B, with plate B now on top.

Real English translation: Build a stack of plates, take the top plate off, remember which plate you took, and then look at what is left.

Code Example 2 - Multiple Pops

```
stack = []
stack.append(10)
stack.append(20)
stack.append(30)

first = stack.pop()
second = stack.pop()

print(first)
print(second)
print(stack)
```

Expected output:

```
3
2
[10]
```

The first pop removes 30 because 30 was added last. After 30 is gone, 20 becomes the new top. The second pop removes 20. The only value left at the bottom is 10.

Out loud programmer thinking: 'I do not just ask what values were added. I ask what the stack looks like after each operation.'

Lesson 6 - Challenge: Stacks (Debug Thinking)

Terms, Definitions, and Why They Matter

Made with Goodnotes

- Push: adding an item to the top of a stack. In Python list stack examples, append() performs the push.
- Pop: removing and returning the top item. pop() changes the list and gives back the removed value.
- Top: the next item that will be removed. In these Python list examples, the top is the rightmost item.
- Bottom: the oldest item in the stack. It cannot be removed until everything above it is gone.
- Stack state: the current contents of the stack after some operations have already happened.
- Mutation: changing the collection itself. pop() mutates the stack because the removed item is no longer inside it.

Common Mistakes and Helpful Tips

- Mistake: thinking pop() only shows the top item. It does not just peek. It removes the item.
- Mistake: forgetting that a variable can store the popped value. If first = stack.pop(), first gets the removed item.
- Mistake: reading the list left-to-right as if the left side is always the top. In the course examples, the right side is thetop.
- Mistake: trying to pop from an empty stack. If there are no items, pop() has nothing to remove and Python canraise an error.
- Helpful tip: after every append or pop, rewrite the stack state. It feels slow, but it prevents almost everystack-tracing mistake.
- Memory hook: append grows the top, pop removes the top, and the top is always the newest item.

Notes

Lesson 7 - What Is FIFO?

Big Idea

FIFO means First In, First Out. The first item that enters a collection is the first item removed. If LIFO feels like a stack of plates, FIFO feels like a line of people waiting their turn.

This lesson matters because it introduces the second major access pattern of the week. Stacks are useful when the newest item should be handled first. Queues are useful when the oldest waiting item should be handled first.

Full Teaching Explanation

A FIFO structure preserves arrival order. Imagine a coffee shop line. The person who arrived first stands at the front, waits the longest, and gets served first. New people join at the back. Nobody should cut ahead of the people already waiting. That is the entire idea behind FIFO.

This is different from LIFO. In LIFO, the most recently added item is the next one removed. In FIFO, the oldest waiting item is the next one removed. Both patterns are about order of removal, but they answer opposite questions. LIFO asks, 'What was added most recently?' FIFO asks, 'Who has been waiting the longest?'

The lesson uses lines, restaurant orders, general admission events, shopping app orders, and service systems because those examples all rely on fairness or chronological order. The first request, order, person, or task should usually be handled before newer ones.

Made with Goodnotes

Examples and Mental Models

Situation	FIFO?	Why
Coffee shop line	Yes	The first person in line gets served first.
Restaurant order queue	Yes	Orders are prepared in the order they arrive.
General admission event	Usually yes	People line up and enter in arrival order unless priority rules exist.
Shopping app orders	Often yes	Earlier orders are usually handled before later orders.
Stack of books	No	That is usually LIFO because the top/newest item is removed first.

Lesson 7 - FIFO vs. LIFO

Direct Comparison

Pattern	Rule	Mental Model	Best When...
LIFO	Last In, First Out	Stack of plates	The newest item should be handled first.
FIFO	First In, First Out	Line at a coffee shop	The oldest waiting item should be handled first.

Real English translation: A stack says, 'take the newest thing.' A queue says, 'serve the person or item that has been waiting the longest.'

Out loud programmer thinking: 'Before I choose a structure, I need to know which end matters. Am I handling the newest item first or the oldest waiting item first?'

Terms, Mistakes, and Helpful Tips

- FIFO: First In, First Out. The first item added is the first item removed.
- Queue: a collection that follows FIFO behavior. New items join at the back, and items leave from the front.
- Front: the end where items leave. In a line, this is where service happens.
- Rear/back: the end where new items enter.
- Mistake: calling any ordered collection FIFO. FIFO is specifically about removal order, not just having an order.
- Mistake: confusing fairness with stack behavior. A fair line is FIFO, not LIFO.
- Memory hook: FIFO is the polite line. First person waiting gets helped first.

Notes

Lesson 8 - Introduce the Queue

Made with Goodnotes

Big Idea

A queue is the data structure version of a line. New items are added at the rear, and items are removed from the front. That makes a queue the natural structure for FIFO behavior.

This lesson takes the idea from Lesson 7 and gives it a computer science name. When the first item in should be the first item out, you are thinking in queue terms.

Full Teaching Explanation

A queue is an ordered collection with two important ends: the front and the rear. The front is where items leave. The rear is where items enter. In a coffee shop, the customer at the front is served next, and new customers join at the back.

The course examples use mobile coffee orders to make this concrete. New drinks get added at the back of the line, customers take their drinks from the front, and the drink that has been waiting the longest leaves first. The order is not random, and it is not newest-first. It is FIFO: First In, First Out.

In Python, an introductory queue can be modeled with a list. The end of the list can act as the rear where append() adds new items. The beginning of the list, index 0, can act as the front where pop(0) removes the oldest item. This is simple and clear for learning the concept. Later, Python's deque gives a more efficient queue implementation.

Queue Direction Model

Queue Part	Meaning	Python List Learning Model
Rear/back	Where new items join	queue.append(item)
Front	Where the next item leaves	queue.pop(0)
Oldest item	The item waiting the longest	The value at index 0
Newest item	The item most recently added	The rightmost value

Lesson 8 - Introduce the Queue (Code Walkthrough)

Code Example - Enqueue and Dequeue with a List

Made with Goodnotes

```
queue = []

queue.append("order A")
queue.append("order B")
queue.append("order C")
served = queue.pop(0)

print(served)
print(queue)
```

Expected output:

```
order A
['order B', 'order C']
```

Line-by-line explanation:

- queue = [] starts with an empty line.
- append('order A'), append('order B'), and append('order C') add new orders to the rear of the queue.
- Because order A was added first, it is at the front.
- served = queue.pop(0) removes and returns the item at index 0, which is the front of the queue.- After order A leaves, order B becomes the new front.

Real English translation: Build a line of orders, serve the oldest order first, and leave the remaining orders waiting in the same order.

Common Mistakes and Helpful Tips

- Mistake: using pop() instead of pop(0). pop() removes from the end, which creates stack-like LIFO behavior insteadof queue-like FIFO behavior.
- Mistake: thinking the rear and front are the same end. Queues use two ends: add at the rear, remove from thefront.
- Mistake: assuming the middle item leaves next. In a standard queue, the front item leaves next.
- Helpful tip: draw queues horizontally. Put FRONT on the left and REAR on the right so pop(0) and append() makevisual sense.
- Memory hook: queue = line. New people join the back. The front person leaves first.

Notes

Lesson 9 - Queue Operations: Enqueue and Dequeue

Big Idea

Made with Goodnotes

Queues follow FIFO: First In, First Out. This lesson turns that idea into actual Python behavior by showing how to add items to the rear of a queue and remove items from the front.

The main operations are enqueue and dequeue. Enqueue means adding a new item to the back of the line. Dequeue means removing and returning the item at the front of the line.

Full Teaching Explanation

A queue is a data structure for waiting turns. The first item added is the first item removed. If a stack feels like a pile of plates, a queue feels like a food-truck line, checkout line, ticket window, or customer-service queue.

In the early examples, the lesson models a queue with a regular Python list. The rear of the queue is the end of the list, so append() works for adding new people to the back. The front of the queue is index 0, so pop(0) removes the oldest item. That matches FIFO because the item that has been waiting the longest leaves first.

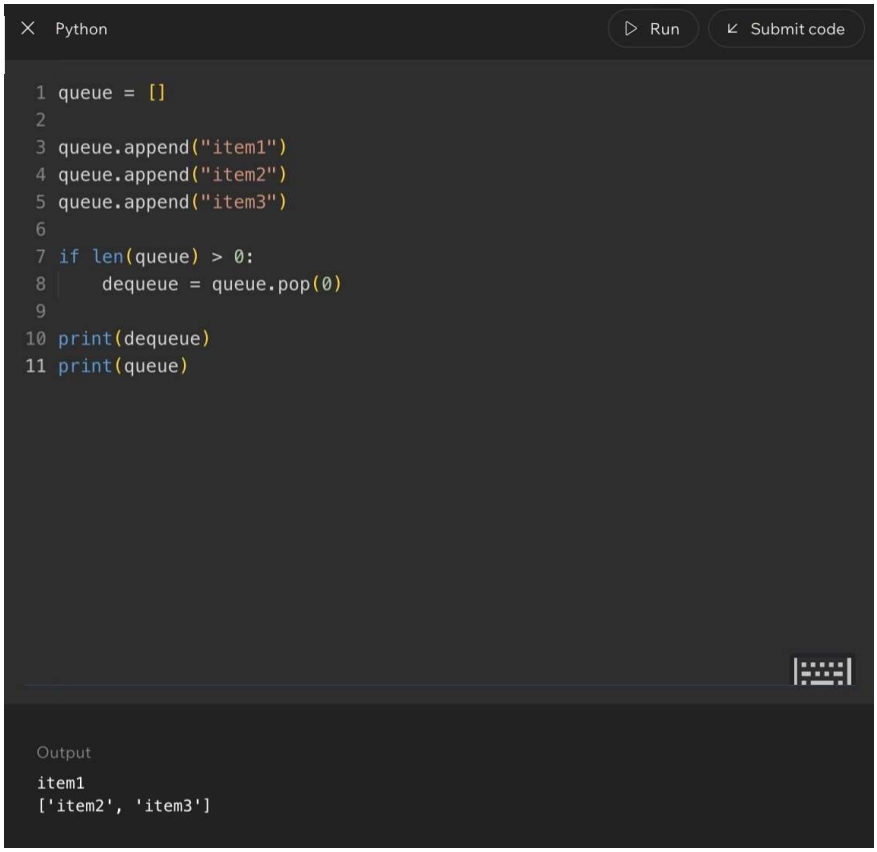
This lesson also introduces an important safety habit: do not dequeue from an empty queue. Before removing from the front, check whether the queue still has items. In plain English, if there is no one in line, you should not try to serve the next person.

Made with Goodnotes

Queue Vocabulary

Term	Meaning	Python List Version
Front	The next item to be removed	Index 0
Rear	Where new items are added	End of the list
Enqueue	Add an item to the rear	queue.append(item)
Dequeue	Remove and return the front item	queue.pop(0)
Empty queue	A queue with no items waiting	len(queue) == 0

Code Example:



Lesson 9 - Queue Operations: Enqueue and Dequeue (Code)

Code Example 1 - Basic Enqueue and Dequeue

Made with Goodnotes

```
queue = ["Alice", "Bob", "Charlie"]

queue.append("Daisy")      # enqueue at the rear
served = queue.pop(0)      # dequeue from the front

print(served)
print(queue)
```

Expected output:

```
Alice
['Bob', 'Charlie', 'Daisy']
```

Line-by-line explanation:

- queue starts with Alice at the front, then Bob, then Charlie.
- queue.append('Daisy') adds Daisy to the rear. She does not cut in front of anyone.
- served = queue.pop(0) removes Alice because Alice is at index 0, the front of the line.
- The remaining queue starts with Bob, then Charlie, then Daisy.

Real English translation: Add Daisy to the back of the line, serve the person at the front, and then show who is still waiting.

Code Example 2 - Functions for Queue Behavior

```
def enqueue(queue, item):
    queue.append(item)


def dequeue(queue):
    if len(queue) > 0:
        return queue.pop(0)
    else:
        return None


queue = []
enqueue(queue, "Alice")
enqueue(queue, "Bob")
enqueue(queue, "Charlie")

served = dequeue(queue)
print(served)
print(queue)
```

Expected output:

```
Alice
['Bob', 'Charlie']
```

Why this version is cleaner: the behavior now has names. enqueue() means add to the rear. dequeue() means remove from the front if there is anyone in line. The if len(queue) > 0 check prevents the function from trying to remove from an empty queue.

Lesson 9 - Queue Operations: Debug Thinking

Line-by-Line Programmer Thinking

Made with Goodnotes

- Before append: ask who is already waiting.
- After append: place the new item at the rear, not the front.
- Before pop(0): check whether the queue has at least one item.
- After pop(0): the old front is gone, and everyone behind that item shifts forward.
- If the queue is empty: return None or handle the situation safely instead of crashing.

Common Mistakes and Helpful Tips

- Mistake: using pop() instead of pop(0). pop() removes from the end, which behaves like a stack, not a queue.
- Mistake: adding new items to the front. In a normal queue, new arrivals go to the rear.
- Mistake: forgetting that pop(0) changes the queue. The removed item is no longer inside the list.
- Mistake: trying to dequeue from an empty queue. Always check len(queue) first when the queue might be empty.
- Helpful tip: draw the queue like a line: front on the left, rear on the right. It makes the direction of movementobvious.
- Memory hook: enqueue enters at the rear; dequeue departs from the front.

Notes

Made with Goodnotes

Code Example:

Python

Run Submit code

```
def enqueue(queue, item):
    queue.append(item)

def dequeue(queue):
    if len(queue) > 0:
        return queue.pop(0)
    else:
        return None

queue = []

enqueue(queue, "Alice")
enqueue(queue, "Bob")
enqueue(queue, "Charlie")

served = dequeue(queue)

print(served)
print(queue)
```

Output

```
Alice
['Bob', 'Charlie']
```

Python

Run Submit code

```
def enqueue(queue, item):
    queue.append(item)

def dequeue(queue):
    if len(queue) > 0:
        return queue.pop(0)
    else:
        return None

queue = []

enqueue(queue, "Mia")
enqueue(queue, "Noah")
enqueue(queue, "Liam")

served1 = dequeue(queue)
print(served1)

served2 = dequeue(queue)
print(served2)

served3 = dequeue(queue)
print(served3)

served4 = dequeue(queue)
if served4 == None:
    print("No one left in line")
```

Output

```
Mia
Noah
Liam
No one left in line
```

Lesson 10 - Implementing Queues with deque

Big Idea

This lesson improves the queue implementation. Instead of using a regular list with pop(0), Python provides deque, a special structure designed for fast additions and removals from both ends.

The queue rule does not change. Items still enter at the rear and leave from the front. What changes is the tool Python uses to make that behavior cleaner and more efficient.

Full Teaching Explanation

A regular Python list can model a queue, but it is not the ideal tool when you need to remove from the front repeatedly. With a list, pop(0) removes the first item, but the remaining items must shift forward. That can become inefficient as the collection grows.

The deque type comes from Python's collections module. The name stands for double-ended queue. It is built to add and remove items efficiently from either end. For FIFO queue behavior, you add to the rear with append() and remove from the front with popleft().

The key concept is that deque lets the code say what it means. append() still means add to the back. popleft() makes the front-removal behavior explicit. Instead of using pop(0), you can read popleft() and immediately understand that the left/front item is leaving.

Made with Goodnotes

List Queue vs. deque Queue

Operation	List Version	deque Version	Meaning
Create queue	queue = []	queue = deque()	Start empty
Add to rear	queue.append(item)	queue.append(item)	New item joins back
Remove front	queue.pop(0)	queue.popleft()	Oldest item leaves
Check empty	len(queue) > 0	len(queue) > 0	Remove only if safe

Code Example:

Python Run Submit code

```python
from collections import deque

queue = deque()

queue.append("Sophia")
queue.append("CJ")
queue.append("Caleb")

person_done = queue.popleft()

print(person_done)
print(queue)
```

Output

```
Sophia
deque(['CJ', 'Caleb'])
```

Python Run Submit code

```python
from collections import deque

# create an empty queue
queue = deque()

# add to the REAR (back)
queue.append("A")
queue.append("B")

# remove from the FRONT
person = queue.popleft()
print(person)   # "A"
print(queue)    # deque(['B'])
```

Output

```
A
deque(['B'])
```

Lesson 10 - Implementing Queues with deque (Code)

Code Example 1 - Basic deque Queue

Made with Goodnotes

```
from collections import deque

queue = deque()

queue.append("A")
queue.append("B") person
= queue.popleft()

print(person)
print(queue)
```

Expected output:

```
A
deque(['B'])
```

Line-by-line explanation:

- from collections import deque imports the queue-friendly structure.
- queue = deque() creates an empty deque.
- queue.append('A') and queue.append('B') add items to the rear.
- queue.popleft() removes the item from the front, so A leaves first.- The queue still contains B afterward.

Real English translation: Build a line, let the first person leave, and show who remains in line.

Code Example 2 - Safe deque Removal

```
from collections import deque

queue = deque()
queue.append("Sophia")
queue.append("CJ")
queue.append("Caleb")

if len(queue) > 0:
    person_done = queue.popleft()
else:
    person_done = None
    print("No one left in queue")

print(person_done)
print(queue)
```

Expected output:

```
Sophia
deque(['CJ', 'Caleb'])
```

The if statement is the safety guard. It checks whether the queue has someone in it before calling popleft(). That matters because popleft() needs a front item to remove. If the queue is empty, there is no front item, so the safe branch stores None instead.

Lesson 10 - Implementing Queues with deque (Practice Pattern)

Code Example 3 - Removing Two People Safely

Made with Goodnotes

```
from collections import deque

ride_line = deque()
ride_line.append("Sophia")
ride_line.append("CJ")
ride_line.append("Caleb")

if len(ride_line) > 0:
    person1 = ride_line.popleft()
else:
    person1 = None
    print("No one left in queue")

if len(ride_line) > 0:
    person2 = ride_line.popleft()
else:
    person2 = None
    print("No one left in queue")

print("Left the line:", person1)
print("Left the line:", person2)
print("Still in line:", ride_line)
```

Expected output:

```
Left the line: Sophia
Left the line: CJ
Still in line: deque(['Caleb'])
```

This example repeats the safe check before each removal. That matters because the queue changes after every popleft(). A queue that was not empty before the first removal might become empty before the second removal.

Terms, Common Mistakes, and Helpful Tips

- deque: a double-ended queue from Python's collections module. It is designed for efficient work at both ends.
- popleft(): removes and returns the item on the left/front side of the deque.
- append(): adds an item to the right/rear side of the deque.
- Mistake: forgetting to import deque before using it.
- Mistake: using pop() when FIFO behavior requires popleft(). pop() removes from the rear.
- Tip: if you are modeling a real line, deque is usually the better tool than a plain list.
- Memory hook: append to the back, popleft from the front.

Notes

Made with Goodnotes

Code Example:

Python

Run | Submit code

```python
from collections import deque

queue = deque()

queue.append("Sophia")
queue.append("CJ")
queue.append("Caleb")

if len(queue) > 0:
    person_done = queue.popleft()
else:
    person_done = None
    print("No one left in queue")

print(person_done)
print(queue)
```

Output

```
Sophia
deque(['CJ', 'Caleb'])
```

Python

Run | Submit code

```python
from collections import deque

ride_line = deque()

ride_line.append("Sophia")
ride_line.append("CJ")
ride_line.append("Caleb")

if len(ride_line) > 0:
    person1 = ride_line.popleft()
    person2 = ride_line.popleft()
else:
    print("No one left in queue")

print("Left the line:", person1)
print("Left the line:", person2)
print("Still in line:", ride_line)
```

Output

```
Left the line: Sophia
Left the line: CJ
Still in line: deque(['Caleb'])
```

Python

Run | Submit code

```python
from collections import deque

ride_line = deque()

ride_line.append("Sophia")
ride_line.append("CJ")
ride_line.append("Caleb")

if len(ride_line) > 0:
    person1 = ride_line.popleft()
else:
    person1 = None
    print("No one left in queue")

if len(ride_line) > 0:
    person2 = ride_line.popleft()
else:
    person2 = None
    print("No one left in queue")

print("Left the line:", person1)
print("Left the line:", person2)
print("Still in line:", ride_line)
```

Output

```
Left the line: Sophia
Left the line: CJ
Still in line: deque(['Caleb'])
```

Made with Goodnotes

Big Idea

This lesson gives you practice tracing queues the same way the stack challenge gave you practice tracing stacks. The goal is to predict what popleft() returns, what remains in the queue, and how repeated removals behave.

The big rule is simple but strict: with a queue, the first item added is the first item removed. Every challenge is really testing whether you can keep that FIFO order straight.

Full Teaching Explanation

Queue challenges are about state over time. The first line creates the queue. Each append adds a new item at the rear. Each popleft removes the current front item. After a removal, the second item becomes the new front. That means you cannot solve queue problems by only looking at the final line of code. You have to trace the queue after each operation.

The lesson also introduces loops with queues. A while loop can keep removing items until the queue is empty. This is useful in real programs because queues often represent work waiting to be handled: messages, jobs, tickets, people, downloads, orders, or tasks. The program keeps processing until there is nothing left.

The danger is trying to remove from an empty queue. A while len(queue) > 0 pattern is safe because it only runs while at least one item exists. That condition protects the popleft() call.

Queue Challenge Mental Model

Pattern	Meaning	What to Watch
first = q.popleft()	Remove the oldest item	first stores the removed front value
second = q.popleft()	Remove the new oldest item	The queue already changed after first
while q:	Repeat while items remain	The loop stops when the queue is empty
len(q) > 0	Safety check	Prevents empty-queue removal

Code Example 1 - Two popleft Calls

Made with Goodnotes

```
from collections import deque

q = deque()
q.append(10)
q.append(20)
q.append(30)

first = q.popleft()
second = q.popleft()

print(first)
print(second)
print(q)
```

Expected output:

```
10
20
deque([30])
```

Line-by-line explanation:

- 10 enters first, so it is at the front.
- 20 enters second, behind 10.
- 30 enters third, at the rear.
- first = q.popleft() removes 10.
- second = q.popleft() removes 20, because 20 becomes the new front after 10 leaves.- The queue still contains 30.

Real English translation: Serve the first two people in line and show who is still waiting.

Code Example 2 - Draining a Queue

```
from collections import deque

q = deque()
q.append("first")
q.append("second")
q.append("third")

while q:
    item = q.popleft()
    print(item)
```

Expected output:

```
first
second
third
```

The while q loop keeps going as long as the queue has at least one item. Each loop removes one item from the front. The output appears in FIFO order because popleft() always removes the oldest remaining item.

Why the Loop Pattern Matters

Made with Goodnotes

A queue often represents work waiting to be processed. The loop pattern is how a program handles every item without knowing ahead of time how many items there will be. The code does not need to know whether the queue contains 3 items, 30 items, or 300 items. It can simply keep processing while the queue is not empty.

This is also why the lesson emphasizes safe queue logic. If the loop condition is correct, popleft() only runs when there is something available to remove. If the condition is missing or wrong, the code may try to remove from an empty queue and crash.

Common Mistakes and Helpful Tips

- Mistake: assuming popleft() always returns the same value. It returns the current front, and the front changes aftereach removal.
- Mistake: forgetting that while q becomes false when the queue is empty.
- Mistake: using append and pop together, which creates stack-like behavior instead of queue behavior.
- Mistake: tracing the items but not the order they entered.
- Helpful tip: label the front and rear before solving any queue trace.
- Memory hook: queues process the oldest waiting item first.

Notes

Big Idea

This lesson combines both structures. A queue controls the order of who gets processed first, and a stack can control the order of items attached to that person or task.

The point is not that one structure is always better. The point is that different parts of the same program may need different order rules.

Full Teaching Explanation

Stacks and queues answer different order questions. A queue answers, 'Who has been waiting the longest?' A stack answers, 'What was added most recently?' In real software, both questions can matter at the same time.

The lesson uses an adventure-style example. Heroes wait in a line, so the heroes are stored in a queue. The first hero in line should go first. But each hero has a backpack of items, and a backpack behaves like a stack. The item packed last is the item that comes out first.

This is a powerful mental model because it shows that data structures can be nested inside a bigger process. The outer process might be FIFO, while an inner collection behaves like LIFO. Professional programming is often about choosing the right structure for each part of the problem.

Made with Goodnotes

Choosing the Right Structure

Part of the Problem	Best Structure	Why
Heroes waiting their turn	Queue	First hero in line should go first
Items inside each backpack	Stack	Last packed item comes out first
Undo history	Stack	Most recent action is undone first
Ticket processing	Queue	Oldest ticket should usually be handled first

Code Example 1 - Queue of Heroes, Stack of Items

```
from collections import deque

heroes_line = deque()

heroes_line.append(("Aria", ["sword", "map"]))
heroes_line.append(("Bryn", ["crossbow", "torch", "key", "potion"]))
heroes_line.append(("Cora", ["gem", "shield", "axe"]))
print(heroes_line)

current_hero, backpack = heroes_line.popleft()
print("Current hero:", current_hero)
print("Starting backpack:", backpack)

while backpack:     item
= backpack.pop()
    print(current_hero, "uses", item)
```

Expected idea:

```
Current hero: Aria
Starting backpack: ['sword', 'map']
Aria uses map
Aria uses sword
```

Line-by-line explanation:

- heroes_line is a queue, so the first hero added is the first hero removed.
- Each hero is stored with a backpack list. That inner list acts like a stack.
- heroes_line.popleft() removes Aria because Aria is first in line.
- backpack.pop() removes Aria's last packed item first, so map is used before sword.

Real English translation: Let the first hero in line take a turn, then use that hero's backpack items from newest to oldest.

Made with Goodnotes

Code Example:

Python Run Submit code

```
1 from collections import deque
2
3 heroes_line = deque()
4
5 heroes_line.append(("Aria", ["sword", "map"]))
6 heroes_line.append(("Bryn", ["crossbow", "torch", "key", "potion"]))
7 heroes_line.append(("Cora", ["gem", "shield", "axe"]))
8
9 print(heroes_line)
```

Output

```
deque([('Aria', ['sword', 'map']), ('Bryn', ['crossbow', 'torch', 'key',
'potion']), ('Cora', ['gem', 'shield', 'axe'])])
```

Code Example 2 - Process Every Hero

```
from collections import deque

heroes_line = deque()
heroes_line.append(("Aria", ["sword", "map"]))
heroes_line.append(("Bryn", ["crossbow", "torch", "key", "potion"]))
heroes_line.append(("Cora", ["gem", "shield", "axe"]))

while heroes_line:     current_hero, backpack =
heroes_line.popleft()     print("Current hero:",
current_hero)

    while backpack:
item = backpack.pop()
        print(current_hero,   "uses",   item)
```

What this code demonstrates:

- The outer while loop is queue behavior. It processes heroes in the order they entered the line.
- The inner while loop is stack behavior. It processes each hero's items from last packed to first packed.
- The queue decides who goes next. The stack decides which item that hero uses next.

This is the real software-engineering lesson: one program can combine multiple data structures when different parts of the problem need different order rules.

Plain-English Algorithm

Made with Goodnotes

- While the hero line is not empty, remove the front hero from the queue.
- Give that hero a turn.
- While that hero's backpack is not empty, pop and use the top item.
- When the backpack is empty, move to the next hero in line.
- Stop when no heroes remain in the queue.

Code Example:

Python Run Submit code

```python
from collections import deque

heroes_line = deque()

heroes_line.append(("Aria", ["sword", "map"]))
heroes_line.append(("Bryn", ["crossbow", "torch", "key", "potion"]))
heroes_line.append(("Cora", ["gem", "shield", "axe"]))

print(heroes_line)


current_hero, backpack = heroes_line.popleft()
print("Current hero:", current_hero)
print("Starting backpack:", backpack)

while backpack:
    item = backpack.pop()
    print(current_hero, "uses", item)

```

Output

```
deque([('Aria', ['sword', 'map']), ('Bryn', ['crossbow', 'torch', 'key',
'potion']), ('Cora', ['gem', 'shield', 'axe'])])
Current hero: Aria
Starting backpack: ['sword', 'map']
Aria uses map
Aria uses sword
```

Made with Goodnotes

Common Mistakes and Helpful Tips

- Mistake: using one structure for everything. The hero line needs a queue, but the backpack needs a stack.
- Mistake: mixing up which operation belongs to which structure. popleft() is queue-front removal; pop() is stack-topremoval.
- Mistake: forgetting that nested data has layers. First you remove a hero from the queue, then you work throughthat hero's backpack.
- Mistake: using pop() on the queue of heroes. That would take the newest hero instead of the oldest waiting hero.
- Helpful tip: write the behavior in English before choosing operations. If the phrase is 'first in line,' think queue. If thephrase is 'last packed,' think stack.
- Memory hook: queue chooses the turn order; stack chooses the most recent item.

Key Takeaways

- Stacks follow LIFO: Last In, First Out.
- Queues follow FIFO: First In, First Out.
- append() can add to the rear of a queue or the top of a stack depending on how the structure is being used.
- pop() removes the last item from a list, so it fits stack behavior.
- popleft() removes the first/front item from a deque, so it fits queue behavior.
- A single program can use both structures when different parts of the problem require different order rules.

Notes

Code Example:

Python

Run Submit code

```
from collections import deque

heroes_line = deque()

heroes_line.append(("Aria", ["sword", "map"]))
heroes_line.append(("Bryn", ["crossbow", "torch", "key", "potion"]))
heroes_line.append(("Cora", ["gem", "shield", "axe"]))

print(heroes_line)


current_hero, backpack = heroes_line.popleft()
print("Current hero:", current_hero)
print("Starting backpack:", backpack)

heroes_line = deque()

heroes_line.append(("Aria", ["sword", "map"]))
heroes_line.append(("Bryn", ["crossbow", "torch", "key", "potion"]))
heroes_line.append(("Cora", ["gem", "shield", "axe"]))

while backpack:
    item = backpack.pop()
    print(current_hero, "uses", item)

while heroes_line:
    current_hero, backpack = heroes_line.popleft()
    print("== Hero:", current_hero, "==")
    while backpack:
        item = backpack.pop()
        print(current_hero, "uses", item)
```

Output

```
deque([('Aria', ['sword', 'map']), ('Bryn', ['crossbow', 'torch', 'key', 'potion']), ('Cora', ['gem', 'shield', 'axe'])])
Current hero: Aria
Starting backpack: ['sword', 'map']
Aria uses map
Aria uses sword
== Hero: Aria ==
Aria uses map
Aria uses sword
== Hero: Bryn ==
Bryn uses potion
Bryn uses key
Bryn uses torch
Bryn uses crossbow
== Hero: Cora ==
Cora uses axe
Cora uses shield
Cora uses gem
```

Made with Goodnotes

Week 2 Review Tools - Stacks and Queues

Master Framework

Week 2 is about order rules. A data structure is not just a container; it is a rule for what item can be added, accessed, or removed next. Stacks and queues both store collections, but they answer different order questions.

A stack asks: what was added most recently? A queue asks: what has been waiting the longest? That difference controls the entire week. If you can identify which end matters, the code usually becomes much easier to choose and trace.

- Stack = LIFO = Last In, First Out.
- Queue = FIFO = First In, First Out.
- Stack operation pair: push with append(), pop with pop().
- Queue operation pair: enqueue with append(), dequeue with pop(0) for a list or popleft() for a deque.

Why This Week Matters

Stacks and queues are small ideas with big consequences. Undo buttons, browser back history, function call stacks, print queues, task schedulers, customer service tickets, and event processing all depend on choosing the right removal order.

The programmer habit underneath the vocabulary is: before writing code, decide what item should leave next. If the newest item leaves next, use stack thinking. If the oldest waiting item leaves next, use queue thinking. If different parts of the program need different rules, combine the structures carefully.

One-Sentence Memory Hook

Stacks are piles. Queues are lines. pop() takes the newest from a stack. popleft() serves the oldest from a queue.

Made with Goodnotes

Stack Review - LIFO Thinking

Full Teaching Review

A stack is a collection where the most recently added item is the next one removed. The physical image is a pile of plates, books, folded towels, or boxes. You can easily reach the top item, but you cannot grab something from the middle without disturbing the stack.

In Python list practice, the top of the stack is the end of the list. Calling append() pushes a new item onto the top. Calling pop() removes and returns the current top item. After pop() runs, the stack has changed; the removed item is no longer inside the list.

The biggest stack skill is tracing state. You should be able to read a sequence of append() and pop() calls and predict both what value gets removed and what remains afterward.

Stack Operation Pattern

```
stack = []

stack.append("A")
stack.append("B")
stack.append("C")

removed = stack.pop()

print(removed)
print(stack)
```

Expected output:

```
C
['A', 'B']
```

Real English translation: Put A down first, then B, then C. Since C was added last, C is on top. pop() removes C and leaves A and B behind.

Stack Mistakes to Watch

- Thinking pop() removes the first item. It does not. On a list, pop() removes the last item by default.
- Forgetting that a pop operation changes the stack permanently.
- Trying to pop from an empty list without checking first.
- Calling a situation LIFO when the oldest item leaves first. That is queue/FIFO behavior, not stack behavior.

Queue Review - FIFO Thinking

Full Teaching Review

A queue is an ordered collection where the first item added is the first item removed. The physical image is a coffee shop line, food-truck line, checkout line, support ticket line, or pickup counter. New items join at the rear, and the next item served is taken from the front.

The front and rear are the only two directions you really need. The front is where items leave. The rear is where items enter. If the problem is about fairness, arrival order, or processing work in the order it arrived, it probably wants a queue.

In early examples, a regular list can imitate a queue with append() and pop(0). Later, deque becomes the better structure because popleft() expresses front removal clearly and handles queue-style removal more efficiently.

Made with Goodnotes

Queue Operation Pattern

```
from collections import deque

queue = deque()
queue.append("Sophia")
queue.append("CJ")
queue.append("Caleb")

served = queue.popleft()

print(served)
print(queue)
```

Expected output:

```
Sophia
deque(['CJ', 'Caleb'])
```

Real English translation: Sophia arrived first, so Sophia leaves first. CJ and Caleb are still waiting in the same order.

Queue Mistakes to Watch

- Using pop() on a queue. That removes from the rear and accidentally creates stack-like behavior.
- Forgetting to import deque before using deque().
- Calling popleft() when the queue is empty.
- Mixing up front and rear. New items enter at the rear; old items leave from the front.

Stack vs Queue Decision Guide

Core Comparison

Question	Stack Answer	Queue Answer
What item leaves next?	Newest item	Oldest waiting item
Order rule	Last In, First Out	First In, First Out
Physical model	Pile of plates	Line at a counter
Python removal	pop()	popleft() with deque, or pop(0) with list
Good for	Undo, browser back, call stack, nested items	Tickets, lines, print jobs, task processing
Common trap	Forgetting pop removes the last item	Using pop instead of popleft

How to Choose During a Quiz

- If the wording says most recent, latest, top, undo, last added, or last packed, think stack.
- If the wording says first arrived, fair order, line, waiting, ticket, request, or oldest task, think queue.
- If the code uses stack.pop(), expect the end of the list to disappear.
- If the code uses queue.popleft(), expect the front/left item to disappear.
- If the code combines a queue and a stack, identify the outer structure first, then trace the inner structure.

Made with Goodnotes

Out Loud Programmer Thinking

I do not start by memorizing the method name. I start by asking what behavior the real-world situation needs. If people should be served in arrival order, the structure is a queue. If the most recent action should be reversed first, the structure is a stack. After the behavior is clear, the Python method becomes easier to remember.

Trace Practice - Predict the State

Practice 1 - Stack Trace

```
stack = []
stack.append("red")
stack.append("blue")
stack.append("green")

first = stack.pop()
second = stack.pop()

print(first)
print(second)
print(stack)
```

Answer:

```
green
blue
['red']
```

Why: red entered first, blue entered second, and green entered last. A stack removes the newest item first, so green leaves, then blue, and red remains.

Practice 2 - Queue Trace

```
from collections import deque

q = deque()
q.append("T1")
q.append("T2")
q.append("T3")
q.append("T4")

first = q.popleft()
second = q.popleft()

print(first)
print(second)
print(q)
```

Answer:

```
T1
T2
deque(['T3', 'T4'])
```

Why: queues remove from the front. T1 entered first and leaves first. T2 becomes the new front and leaves second.

Weekly Review Question Breakdown

How the Review Questions Are Testing You

Made with Goodnotes

The weekly review is not really testing whether you can recite definitions in isolation. It is testing whether you can recognize order rules in real situations, map those rules to stack or queue vocabulary, and trace Python operations correctly.

For each question, underline the clue word. Words like top, recent, undo, and last added point toward stack/LIFO. Words like line, first arrived, waiting, support ticket, and fair order point toward queue/FIFO.

Question Type Guide

Question Type	What It Checks	Strategy
Real-world LIFO	Can you spot stack behavior?	Look for newest/top item leaving first.
Push operation	Do you know how stacks add items?	push means add to the top.
Pop trace	Can you predict the top after removal?	Label stack order before and after pop().
Undo behavior	Can you connect recent actions to stacks?	Undo removes the most recent action first.
Real-world FIFO	Can you spot queue behavior?	Look for first arrival served first.
Enqueue operation	Do you know where queues add?	enqueue adds to the rear/end.
Stack + queue blanks	Can you combine both rules?	Queue for turn order; stack for last-added inner items.
Support queue trace	Can you process tickets correctly?	Append in arrival order, then dequeue from the front.

Review Tip

When a question includes code, do not answer from vibes alone. Write the items in order, mark front/top, then cross out each removal. Yes, it feels extra. No, the quiz does not care about our feelings. Rude, but educational.

Common Exam Traps and Debug Thinking

Trap 1 - pop() vs pop(0) vs popleft()

These three removals look similar, but they do different things. pop() removes the last item from a list. pop(0) removes the first item from a list. popleft() removes the left/front item from a deque.

Expression	Removes	Behavior
items.pop()	Last item	Stack/LIFO
items.pop(0)	First item	Queue/FIFO with list
queue.popleft()	Left/front item	Queue/FIFO with deque

Trap 2 - Correct Structure, Wrong End

Made with Goodnotes

A lot of wrong answers come from using the right structure name but removing from the wrong end. A queue is only FIFO if the front item leaves first. A stack is only LIFO if the most recent item leaves first.

- If your queue removes from the rear, it is acting like a stack.
- If your stack removes from the bottom, it is not normal stack behavior.
- If you forget the empty check, your logic may work for normal examples but break on edge cases.

Debugging Checklist

- Step 1: Identify whether the problem is asking for LIFO or FIFO.
- Step 2: Label the top for stacks or front/rear for queues.
- Step 3: Trace each append, pop, pop(0), or popleft call in order.
- Step 4: After every removal, update the collection before reading the next line.
- Step 5: Check empty-collection cases before removing.

Week 2 Cheat Sheet

Syntax Reference

```
# Stack with a list
stack = []
stack.append("A")       # push onto top
stack.append("B")
top = stack.pop()       # removes "B"

# Queue with a deque
from collections import deque
queue = deque()
queue.append("A")       # enqueue at rear
queue.append("B")
front = queue.popleft() # removes "A"
```

Vocabulary Reference

Term	Meaning
LIFO	Last In, First Out. The newest item leaves first.
FIFO	First In, First Out. The oldest waiting item leaves first.
Stack	A collection that behaves like a pile. Use append and pop.
Queue	A collection that behaves like a line. Use append and popleft with deque.
Push	Add an item to the top of a stack.
Pop	Remove and return the top item of a stack.
Enqueue	Add an item to the rear of a queue.
Dequeue	Remove and return the front item of a queue.
deque	Python's double-ended queue structure from collections.

Final Week 2 Memory Sheet

Made with Goodnotes

If You Remember Nothing Else

- Stacks remove the most recently added item first.
- Queues remove the earliest added item first.
- append() adds to the end of a list or deque.
- pop() removes the end of a list, which fits stack behavior.
- popleft() removes the front/left side of a deque, which fits queue behavior.
- Always trace state after every removal. The collection changes.
- Always consider what happens if the structure is empty.

Final Concept Map

Week 2 can be reduced to one chain of ideas:

Data structures organize behavior. Stacks organize newest-first behavior. Queues organize oldest-first behavior. Python lists can model stacks easily. Python deque handles queues more cleanly. Real programs often combine structures when one part needs FIFO and another part needs LIFO.

Final Reflection

The deeper lesson is that programming is not just about storing data. It is about preserving the meaning of the data. If the order matters, the structure must protect that order. That is why stacks and queues show up everywhere: they let code model real-world rules in a reliable way.

Notes

CS102 - Week 3 Notes Outline and Study Guide

Table of Contents

Made with Goodnotes

What is inside

- Week 3 overview and master framework: algorithms, ordered steps, counting work, algorithmic complexity, Big O, comparing approaches, and safe input handling.
- Lesson 1 - Welcome to Algorithms
- Lesson 2 - Why We Need Algorithms in Real Life
- Lesson 3 - Turning Everyday Problems Into Step-by-Step Solutions
- Lesson 4 - Challenge: Algorithms
- Lesson 5 - Counting Work: How Many Steps Does an Algorithm Take
- Lesson 6 - Complexity: What It Means and Why It Matters
- Lesson 7 - Introduce the Big O Notation
- Lesson 8 - Analyzing Algorithms with Big O
- Lesson 9 - Challenge: Algorithm Analysis
- Lesson 10 - Comparing Two Solutions to the Same Problem
- Lesson 11 - Safe Algorithms: Handling Edge Cases and Bad Input
- Lesson 12 - Challenge: Advanced Algorithm Analysis
- Review tools, glossary, common mistakes, trace practice, and cheat sheet

Week 3 at a Glance

Week 3 is where algorithms stop sounding like a fancy computer science word and start becoming a way to think clearly. The week begins with the simplest idea: an algorithm is an ordered set of steps for solving a problem. Then it keeps raising the standard. Good algorithms are not just steps; they are clear, repeatable, precise, testable, and useful when the input gets bigger.

The early lessons use everyday examples because your brain already understands algorithms before you ever write code. Ordering pizza, brushing your teeth, washing clothes, checking a bank balance, making a sandwich, and sending a text message all involve a goal, a starting point, and steps that must happen in a sensible order. The course uses those familiar patterns to build the coding habit underneath them.

The second half of the week will move from writing steps to counting work. That is the bridge into complexity and Big O. Instead of only asking whether code gives the right answer, you begin asking how much work the code does as the problem grows. This is the beginning of algorithm analysis: comparing approaches by how they scale, not just whether they technically work once.

Memory hook: clear steps first, then count the work, then compare growth.

Skill**Skill**	**What it means**What it means	**Why it matters**Why it matters
Algorithm thinkingAlgorithm thinking	Break a task into clear ordered steps.Break a task into clear ordered steps.	Turns vague goals into something a computer or person canTurns vague goals into something a computer or person can follow. follow.
Pseudocode thinking	Write instructions in plain English before Python.	Lets you solve the logic before wrestling syntax goblins.
Pseudocode thinking Write instructions in plain English before Python. Lets you solve the logic before wrestling syntax goblins. Counting work Track how many times a step runs. Shows why more input often means more runtime.		
Counting work	Track how many times a step runs.	Shows why more input often means more runtime.
Complexity thinking	Describe how work grows as input grows.	Helps compare solutions that are both technically correct.
Complexity thinking Describe how work grows as input grows. Helps compare solutions that are both technically correct. Safe algorithm thinking Plan for empty input, missing values, and weird casePrevents crashes and makes code reliable.s.		
Safe algorithm thinking	Plan for empty input, missing values, and weird cases.	Prevents crashes and makes code reliable.

Made with Goodnotes

Big Idea

An algorithm is a clear set of ordered steps used to solve a problem. The important word is ordered. A random pile of advice is not an algorithm. A picture of the result is not an algorithm. A vague goal like "make pizza" is not an algorithm. The algorithm is the specific path from the starting point to the finished result.

Full Teaching Explanation

This lesson introduces algorithms in the friendliest way possible: by showing that you already use them constantly. When you order pizza for friends, you do not simply shout "pizza" into the universe and hope dinner materializes. You follow a sequence: count how many people are eating, ask what toppings they want, choose the size and number of pizzas, call or order from the store, and pay.

That ordered list matters because each step depends on earlier information. You cannot choose the right number of pizzas before you know how many people are eating. You cannot order accurately before you know toppings. You cannot pay before the order exists. The lesson is training you to see that solving problems is often about arranging the work in the right order.

In programming, an algorithm is the plan behind the code. Python syntax is how you express the plan, but the algorithm is the thinking underneath it. Two students can write different Python code and still use the same algorithm. They may name variables differently or format the code differently, but if the steps and decisions are the same, the algorithm is basically the same.

The key beginner mistake is thinking algorithms are automatically complicated. They are not. A simple recipe, a checkout process, or a set of directions can be an algorithm. Later, algorithms can become much more formal, but the foundation is still the same: a goal, an ordered process, and a way to know when the task is finished.

Examples and Mental Models

Situation	Is it an algorithm?	Why
Ordering pizza	Yes	There is a goal and a clear order of steps from count people to pay.
A photo of pizza	No	It shows the result, but not instructions for reaching it.
A friend saying "I like pizza"	No	It is information, not a procedure.
A numbered recipe	Yes	It tells you what to do first, next, and last.
A vague goal like "get dinner"	Not yet	It needs precise steps before it becomes an algorithm.

Out Loud Programmer Thinking

When I see a problem, I should ask: What is the goal? What information do I need first? What step must happen before another step? What decision points exist? What tells me I am done?

This habit matters because computers do not understand vague intention. They only follow instructions. The more precise the algorithm, the easier it becomes to turn the plan into code.

Made with Goodnotes

Pseudocode Example - Ordering Pizza

```
# Goal: order enough pizza for a group
count_people
ask_topping_choices
choose_pizza_sizes
choose_number_of_pizzas
place_order
pay_for_order
wait_for_delivery
```

Line-by-line explanation: the first line names the goal, and each later line is one action in the plan. This is pseudocode because it is not meant to run in Python yet. It is meant to make the algorithm clear before coding.

Terms, Definitions, and Why They Are Called That

- Algorithm: a step-by-step process for solving a problem. It is called an algorithm after historical mathematical procedures for calculation, but in beginner terms you can think of it as a reliable recipe for solving a task.
- Step: one action inside an algorithm. It is called a step because it moves you forward from where you are toward the result.- Order: the sequence in which steps happen. Order matters because changing the order can change the result or break the process.
- Procedure: a repeatable set of instructions. It is useful when the same task must be done more than once.
- Pseudocode: plain-English code-like planning. It is called pseudo-code because it resembles code but is not actual executable code.

Common Mistakes and Helpful Tips

- Mistake: thinking an algorithm must be written in Python. The algorithm is the plan; code is one way to express it.- Mistake: writing the goal instead of the steps. "Make a sandwich" is not enough; the algorithm explains how.
- Mistake: forgetting order. "Pay, choose toppings, call the store" is technically a list, but it is not a useful algorithm.
- Tip: use verbs at the start of steps: count, ask, choose, call, pay, print, check, return.
- Memory hook: an algorithm is not the wish. It is the route.

Notes

Made with Goodnotes

Big Idea

Algorithms matter because real-life tasks only become reusable, teachable, and programmable when they are broken into clear steps. A human can often infer missing steps, but a computer cannot. If the directions are vague, incomplete, or out of order, the result becomes unreliable.

Full Teaching Explanation

This lesson pushes the algorithm idea beyond pizza and into normal daily routines. Imagine trying to teach a robot how to brush its teeth. You cannot say, "just brush your teeth," because the robot has no hidden common sense. You have to describe the task in steps: go to the sink, pick up the toothbrush, put toothpaste on the brush, turn on the water, brush, rinse, and put everything back.

That example is funny because humans skip these details automatically. We use background knowledge. We understand that toothpaste belongs on the brush and that the brush goes in your mouth, not in your ear. Computers do not get that luxury. In programming, every important action must be represented somehow. Algorithms are how we make invisible human knowledge visible.

The lesson also uses examples like washing clothes, giving a weekly injection, and checking a bank balance on your phone. Each task has a starting point, a goal, and a repeatable process. If you do the steps in a different order, the result may be wrong or unsafe. For example, with laundry, you do not start the dryer before the clothes have been washed. With checking a bank balance, you do not check the balance before unlocking the phone and opening the app.

This is why algorithms help programming. Programs are built out of procedures. A login process, checkout flow, search feature, photo upload, bank transfer, medication reminder, or order tracker all require a sequence of precise actions. When you learn to describe real-life tasks algorithmically, you are learning to design code before you type code.

Everyday Algorithm Example - Checking a Bank Balance

```
# Goal: check a bank balance on a phone
unlock_phone
open_banking_app
enter_pin_or_use_face_id
tap_account
read_current_balance
close_app
```

This is not Python yet. It is an algorithm-style outline. Each line is a specific action, and the order matters. You cannot read the balance before the banking app is open and authenticated.

Question**Question**

Does the task have a goal?Does the task have a goal?

Can the task be repeated?Can the task be repeated?

Does order matter?Does order matter?

Are there hidden assumptions?Are there hidden assumptions?

Could a computer follow it?Could a computer follow it?

Why it matters Why it matters

Made with Goodnotes

Algorithms are used to reach a result, not merely describe something.Algorithms are used to reach a result, not merely describe something.	Real English "Check my bank balance." "Do laundry." "Give an injection." "Brush teeth."
A repeatable task can be written as a reusable procedure.A repeatable task can be written as a reusable procedure.	
If changing the order changes the outcome, algorithm thinking matters.If changing the order changes the outcome, algorithm thinking matters.	
Those assumptions may need to become explicit steps.Those assumptions may need to become explicit steps.	Programmer Thinking What app opens? What authentication is required? What account is selected? What output should appear?
The instructions must be specific enough for execution.The instructions must be specific enough for execution.	What goes first: sort, add soap, choose cycle, start washer, move to dryer, fold? What can go wrong?

Terms, Definitions, and Why They Are Called That

- Instruction: a specific command telling someone or something what to do. It is called an instruction because it directs action.- Sequence: steps arranged in order. It is called a sequence because one step follows another.
- Input: information the process needs before it can work. In a bank example, the input might be your PIN or login credentials.
- Output: the result produced by the process. In the bank example, the output is the account balance.
- Repeatable process: a task that can be done the same way again. This matters because code is usually written to handle repeated tasks, not one-time guesses.

Real English vs Programmer Thinking

Real English vs Programmer Thinking	What supplies are needed? What safety checks happen before, during, and after the process?
	What tiny steps does a robot need because it has no common sense?

Common Mistakes and Helpful Tips

- Mistake: assuming obvious human steps are obvious to the computer. They are not.
- Mistake: writing steps that are too large, like "prepare medicine" instead of naming what preparation requires.
- Mistake: ignoring the starting condition. A process can fail if the phone is locked, the washer is full, or supplies are missing.
- Tip: ask, "Could a very literal robot follow this?" If not, add more detail.

-- Mistake: assuming obvious human steps are obvious to the computer. They are not. Tip: if a step contains the word "and," it may need to be split into two separate steps.

-- Mistake: writing steps that are too large, like "prepare medicine" instead of naming what preparation requires. Memory hook: real-life algorithms reveal the hidden instructions behind everyday tasks.

- Mistake: ignoring the starting condition. A process can fail if the phone is locked, the washer is full, orsupplies are missing.
- Tip: ask, "Could a very literal robot follow this?" If not, add more detail.
- Tip: if a step contains the word "and," it may need to be split into two separate steps.
- Memory hook: real-life algorithms reveal the hidden instructions behind everyday tasks.

Made with Goodnotes

Big Idea

This lesson turns real-life tasks into algorithmic thinking by focusing on one clean rule: one action per line. When steps are too broad or smashed together, the algorithm becomes harder to follow and harder to translate into code.

Full Teaching Explanation

Lesson 3 takes the idea of algorithms and makes it more practical. It asks you to take an everyday task, like making hot chocolate, making a sandwich, or sending a text message, and break it into clear, ordered instructions. This is the same skill you will use when turning a programming problem into pseudocode.

The hot chocolate example starts with a vague goal: make hot chocolate. A better algorithm begins with concrete steps: get a mug, pour milk into the mug, heat the milk, add chocolate, stir, and serve. The sandwich example gets even more detailed. Instead of saying "make a sandwich," the algorithm says to get a plate, get bread, gather ingredients, put two slices of bread on the plate, apply condiments, add fillings, put the bread together, and eat.

The important idea is that each line should do one small thing. If a line says "get condiments, get meat, get cheese, and assemble the sandwich," it is doing too much. A programmer reading that line has to mentally unpack it. A computer would need those actions separated anyway. Good algorithms reduce that unpacking work by making the steps visible.

This is also where pseudocode becomes useful. Pseudocode is not about perfect Python syntax. It is about making the plan readable. Before you write a loop, condition, function, or data structure, you can sketch the behavior in plain English. That sketch helps you see missing steps, bad order, repeated work, and edge cases before the code gets involved and starts flinging syntax errors like confetti.

Messy Step vs Clean Step

Messy version	Cleaner algorithm version
Make a sandwich.	Get bread -> add filling -> close sandwich -> eat.
Get stuff and send the text.	Unlock phone -> open messages -> choose contact -> type message -> send.
Prepare hot chocolate.	Get mug -> pour milk -> heat milk -> add chocolate -> stir.
Fix the problem.	Identify error -> choose solution -> test solution -> verify result.

```
tap_create_message
choose_friend_name
type_message
review_message
press_send
```

Line-by-line explanation: every line starts with an action verb and represents one clear action. If the task were being turned into code, each line could eventually become a function call, a user action, or a step in a larger workflow.

Python-Flavored Example - Store Steps in a List

Made with Goodnotes

```
steps = [
    "get bread",
    "put cheese on bread",
    "add ham",
    "eat sandwich"
]

for step in steps:
print(step)
```

This example shows the connection between algorithms and code. The list stores the ordered steps. The loop processes each step one at a time. The algorithm is still the sequence; Python is just helping display or execute it.

Line-by-Line Explanation

- steps = [...] creates an ordered list of instructions.
- Each string inside the list is one action.
- for step in steps: moves through the algorithm one instruction at a time.
- print(step) displays the current instruction.
- Real English translation: "Go through the recipe and say each step in order."
- Out loud programmer thinking: "If I can describe the steps clearly, I can start turning them into code."

Terms, Definitions, and Why They Are Called That

- Pseudocode: a plain-English version of code logic. It is called pseudo-code because it looks code-like but is not strict Python.
- Granularity: how large or small each step is. Fine-grained steps are small and specific; coarse-grained steps are broad.
- Action verb: a verb that tells the reader what to do, such as get, open, choose, type, check, or print.
- Sequential order: an order where steps happen one after another. It matters because later steps often depend on earlier ones.
- Trace: to walk through steps in order and observe what happens. It is called tracing because you follow the path of execution.

Common Mistakes and Helpful Tips

- Mistake: combining several actions into one line. Split "get bread and add cheese" into separate steps if the details matter.
- Mistake: starting in the middle of the task. Always define the starting condition.
- Mistake: writing steps out of order. A good test is to ask whether a literal robot could follow the instructions without guessing.- Tip: use one verb per step whenever possible.
- Tip: after writing steps, trace them out loud and look for missing actions.
- Memory hook: if the summary makes you say "wait, how?", the algorithm needs more steps.

Notes

Made with Goodnotes

Lesson 4 - Challenge: Algorithms

Big Idea

This challenge lesson checks whether you can recognize an algorithm when you see one. An algorithm is not just a topic label, a random list, or a vague goal. It is a clear, ordered set of steps that can be followed to solve a problem or complete a task.

Full Teaching Explanation

The point of this lesson is to strengthen your algorithm instinct before the course gets more technical. The screenshots walk through everyday examples like laundry, making a sandwich, sending a text, boiling water for tea, brushing your teeth, and helping a friend make a peanut butter and jelly sandwich. Those examples may look simple, but they train the same thinking used in code: break the task into ordered instructions that a person or computer could follow.

A strong algorithm has three important qualities. First, it is ordered. If the steps happen in the wrong order, the result can fail. You cannot press send before typing the message, and you cannot start the washing machine before putting clothes and soap inside. Second, each step is actionable. A good step tells the doer what to do, not just what the final goal is. Third, the steps are clear enough that someone else could follow them without guessing.

This is why the lesson keeps pushing you toward pseudocode style: numbered steps, verb-first instructions, and one clear action per line. Pseudocode is not Python yet. It is a planning language that lets you prove the logic makes sense before you worry about exact syntax.

Algorithm or Not? Everyday Challenge Examples

Example	Is it an algorithm?	Why
Put clothes in, add soap, choose cycle, start, wait, rinse, finish	Yes	It is ordered, step-by-step, and tells exactly what to do.
A picture of a sandwich	No	It shows the goal, but it does not give instructions.
TEXT written on paper	No	It is a word, not a procedure.
Open chat, choose friend, type message, press send	Yes	It gives a repeatable process for sending a text.
Brushing teeth is good for you	No	That is a statement, not a sequence of steps.

Pseudocode Example - Making a Sandwich

Made with Goodnotes

```
1. Get bread.
2. Put cheese on the bread.
3. Add ham.
4. Eat the sandwich.
```

Line-by-Line / Real English Translation

This is intentionally plain English, but it behaves like an algorithm because each line is a small action and the steps happen in order. The sandwich example also shows why vague or combined steps can be a problem. A line like 'make the sandwich' is too broad because it hides the work inside one giant step.

Real English translation: start with the materials, perform one action at a time, then reach the final result. Out loud programmer thinking: 'Can another person follow this without asking what I meant?'

Terms, Definitions, and Why They Are Called That

- **Algorithm:** a clear set of steps for solving a problem. It is called an algorithm because it names the procedure, not just the answer.
- **Pseudocode:** plain-English planning code. It is called pseudo-code because it looks code-like but is not meant to run in Python.
- **Instruction:** one action the doer can perform. In algorithms, instructions should usually start with a verb such as get, open, choose, type, add, or print.
- **Sequence:** the order in which steps happen. A sequence matters when changing the order changes the result.
- **Ambiguity:** wording that can be interpreted multiple ways. Ambiguous instructions create bugs because the computer cannot guess what you meant.

Common Mistakes and Helpful Tips

- **Mistake:** writing the goal instead of the steps, like "clean the room" or "make sandwich." The goal is not the algorithm; the algorithm is how to get there.
- **Mistake:** putting too many actions on one line. If a tiny robot could get confused, split the step.
- **Mistake:** forgetting order. Some tasks, like texting or doing laundry, only work if the setup steps happen before the action step.
- **Tip:** read the algorithm out loud. If it sounds like a real set of instructions, you are close.
- **Memory hook:** clear steps, correct order, one action at a time.

Notes

Lesson 5 - Counting Work: How Many Steps Does an Algorithm Take?

Big Idea

Made with Goodnotes

This lesson introduces the idea that algorithms do work, and we can count that work. The main pattern is simple: if a loop runs once for each item in a collection, then more items means more loop steps.

Full Teaching Explanation

The lesson starts with a postcard or stamping example: if you have 3 postcards and do 2 stamping moves for each postcard, the total work grows as the number of postcards grows. Then it moves into code with a list of names. The computer goes through the list one name at a time. Each loop pass is one unit of work for the repeated part of the algorithm.

This is a major bridge between normal coding and algorithm analysis. Beginners often count only the lines of code they see. But computers do not run a loop line just once. A loop body runs once per item. If the list has 3 names, the body runs 3 times. If the list has 6 names, the same body runs 6 times. The code did not get longer, but the amount of work changed because the input got bigger.

That idea matters because real programs rarely process only 3 cute example items. They process photos, contacts, orders, messages, users, records, or search results. A tiny loop can be harmless with a small input and slow with a huge input. Counting work helps you predict how an algorithm will behave before it becomes a problem.

Code Example 1 - Counting One Step Per Name

```
names = ["Ana", "Ben", "Leo"]
steps = 0

for name in names:     steps
= steps + 1     print(name)
print("Total steps:", steps)
```

Line-by-Line / Real English Translation

Line-by-line explanation: **names** holds the input collection. **steps = 0** starts the counter before any work happens. **for name in names:** loops once per item. Inside the loop, **steps = steps + 1** records one more pass through the repeated work.

Real English translation: 'For every name in the list, count one step and print the name.' With Ana, Ben, and Leo, the loop runs 3 times, so the final step count is 3.

How the Step Count Changes

Input size	Loop body runs	Final step count	What it teaches
3 names	3 times	3	One pass per item.
6 names	6 times	6	Doubling the list doubled the loop work.
5 numbers	5 times	5	A single loop over a list grows with the list.
8 items	8 times	8	The number of loop steps follows the input size.

Made with Goodnotes

Code Example 2 - Same Algorithm, Bigger Input

```
names = ["Ana", "Ben", "Leo", "Mia", "Ray", "Zoe"] steps =
0

for name in names:     steps =
steps + 1     print(name)
print("Total steps:", steps)
```

Line-by-Line / Real English Translation

The instructions are the same as the earlier code. The only thing that changed is the input list. Because the loop still runs once for each item, 6 names means 6 passes through the loop body. This is the key idea: the work grows when the input grows.

Out loud programmer thinking: 'I am not counting how many lines are written. I am counting how many times the repeated part runs.'

Terms, Definitions, and Why They Are Called That

- **Step count:** the number of basic actions an algorithm performs. It is called step count because we are counting the work step-by-step.
- **Input size:** how much data the algorithm receives, such as the number of names, numbers, photos, or records. It matters because bigger input often creates more work.
- **Loop iteration:** one complete pass through a loop body. It is called iteration because the loop repeats the same pattern again and again.
- **Linear growth:** work that grows in direct proportion to input size. If the list doubles and the work doubles, that is linear growth.
- **Counter variable:** a variable used to track how many times something happened, like **steps** in the examples.

Common Mistakes and Debug Thinking

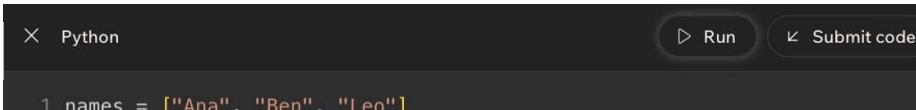
- **Mistake:** counting the loop code line only once. The loop header appears once, but the loop body runs once per item.
- **Mistake:** forgetting to set the counter to 0 before the loop starts.
- **Mistake:** assuming all algorithms grow the same way. Some grow slowly, some grow quickly, and some explode with bigger inputs.
- **Debug tip:** print the counter inside the loop if you want to see it grow one pass at a time.
- **Memory hook:** number of items in the list = number of loop steps for a simple one-pass loop.

Notes

Made with Goodnotes

Made with Goodnotes

Code Example:



Lesson 6 - Complexity: What It Means and Why It Matters

Big Idea

Complexity describes how the amount of work an algorithm does grows as the problem gets bigger. Two algorithms can both be correct, but one may stay fast as the input grows while the other becomes painfully slow.

Full Teaching Explanation

This lesson takes the step-count idea from Lesson 5 and zooms out. Instead of asking, 'How many steps does this exact tiny example take?' complexity asks, 'What happens when the problem gets bigger?' That is the question that matters in real programs, because your code may handle 5 contacts, 500 photos, or 50,000 records depending on the situation.

The lesson uses a contact-list story. One approach checks each name once until it finds the person. Another approach keeps re-checking pairs of contacts or doing extra repeated comparisons. Both approaches may eventually find the right answer, but the second one does far more unnecessary work as the list grows. With a few contacts, both feel fine. With thousands of contacts, the difference becomes obvious.

This is the heart of algorithmic complexity. Correctness tells us whether the algorithm gets the right answer. Complexity tells us how the work grows while getting that answer. A program can be correct and still feel slow, laggy, or frustrating if the algorithm does too much work for the input size.

One Scan vs Many Re-Checks

Number of contacts	Fast method: check each once	Slow method: compare many pairs	What happens
5	5 checks	10 checks	Both feel fast.
50	50 checks	1,225 checks	The slow method already does much more work.
500	500 checks	124,750 checks	The slow method starts to feel heavy.
5,000	5,000 checks	12,497,500 checks	The slow method explodes as the input grows.

Pseudocode Comparison - Same Goal, Different Work

```
# Approach A: one scan
for contact in contacts:
if contact == target:
return contact

# Approach B: repeated re-checking
for contact in contacts:     for
other_contact in contacts:
        compare(contact, other_contact)
```

Line-by-Line / Real English Translation

Made with Goodnotes

Approach A has one loop over the contacts. If the list gets bigger, the work grows in a direct, predictable way. Approach B has a loop inside another loop, so each contact can cause many more checks. That nested repeated work grows much faster.

Real English translation: 'Your way' checks each contact once. The slow way keeps comparing contacts with other contacts, even when those extra comparisons may not help. Both can be correct, but one wastes far more work as the list grows.

Why Complexity Matters in Real Life

Complexity is why some apps feel snappy and others feel like they are thinking about their life choices. A game might work fine with a few players, then lag when there are many players. A messaging app might search quickly through a small chat history but drag through years of messages. A photo app might load one album quickly but struggle when scanning thousands of images.

The user experience is often the visible symptom of algorithmic complexity. You do not always see the loop, scan, search, or comparison happening behind the scenes. You just feel the result: fast load, slow search, lag, freezing, or delay.

Terms, Definitions, and Why They Are Called That

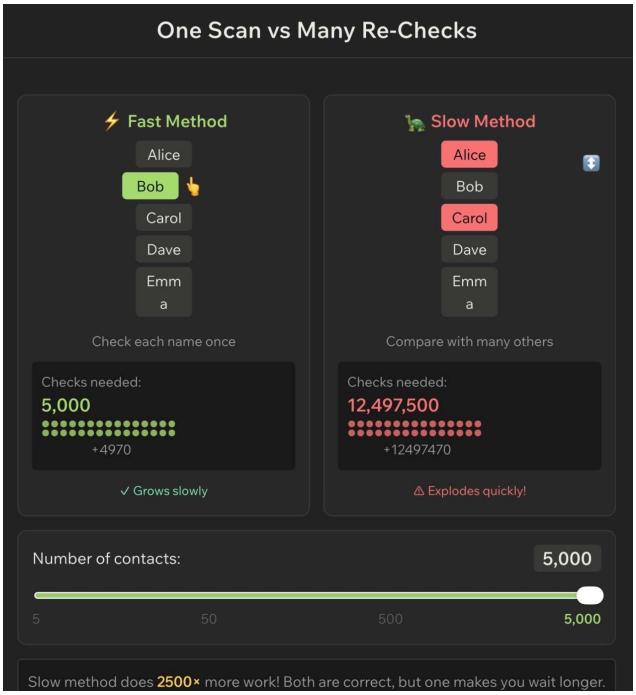
- **Algorithmic complexity:** how the amount of work grows as the problem gets bigger. It is called complexity because it describes the growth behavior, not just one exact step count.
- **Growth rate:** the pattern of how work increases. If input doubles and work doubles, the growth is gentle; if input doubles and work quadruples, it grows much faster.
- **Scalable:** able to handle bigger input without becoming unreasonable. Scalable code keeps working well as data grows.
- **Efficient:** doing the needed job without wasting unnecessary work. Efficiency is about using fewer steps, time, or resources for the same result.
- **Nested loop:** a loop inside another loop. Nested loops often create much faster-growing work because repeated work happens inside repeated work.

Common Mistakes and Helpful Tips

- **Mistake:** thinking correct code is automatically good enough. Correct code can still be too slow for large inputs.
- **Mistake:** focusing only on exact seconds. Complexity cares more about the pattern of growth than one measured time on one machine.
- **Mistake:** ignoring input size. A solution that feels fine with 5 items may fall apart with 5,000 items.
- **Tip:** ask, "When the input grows, does the work grow slowly, directly, or explosively?"
- **Memory hook:** complexity = how work grows when the problem grows.

Made with Goodnotes

Notes



Lesson 7 - Introduce the Big O Notation

Big Idea

Big O notation is a way to describe the **shape of growth** in an algorithm. It does not try to predict the exact number of seconds a program will take. Instead, it asks: as the input gets bigger, how does the amount of work grow?

This lesson connects the earlier idea of counting steps to a cleaner language programmers use for comparison. Once you can say that one algorithm is O(n) and another is O(n^2), you can explain why both may work on small data, but one becomes painful when the data gets larger.

Made with Goodnotes

Full Teaching Explanation

Big O is useful because exact timing is noisy. One computer may run faster than another, one phone may be older, and one app may be busy with background work. If we only measure seconds, the answer changes depending on the device. Big O ignores those machine-specific details and focuses on the pattern: does the work stay about the same, grow one step per item, grow as pairs of items, or explode much faster?

The letter **n** usually means the input size. If you are searching a list of contacts, n is the number of contacts. If you are looping through photos, n is the number of photos. If you are comparing every item with every other item, the number of checks may grow as n times n, which becomes O(n^2).

A helpful mental model is a growth meter. If adding more input barely changes the work, the algorithm is close to constant time. If doubling the input roughly doubles the work, it is linear. If doubling the input makes the work around four times larger, you are probably looking at quadratic growth. Big O is the vocabulary for naming those growth patterns.

Types of Measurement

Type	Meaning	Example
Best case	The easiest possible input. The algorithm gets lucky.	The first contact you check is the one you need.
Worst case	The hardest normal input. The algorithm has to do the most work.	The contact is last, or not found at all.
Average case	The expected middle-ground behavior across many normal inputs.	Sometimes the contact is near the front, sometimes near the end.

Core Big O Shapes

Big O	Called	Plain-English Meaning	Tiny Example
O(1)	constant time	Same amount of work no matter how many items exist.	Read the first item in a list; turn on a light.
O(log n)	logarithmic time	Work shrinks by cutting the problem down, often in half.	Binary search in a sorted list.
O(n)	linear time	Work grows one step for each item.	Loop through every name once.
O(n log n)	linearithmic time	Work grows a little faster than linear because items are split and recombined.	Many efficient sorting algorithms.
O(n^2)	quadratic time	Work grows as pairs of items, often from nested loops.	Compare every person with every other person.
O(2^n)	exponential time	Work doubles with each added choice.	Trying every yes/no combination.

Quick wrap-up for you:

- `n` = how many items the algorithm works on.
- **Big O** = how the *amount of work* grows when `n` grows (the shape, not the exact time).
- **O(1)** → flat line (same work no matter `n`).
- **O(n)** → straight line (double `n`, about double work).
- **O(n^2)** → curvy hill (`n` × `n` work).
- **O(2^n)** → rocket (explodes super fast).

Code Example 1 - Constant Time O(1)

Made with Goodnotes

```
# Same number of steps no matter how large the list
is. names = ["Ana", "Ben", "Leo", "Mia"] first_name
= names[0] print(first_name)
```

Line-by-line explanation: the list could hold 4 names or 4 million names, but this code still asks for index 0 one time. The amount of work doesLine-by-line explanation: the list could hold 4 names or 4 million names, but this code still asks for index 0 one time. The amount of work does not not grow with n, so the growth class is O(1).grow with n, so the growth class is O(1).

Code Example 2 - Linear Time O(n)

```
# One pass through the list.
names = ["Ana", "Ben", "Leo",
"Mia"] for name in names:
print(name)
```

Line-by-line explanation: the loop body runs once per name. If there are 4 names, it runs 4 times. If there are 400 names, it runs 400 times.Line-by-line explanation: the loop body runs once per name. If there are 4 names, it runs 4 times. If there are 400 names, it runs 400 times. The The work grows in a straight-line pattern with n, so this is O(n).work grows in a straight-line pattern with n, so this is O(n).

Code Example 3 - Quadratic Time O(n^2)

```
# Every item is compared with every other
item. names = ["Ana", "Ben", "Leo"] for
first in names:     for second in names:
        print(first, second)
```

Line-by-line explanation: the outer loop runs n times, and for each outer loop, the inner loop also runs n times. That gives n * n totalLine-by-line explanation: the outer loop runs n times, and for each outer loop, the inner loop also runs n times. That gives n * n total combinations. combinations. This is why nested loops over the same collection often mean O(n^2).This is why nested loops over the same collection often mean O(n^2).

Terms, Definitions, and Why They Are Called That

- **Big O notation**: a notation for describing how algorithmic work grows. It is called Big O because it describes the order of growth, not the exact count.
- **n**: the input size. It is usually the number of items the algorithm must handle.
- **Constant time**: work that stays the same size. It is called constant because it does not change when n changes.
- **Linear time**: work that grows in a straight-line relationship with n. Double the items, roughly double the work.
- **Quadratic time**: work that grows like n squared. It is called quadratic because the exponent is 2.
- **Dominant term**: the fastest-growing part of the expression. Big O keeps this part because it controls the behavior for large inputs.

Common Mistakes and Helpful Tips

- Mistake: thinking Big O means exact seconds. It does not; it describes growth pattern.
- Mistake: treating n like a fixed number. n means input size, so it changes with the problem.
- Mistake: assuming all loops are automatically bad. A single loop is usually normal linear work.
- Mistake: ignoring nested loops. A loop inside a loop over the same input often changes the shape dramatically.
- Tip: ask what happens when the input doubles. If work doubles, think O(n). If work quadruples, think O(n^2).
- Memory hook: Big O is not a stopwatch. Big O is a growth meter.

Notes

Lesson 8 - Analyzing Algorithms with Big O

Made with Goodnotes

Big Idea

Lesson 8 turns Big O from vocabulary into a reading skill. Instead of only recognizing the labels O(1), O(n), and O(n^2), you practice looking at code and deciding which growth pattern it matches.

The main move is to focus on the parts of the code that grow with the input. Fixed setup lines, fixed print statements, and constant multipliers matter in real programs, but they do not usually change the Big O class. Big O keeps the fastest-growing part because that is the part that dominates when n gets large.

Full Teaching Explanation

When you analyze code, do not start by counting every tiny operation forever. Start by identifying the shape. A plain statement like **total = 0** is O(1) because it runs once. A loop over a list is usually O(n) because it runs once for each item. A loop inside another loop is often O(n^2) because the inner loop repeats for every pass of the outer loop.

The course examples use simple step counts to build intuition. If a list has 3 names, a loop runs 3 times. If the list grows to 6 names, the loop runs 6 times. That is linear growth. But if each item has to be compared with every other item, the work grows much faster because every new item creates many new pair checks.

Big O also drops details that do not affect the long-term shape. In 5n + 7, the +7 is fixed extra work and the 5 is a constant multiplier, so the expression becomes O(n). In n^2 + 5n + 20, the n^2 term grows faster than the linear and constant parts, so the whole expression becomes O(n^2). This is the "dominant term" rule.

Rule	What to Do	Example
Ignore constants	Remove fixed multipliers because they do not change the growth shape.	7n becomes O(n).
Drop lower-order terms	Keep the fastest-growing term and remove the smaller ones.	n^2 + n becomes O(n^2).
Single loop	If the loop runs once per item, it is usually linear.	for item in items: print(item) is O(n).
Sequential loops	If loops happen one after another, add them first, then simplify.	O(n) + O(n) becomes O(n).
Nested loops	If one loop runs inside another over the same input, multiply the work.	n * n becomes O(n^2).
Worst-case thinking	When unsure, describe the upper bound of work the algorithm might need.	Searching may need the whole list if the item is last.

Code Example 1 - Fixed Work Is O(1)

```
print("hello")
print("hey")
print("again")
```

Line-by-line explanation: these three print lines run one time each. They do not depend on n. Whether your data set has 10 items or 10,000 items, this exact code still runs three print statements, so it is O(1).

Code Example 2 - One Loop Is O(n)

```
def print_items(items):
for item in items:
print(item)
```

Line-by-line explanation: the loop runs once per item. If items has 5 values, print runs 5 times. If items has 500 values, print runs 500 times. The work grows with n, so the function is O(n).

Code Example 3 - Two Separate Loops Are Still O(n)

```
def print_twice(items):
for item in items:
print(item)

    for item in items:
print(item)
```

Line-by-line explanation: the first loop is n steps and the second loop is n steps, so the raw count is 2n. Big O drops the constant multiplier 2, leaving O(n). The code does twice as much work in real life, but the growth shape is still linear.

Code Example 4 - Nested Loops Are O(n^2)

```
def print_pairs(items):
for first in items:
for second in items:
            print(first, second)
```

Line-by-line explanation: for each value in the outer loop, the inner loop runs through the whole list. If n is 3, that is 9 pair checks. If n is 10, that is 100 pair checks. The total grows like n * n, so the function is O(n^2).

Code Shape Quick Reference

Code Shape	How to Read It	Big O
A few fixed lines	Runs the same number of steps no matter input size.	O(1)
One loop over a list	Runs once for each item.	O(n)
Two loops one after another	Runs n + n steps, then constants are dropped.	O(n)
Loop inside a loop	Runs n times n steps.	O(n^2)
Expression 5n + 7	Drop the 5 and 7, keep n.	O(n)
Expression 2n^2 + 4n + 100	Drop constants and lower-order terms, keep n^2.	O(n^2)

Terms, Definitions, and Why They Are Called That

- **Analysis**: the process of breaking code down to understand how much work it does. It is called analysis because you are examining the structure, not just running it.
- **Step count**: a simplified count of operations. It helps reveal whether work grows slowly or quickly.
- **Constant**: a fixed number that does not change with input size, like 7 or 100.
- **Lower-order term**: a slower-growing part of an expression, like n inside n^2 + n.
- **Nested loop**: a loop inside another loop. It is called nested because one structure sits inside the other, like boxes inside boxes.
- **Dominant term**: the term that eventually matters most as n gets large. It dominates because it outgrows the others.

Common Mistakes and Debug Thinking

Made with Goodnotes

- Mistake: keeping every constant in the final Big O answer. Big O simplifies growth classes.
- Mistake: saying two separate loops are O(n^2). Separate loops add; nested loops multiply.
- Mistake: assuming every nested loop is automatically bad. Sometimes the input is small enough, but the growth pattern is still important.
- Mistake: choosing O(1) just because the code looks short. If a loop depends on input size, it grows with n.
- Debug tip: before choosing the Big O, mark which lines run once, which lines run once per item, and which lines run once per pair.- Memory hook: code shape first, exact count second, dominant term last.

Mini Practice - Name the Growth

Question	Reasoning	Answer
for x in items: print(x)	One pass through the collection.	O(n)
print("start"); print("done")	Fixed number of steps.	O(1)
for a in items: for b in items: print(a, b)	Every item pairs with every item.	O(n^2)
3n + 20	Drop constant multiplier and fixed extra work.	O(n)
n^2 + 4n + 100	n^2 grows fastest	O(n^2)

Notes

Made with Goodnotes

Big Idea

This challenge is about proving that you can look at small Python snippets and name the growth pattern. Instead of only asking "what does this code print," the challenge asks, "how much work does this code do as the input gets bigger?" That is the difference between running code and analyzing code.

Full Teaching Explanation

Algorithm analysis is the skill of looking beneath the surface of code. Two programs can both be correct, but one may stay fast as the list grows while the other becomes painfully slow. In this lesson, you practice recognizing the common shapes: fixed work, one pass through a list, nested passes, and expressions where one term eventually dominates the others.

The most important habit is to identify what depends on the input size. If a line always runs once, it is constant work. If a loop runs once for every item in the list, the work grows with the list, so it is linear. If one loop runs inside another and both depend on the same input, the work usually grows like pairs of items, which is quadratic. If you see an expression like n^2 + 12n, the n^2 part eventually dominates, so the simplified Big O is O(n^2).

The challenge questions also teach you to avoid over-counting. Big O is not asking for the exact number of steps. It is asking for the growth shape. That means constants and smaller terms get dropped after you understand what they mean. This does not mean constants never matter in real apps; it means Big O is zoomed out enough to compare how algorithms scale.

Code Example - One Loop Is O(n)

```
items = [1, 2, 3, 4,
5] for n in items:
print(n)
```

Line-by-line explanation: the list has five values here, so the print statement runs five times. If the list had 500 values, it would run 500 times. The work grows in direct proportion to the number of items, so this is O(n).

Code Example - Fixed Prints Are O(1)

```
print("hello")
print("hello again")
print("one more hello")
```

Line-by-line explanation: there are exactly three print statements. That number does not change when input size changes, because there is no input-driven loop. Big O ignores the fixed constant 3, so this is O(1).

Made with Goodnotes

Code Example - Nested Loops Are O(n^2)

```
for i in range(n):
for j in range(n):
print(i, j)
```

Line-by-line explanation: for each value of i, the inner loop runs through every value of j. If n is 4, there are 16 prints. If n is 10, there are 100. That n times n pattern is O(n^2).

Challenge Pattern Match

Code Shape or Expression	Raw Thinking	Final Big O
One line that reads numbers[0]	Looks at one item no matter how long the list is.	O(1)
for item in items	One pass through all items.	O(n)
for i in items: for j in items	Every item is paired with every item.	O(n^2)
n^2 + 12n	n^2 grows faster than 12n as n gets large.	O(n^2)
7n + 30	Drop the constant multiplier and fixed add-on.	O(n)
Five print statements	Same amount of work every time.	O(1)

How to Answer These Questions Without Panicking

- **Step 1:** Find the part that repeats. If nothing repeats and no input-driven helper is hiding, it may be O(1).
- **Step 2:** Ask what controls the repeat count. A loop over a list usually depends on n.
- **Step 3:** Check whether loops are separate or nested. Separate loops add; nested loops multiply.
- **Step 4:** Simplify the expression. Drop constants and keep the fastest-growing term.
- **Step 5:** Match the shape: flat, straight line, curve, or explosion.

Common Mistakes and Helpful Tips

- **Mistake:** choosing O(1) just because the code looks short. A short loop can still run n times.
- **Mistake:** choosing O(n^2) for two loops that happen one after another. That is O(n) + O(n), which simplifies to O(n).
- **Mistake:** keeping n^2 + 12n as the final answer. In Big O, simplify to the dominant term.
- **Tip:** write a tiny input count: if n = 3, how many times does the key line run?
- **Memory hook:** count the repeating work, then simplify the growth.

Notes

Made with Goodnotes

Big Idea

This lesson shows why "it works" is not the finish line. Two solutions can produce the same answer, but one may be faster, cleaner, easier to explain, or safer to maintain. Algorithm analysis gives you a way to compare those choices instead of choosing based on vibes alone - although, tragically, vibes remain undefeated in group projects.

Full Teaching Explanation

The lesson compares solutions to the same task so you can see the trade-off between runtime and readability. One version may loop through a list once and keep a running total. Another version may use nested loops or unnecessary extra work. Both might eventually calculate the correct answer, but their growth patterns are different.

For small input, the difference may not feel dramatic. If a list has three items, O(n) and O(n^2) may both finish instantly. The difference appears when the input grows. With 1,000 items, an O(n) solution might do about 1,000 pieces of work, while an O(n^2) solution might do about 1,000,000 pair checks. That is why Big O matters before the code becomes painfully slow.

This lesson also adds nuance: faster is not the only measurement. If two solutions have the same Big O, the clearer one is usually better. Readability matters because future-you, classmates, teammates, and instructors need to understand the logic. A clever trick that technically works but nobody can explain is often worse than a straightforward solution with the same growth class.

Solution A - Clean Single Loop

Made with Goodnotes

```
def sum_list(nums):
    total = 0     for n
in nums:         total =
total + n     return
total
```

Line-by-line explanation: total starts at 0, then each item is added once. The loop runs n times, so this is O(n). It is also readable because the variable names match the task: sum the list.

Solution B - Nested Loop Version

```
def sum_list_slow(nums):     total
= 0     for i in range(len(nums)):
for j in range(len(nums)):
total = total + nums[i]     return
total
```

Line-by-line explanation: the outer loop runs n times and the inner loop also runs n times for each outer step. That creates n * n work, or O(n^2). It is harder to read and does far more work than needed.

Comparing Two Correct Solutions

Question	Solution A: Single Loop	Solution B: Nested Loop
Does it solve the problem?	Yes, it can total the list.	It may produce a result, but it does unnecessary repeated work.
Growth pattern	O(n) - one pass.	O(n^2) - pair-style repeated work.
Readability	Clear: total each number once.	Confusing: why loop over every pair?
Best use	Beginner-friendly and efficient.	Mostly useful as a warning example.
Main lesson	Prefer clear, direct work.	Avoid repeated work unless the problem truly requires it.

Readability vs Speed

A solution can be faster but harder to read, or slower but easier to understand. The best answer depends on the situation. For beginner CS102 work, the first question is usually: can you explain it clearly? After that, ask whether the Big O is reasonable. If one solution is both clearer and faster, choose that one and give yourself a tiny parade.

If two solutions have the same Big O, readability may decide the winner. For example, a clean single loop and a clever loop trick might both be O(n). If the clean version is easier to understand and less likely to break, it is usually the better choice. Big O tells you about growth, but it does not replace judgment.

Made with Goodnotes

Same Big O, Different Readability

```
def count_evens_v1(nums):
count = 0     for n in nums:
if n % 2 == 0:
count = count + 1     return
count


def count_evens_v2(nums):     return sum(1
for n in nums if n % 2 == 0)
```

Line-by-line explanation: both versions check each number once, so both are O(n). Version 1 is more beginner-readable because it shows the count changing step by step. Version 2 is shorter, but it hides more Python knowledge inside one line.

Decision Guide for Choosing a Solution

Decision Factor	Ask Yourself	What to Prefer
Correctness	Does it always produce the right answer?	Correct first. Fast wrong code is still wrong code.
Growth	How does work change as input grows?	Lower growth when input can get large.
Readability	Can I explain this to a beginner?	Clear code unless speed truly requires complexity.
Edge cases	Does it handle empty or weird input?	Safe code that does not crash unnecessarily.
Maintainability	Would future-me understand this next week?	Names and structure that reveal the logic.

Common Mistakes and Helpful Tips

- **Mistake:** assuming the shortest code is always the best code. Short can be elegant, but short can also be cryptic.
- **Mistake:** comparing exact steps only for tiny lists. Growth matters most when input gets bigger.
- **Mistake:** ignoring readability because one solution feels more advanced. Advanced code should still be explainable.
- **Tip:** when comparing two solutions, write: correctness, Big O, readability, edge cases.
- **Memory hook:** first make it right, then make it clear, then make it faster when needed.

Notes

Made with Goodnotes

Big Idea

A safe algorithm does not only work when the input is perfect. It plans for edge cases: empty lists, missing values, zero denominators, weird types, or unexpected user input. This lesson is about writing code that fails less dramatically - because code throwing a tantrum is apparently not considered a feature.

Full Teaching Explanation

An edge case is a special situation at the edge of what your algorithm expects. If your function returns the first item in a list, the normal case is a list with at least one item. The edge case is an empty list. If your function divides two numbers, the normal case is a nonzero denominator. The edge case is division by zero.

Safe algorithms begin by checking dangerous conditions before they cause errors. These early checks are often called guards or guard clauses because they protect the rest of the code. A guard might say, "if the list is empty, return None," or "if the denominator is zero, return None." The point is not to ignore the problem; the point is to choose a safe, predictable response instead of letting Python crash with an error.

This matters because real programs rarely receive perfect input. Users leave forms blank. Lists are empty. Dictionaries are missing keys. Numbers arrive as strings. Files do not exist. APIs respond weirdly. Safe algorithms are more reliable because they expect imperfect conditions and handle them intentionally.

Unsafe Version - First Item Without a Guard

```
def first_item(numbers):
return numbers[0]
```

Line-by-line explanation: this works only when numbers contains at least one item. If numbers is an empty list, numbers[0] causes an IndexError because there is no first position to access.

Made with Goodnotes

Safe Version - Guard Against an Empty List

```
def first_item(numbers):
if len(numbers) == 0:
        return None
return numbers[0]
```

Line-by-line explanation: the if statement checks the edge case first. If the list is empty, the function returns None safely. Otherwise, the function continues and returns the first item.

Safe Average - Guard Against Division by Zero

```
def average(numbers):
if len(numbers) == 0:
return None

    total = 0     for n
in numbers:
total = total + n

    return total / len(numbers)
```

Line-by-line explanation: the guard handles the empty list before division happens. Without the guard, total / len(numbers) would become total / 0, which causes a division-by-zero error.

Common Edge Cases

Edge Case	Why It Is Risky	Safe Response
Empty list []	There may be no item at index 0 and no count for averages.	Return None, 0, or an empty result depending on the task.
One-item list	Loops and comparisons may assume multiple items exist.	Check whether the algorithm still makes sense for one value.
Missing dictionary key	Accessing data[key] can crash with KeyError.	Use .get(), check key membership, or provide a default.
Zero denominator	Division by zero causes an error.	Check if denominator == 0 before dividing.
Bad user input	A string like "cat" cannot become an integer.	Use validation or try/except.
None value	None is not the same as 0 or an empty string.	Check for None before math or string operations.

Safe Divide

Made with Goodnotes

```
def safe_divide(a, b):
    if b == 0:
return None
return a / b

print(safe_divide(10, 2))
print(safe_divide(10, 0))
```

Line-by-line explanation: the function checks b before dividing. When b is 0, it returns None instead of crashing. When b is 2, it returns 5.0.

Safe Dictionary Lookup

```
def get_user_name(user):
if "name" not in user:
return "Unknown"
return user["name"]
```

Line-by-line explanation: the guard checks whether the dictionary contains the expected key. If not, the function returns a default value instead of causing a KeyError.

Safe User Input Conversion

```
def to_int(value):
try:         return
int(value)      except
ValueError:
return None
```

Line-by-line explanation: try/except is useful when an operation might fail even after you do your best to check it. If value can become an integer, the function returns the integer. If not, it returns None safely.

Terms, Definitions, and Why They Are Called That

- **Edge case:** an unusual but possible input near the boundary of normal expectations. It is called an edge case because it lives on the edge of the problem space.
- **Guard clause:** an early check that protects the rest of the function. It is called a guard because it stands at the entrance and blocks unsafe conditions.
- **Validation:** checking whether input is acceptable before using it.
- **Default value:** a backup value used when expected data is missing.
- **None:** Python value meaning "no value here." It is useful for safe failure when there is no meaningful answer. - **Exception:** an error event Python raises when something goes wrong, such as ValueError, KeyError, or ZeroDivisionError.

Safe Algorithm Checklist

Made with Goodnotes

- What happens if the list is empty?
- What happens if there is only one item?
- What happens if a key is missing?
- What happens if a number is zero where division is needed?
- What happens if user input is the wrong type?
- What should the function return when it cannot produce a normal result?
- Can I explain the safe response clearly?

Notes

Made with Goodnotes

Code Example:



Lesson 12 - Challenge: Advanced Algorithm Analysis

Big Idea

This final lesson pulls the week together with a challenge-style review. You identify growth classes, simplify Big O expressions, recognize code shapes, and think about safe behavior. It is the boss battle: tiny Python snippets, big runtime feelings.

Full Teaching Explanation

Made with Goodnotes

By Lesson 12, you are expected to combine several skills at once. You should be able to look at a code snippet and ask: Does this run a fixed number of times? Does it loop once per item? Does it use a loop inside another loop? Does it double the number of possibilities each time? Then you should simplify the answer using Big O rules.

This lesson also reinforces that Big O is about growth, not exact timing. A function that checks only numbers[0] is O(1), because it looks at one item no matter how large the list is. A loop that checks every item is O(n), because the work grows with the number of items. A nested loop that compares every pair is O(n^2), because the work grows like n times n. A branching process that doubles possibilities can behave like O(2^n), which grows extremely fast.

The challenge also mixes in safe algorithm thinking. Advanced analysis is not only about speed. A good algorithm should be understandable, should handle edge cases, and should avoid unnecessary work. The strongest answer is often the one that is correct, clear, safe, and reasonably efficient.

Round 1 - Constant or Not?

```
def print_first(numbers):
if len(numbers) > 0:
print(numbers[0])
```

Answer: O(1). The function only checks the length and prints the first item if one exists. It does not scan the whole list, so the work does not grow with n.

Round 2 - One Loop

```
def print_all(numbers):
for n in numbers:
print(n)
```

Answer: O(n). The loop runs once for each number. If the list doubles, the number of print operations doubles too.

Round 3 - Nested Pair Checks

```
def print_pairs(numbers):
    for a in numbers:
for b in numbers:
print(a, b)
```

Answer: O(n^2). For every outer item, the inner loop runs through every item. This creates pair-style work: n times n.

Advanced Challenge Quick Reference

Made with Goodnotes

Pattern	Plain-English Meaning	Big O
Look at first item only	One check regardless of list length.	O(1)
Loop once over a list	One action per item.	O(n)
Two separate loops	n + n steps, simplified.	O(n)
Loop inside a loop	Each item pairs with each item.	O(n^2)
Expression 5n + 8	Drop constants and keep n.	O(n)
Expression n^2 + 3n + 20	n^2 grows fastest.	O(n^2)
Repeated doubling	Possibilities double each round.	O(2^n)

Boss Battle Strategy

- **Round 1:** Check whether the code only does fixed work. If yes, it is usually O(1).
- **Round 2:** Look for a single loop over the input. That usually gives O(n).
- **Round 3:** Look for nested loops. If both depend on n, think O(n^2).
- **Round 4:** Simplify expressions by dropping constants and lower-order terms.
- **Round 5:** Watch for doubling patterns, which can point toward O(2^n).
- **Round 6:** Do a safety check: empty input, missing values, bad values, and clear return behavior.

Mini Practice - Final Growth Check

Snippet or Expression	How to Think	Answer
numbers[0]	Only one item is accessed.	O(1)
for n in numbers	Loop runs once per item.	O(n)
for a in nums: for b in nums	Every item combines with every item.	O(n^2)
2n + 100	Constants do not change growth class.	O(n)
7n^2 + n + 4	n^2 dominates.	O(n^2)
Choices double every step	Work explodes as possibilities double.	O(2^n)

Common Mistakes and Helpful Tips

Made with Goodnotes

- **Mistake:** confusing O(1) with "one line of code." A one-line list operation can still hide more work.
- **Mistake:** thinking O(n) means slow. O(n) is often perfectly fine; it just grows with input size.
- **Mistake:** calling every loop O(n^2). One loop is O(n); nested loops are what usually create O(n^2).
- **Mistake:** forgetting safety while analyzing speed. Efficient code that crashes on empty input is not finished.
- **Tip:** when stuck, test tiny values: n = 1, n = 2, n = 3. Count the key operation.
- **Memory hook:** find the code shape, count the growth, simplify the answer, then check safety.

Final Week 3 Connection

Lessons 9-12 close the loop on the week. You started by learning that algorithms are clear steps. Then you learned to count those steps, describe how the work grows, compare two possible solutions, and make algorithms safer. That full chain is what makes programming feel less like guessing and more like structured problem solving.

When you return to this later, remember the flow: **write the steps, count the work, name the growth, compare choices, protect edge cases.** That is the Week 3 brain map.

Notes

Week 3 Review Tools - Algorithms and Big O

Big Picture Review

Week 3 is the bridge from 'I can write code' to 'I can reason about code.' The key skill is learning to describe an algorithm clearly, count the amount of work it does, and compare two possible solutions without getting hypnotized by small details.

For this week, do not memorize Big O like random alphabet soup. Treat it like a growth story: when the input gets bigger, does the work stay flat, grow one step at a time, grow in pairs, or explode into repeated doubling? That growth pattern is what tells you whether an algorithm will still feel reasonable when the data gets large.

Week 3 Master Map

Topic	What You Should Remember	Fast Exampl
Algorithm	A clear ordered set of steps used to solve a problem.	Order pizza, brush teeth, clean room

Made with Goodnotes

Pseudocode	Plain-English steps that plan the logic before Python.	Get plat -> add bread -> eat
Counting work	Count how many times important steps run.	A loop over 5 names prints 5 times
Complexity	How work grows when the input grows.	More contacts = more work
Big O	A simplified growth label for algorithm efficiency.	O(1), O(n), O(n^2)
Trade-off	Choosing between speed, clarity, safety, and maintainability.	Fast code vs readable code

One-Sentence Memory Hook

- Algorithm analysis means: write clear steps, count the growing work, simplify the growth class, compare choices, and protect the weird inputs.

Notes

Week 3 Glossary - Terms, Meanings, and Examples

How to Use This Glossary

Use this page like your translation dictionary. When a word looks fancy, translate it back into plain English first. Most Week 3 confusion comes from treating vocabulary like magic instead of remembering that every term is describing something ordinary: steps, repeated work, growth, or safety.

Core Vocabulary

Term	Definition and Why It Is Called That	Exampl
Algorithm	A clear set of ordered steps used to solve a problem. It is called an algorithm because it is a repeatable method, not a vague idea.	Brush teeth, make lunch, send a text
Step	One action inside an algorithm. A step should be small enough that a person or computer can follow it.	Open app; typ messag

Made with Goodnotes

Pseudocode	Code-like planning written in plain English. It is called pseudocode because it is not real Python yet, but it acts like a code blueprint.	IF hungr THEN e
Input size / n	The amount of data the algorithm works on. It is often called n because n represents a variable count.	A list wit n names
Step count	How many times a key operation runs. Helps reveal growth.	Loop through names = 6 steps
Complexity	How the work grows as the problem gets bigger. It is not exact seconds; it is growth behavior.	A phone search getting slower
Big O	A simplified label for growth rate. The O stands for order: general size of growth.	One loo over n items = O(n)
Dominant term	The fastest-growing part of an expression. It dominates because it eventually matters most.	n^2 dominat 5n + 20
Constant	A fixed number that does not change with input size. Big O drops constants when describing growth.	3 print lines = O(1)
Edge case	A possible but unusual input near the boundary of normal use. It can break code if not protected.	An emp list passed
Guard clause	An early check that protects the function. It guards the rest of the code from unsafe situations.	If list is empty: return None

Algorithm and Pseudocode Review

Full Teaching Review

An algorithm is not the same thing as code. Code is one way to express an algorithm, but the algorithm is the thinking underneath. This is why th course begins with everyday examples like ordering pizza, washing clothes, and making sandwiches. If you can break a real task into clear steps you can eventually turn that same logic into Python.

Pseudocode is the middle step between natural English and Python. It lets you plan the logic without worrying about exact syntax yet. Good pseudocode should be clear, ordered, and small enough to translate. If one line secretly contains five actions, split it. A computer cannot infer what you meant the way a person might.

Algorithm vs Pseudocode vs Python

Term	What It Looks Like	What It Is For
Algorithm idea	Clean room / make sandwich / send text	Names the problem or task
Pseudocode	IF list is empty THEN return None	Plans logic before exact Python
Python	if len(items) == 0: return None	Runs th algorithm in a real languag

Made with Goodnotes

Pseudocode Example - Safe First Item

```
Task: Safely get the first item from a list

IF the list is empty:
    return None
ELSE:     return the first item
```

Real English: Before touching the list, check that something is there. If the list has items, take the first one. If not, return a safe value instead of crashin

Common Mistakes and Fixes

- Mistake: writing a giant step like "do laundry." Fix: split it into load washer, add soap, start machine, move to dryer, fold.
- Mistake: jumping straight to Python before the logic is clear. Fix: write pseudocode first.
- Mistake: making pseudocode too vague. Fix: use tiny, ordered actions.

Counting Work and Big O Decision Guide

Full Teaching Review

Counting work means identifying the part of the algorithm that repeats or grows. You usually do not count every tiny machine operation. Instead, you focus on the key action: printing a name, checking an item, comparing a pair, adding a number, or processing one record.

Big O then turns that count into a growth category. If the key work stays the same no matter how much input exists, it is O(1). If the work goes one-for-one with the number of items, it is O(n). If each item gets compared with many other items, the work usually grows as O(n^2).

Big O Decision Guide

Question to Ask	What It Usually Means	Big O
Does the same amount of work happen no matter how big input is?	Fixed work	O(1)
Does one loop visit each item once?	One pass through input	O(n)
Are there two separate loops, one after another?	n + n, which simplifies	O(n)
Is one loop nested inside another over the same input?	n times n pair work	O(n^2)
Does input size double but work grows by one step?	Repeated halving / divide-and-conquer shape	O(log n)
Does every choice create more choices?	Exploding branching growth	O(2^n) or worse

Trace Example - Single Loop

```
names = ["Ana", "Ben", "Leo"] steps
= 0

for name in names:
steps = steps + 1
print(name)

print("Total steps:", steps)
```

Line-by-line: the loop visits each name once. Three names means three loop runs. Six names means six loop runs. Because the work grows one-for-one with the input size, this shape is O(n).

Big O Growth Classes - What They Feel Like

Made with Goodnotes

Full Teaching Review

Big O names the shape of the work. The shape matters more than the exact number of seconds. A program that changes five things may seem fa
every time the input grows.

A helpful mental test: ask whether adding more input causes the work to stay the same, grow smoothly, grow by pairs, or explode. The more dran

Growth Class Comparison

Big O	Plain-English Meaning	Common Code Shape
O(1)	Flat work: input size does not change the amount of work.	Direct access / fixed steps
O(log n)	Work grows slowly because the problem shrinks each round.	Repeated halving
O(n)	Straight line growth: one main pass over input.	One loop through n items
O(n log n)	More than one loop but usually much better than quadratic.	Efficient sorting-ish patterns
O(n^2)	Pair work: each item gets compared with many others.	Nested loops over same input
O(2^n)	Branching explosion: possibilities double.	Try every yes/no combination

Simplifying Expressions

Expression	What to Drop
5n	Constant multiplier 5
n + 20	Small fixed extra work
n^2 + n	Lower-order term n
7n^2 + 3n + 10	Drop constants and lower terms
20	Fixed work

Code Shape Practice - Trace Before You Guess

How to Practice

When a question asks for Big O, do not start by staring at the whole code like it owes you money. First find the repeated shape: one loop, two separate loops, nested loops, fixed print lines, or branching choices. Then count what grows with input and ignore the small constants.

Practice 1 - What Is the Growth?

Made with Goodnotes

```
# Practice 1 - What is the growth?
for name in names:     print(name)
```

Answer: O(n). The function checks one name each loop. More names means more loop runs.

Practice 2 - What Is the Growth?

```
# Practice 2 - What is the growth?
print("start")
print("middle") print("end")
```

Answer: O(1). The function prints three fixed lines. The number of input items does not change the work.

Practice 3 - What Is the Growth?

```
# Practice 3 - What is the growth?
for a in nums:     for b in nums:
print(a, b)
```

Answer: O(n^2). For every outer item, the inner loop runs across the whole list.

Trace Table - Tiny n Values

One Loop Runs	Nested Loop Work	What It Shows
1	1	Both are tiny
2	4	Nested work is already multiplying
10	100	Pairs grow quickly
100	10,000	Quadratic growth is much heavier
1,000	1,000,000	This is why Big O matters before production-size data

Comparing Solutions and Writing Safe Algorithms

Full Teaching Review

A correct answer is not automatically the best answer. Two algorithms can return the same result while having different speed, clarity, memory use, and safety. Week 3 asks you to keep thinking like a programmer who has to choose between options, not just make any code work once.

When two solutions have different Big O, the better-scaling option often matters more as input grows. When two solutions have the same Big O, readability becomes a major deciding factor. Safe algorithms also check bad or unusual inputs before those inputs crash the code.

Solution Comparison Checklist

Situation	Better Choice	Why
Need to count every item once	Single loop	Clear O(n) and easy to read
Nested loop does same job as one loop	Single loop	Avoids unnecessary O(n^2) work
Both are O(n), but one is clearer	Readable version	Humans maintain code later

Made with Goodnotes

Input might be empty	Guard clause	Prevents index/division errors
Input might be weird type	Validate before processing	Avoids confusing crashes

Safe First Item With Guard

```
def first_item(numbers):
if len(numbers) == 0:
        return None
return numbers[0]
```

The guard clause handles the empty-list edge case before the code touches index 0.

Edge Case Guard Guide

Risky Situation	Possible Guard
Empty list	if len(items) == 0: return None
Dividing by zero	if len(numbers) == 0: return None or message
Missing dictionary key	if key not in data: return fallback
Bad numeric input	try/validate range before math
None value	if value is None: return safe_default

Made with Goodnotes

Weekly Review Question Breakdown

How the Review Questions Think

The Week 3 review questions are not trying to make you memorize every screenshot. They are checking whether you can recognize the underlying pattern: clear steps, input size, repeated work, simplified Big O, comparison trade-offs, and safe edge-case handling.

Question Type Breakdown

Question Type	What It Is Really Asking	How to Answer
Best example of an algorithm	Which option has clear ordered steps?	Choose the list of actions, not a vague sentenc or pictur
What does n mean?	How many items does the input contain?	n = inpu size
Big O for a loop	Does work grow with n?	One loo over n items = O(n)
Big O for nested loops	Does one loop run inside another?	n times = O(n^2
Simplify expression	Which terms govern growth?	Drop constan and low order terms
Compare two solutions	Which is faster, clearer, or safer?	Mention speed a readabil not just code length
Edge case question	What happens with empty or bad input?	Look for guard clause o safe return

Mini Answer Key Style Explanations

- Algorithm example: "open app -> enter PIN -> check balance" is an algorithm because it is ordered and repeatable.
- Three fixed print statements: O(1), because the number of prints does not depend on list size.
- One loop over a list: O(n), because work grows once for each item.
- Nested loops over the same list: O(n^2), because each outer item triggers a full inner pass.
- 5n + 20 simplifies to O(n), because constants and fixed additions do not change the growth shape.

Notes

Made with Goodnotes

Common Exam Traps and Debug Thinking

Trap Filter

The trap questions usually do not ask something wildly new. They take a familiar pattern and hide it inside extra words, constants, or a tiny code snippet. The goal is to ask whether you recognize the shape instead of panic-clicking whatever answer has the most intimidating parentheses.

Common Traps

Trap	Why It Tricks People	Correct Thinking
Three fixed print lines	People assume one line means O(1), three lines means O(3)	Still O(1), fixed work
Two separate loops	People see two loops and choose O(n^2)	Separate loops add: O(n)+O(n)=O(n)
Nested loops	People count only the visible outer loop	Inner loop runs for every outer item
Constants in expression	People keep 5n or 20 in answer	Drop constants and lower-order terms
Edge cases	Code works on normal input, so it seems safe	Test empty lists, zero, missing keys, and bad types
Faster but unreadable	People assume faster always wins	If Big O is same, readability may matter more

Debug Thinking Checklist

- What is the input size n?
- Which line or operation is repeated work?
- Does the code visit the same amount of work no matter what? If yes, think O(1).
- Is there one loop over input? Think O(n).
- Is there a loop inside another loop over input? Think O(n^2).
- Does the expression or code contain constants to drop?
- Does the function have a guard for empty or bad input?

Trap Example - Separate Loops Are Not Nested

```
# Separate loops are NOT nested
for x in nums:     print(x)

for y in nums:
print(y)
```

Answer: O(n). The loops are separate, so the work is n + n. Big O drops the constant multiplier and keeps O(n).

Made with Goodnotes

Week 3 Cheat Sheet

Fast Recall Table

Concept	Quick Version
Algorithm	Ordered steps to solve a problem
Good algorithm	Clear, ordered, repeatable, testable, and safe
Pseudocode	Plain-English logic before real syntax
Step count	How many times the key action runs
Input size n	How much data the algorithm receives
Complexity	How work grows when n grows
Big O	Simplified growth label
O(1)	Flat work
O(n)	One pass through input
O(n^2)	Nested pair-style work
Dominant term	Fastest-growing part of an expression
Guard clause	Early safety check
Edge case	Unusual but possible input that needs safe handling

Big O Naming Recipe

Step	Ask Yourself
1	What is n?
2	What line repeats?
3	How many times does it run?
4	Could two loops be separate or nested?
5	What is the raw expression?
6	Drop constants and lower-order terms
7	Name the Big O growth class
8	Check edge cases and readability before calling it done

Three Shapes to Memorize

Made with Goodnotes

```
O(1) print("hello")

O(n) for item in
items:
print(item)

O(n^2) for a in
items:     for b in
items:
print(a, b)
```

Real English: fixed work, one pass, and pair work. If you can spot those three, most Week 3 questions get much less rude.

Made with Goodnotes

Final Week 3 Memory Sheet

The Whole Week in One Teaching Flow

An algorithm starts as a plan: a clear set of steps that transforms a starting situation into a finished result. Pseudocode lets you write that plan without worrying about Python syntax yet. Once the plan exists, you can count how much work it does. If the work is fixed, it is constant. If the work grows with every item, it is linear. If every item gets paired with every other item, it is quadratic.

Big O is the language for naming those growth shapes. It does not care about exact seconds or exact tiny operations. It cares about how the work grows as the input gets bigger. That is why constants and smaller terms usually disappear. What matters most is the dominant shape.

The final layer is judgment. A better solution is not only one that works. It should be clear enough to understand, efficient enough for the input size, and safe enough to handle edge cases. That is the Week 3 mindset: clear steps, counted work, named growth, thoughtful choices, safe behavior.

Final Self-Check Questions

- Can I explain what an algorithm is without using code? - Can I turn an everyday task into ordered pseudocode?
- Can I identify the input size n?
- Can I count repeated work in a loop?
- Can I tell the difference between separate loops and nested loops?
- Can I simplify 7n^2 + 20n + 10?
- Can I compare two solutions by speed and readability?
- Can I add a guard clause for an empty list or zero divisor?
- Can I explain my Big O answer in plain English?

Last-Minute Memory Hooks

Phrase	Meaning
Clear steps first	Do not code a plan you cannot explain
Count the repeated work	Big O usually comes from what repeats
Separate loops add	n + n becomes O(n)
Nested loops multiply	n x n becomes O(n^2)
Fastest term wins	n^2 beats n; n beats 20
Guard the edge	Handle empty, missing, zero, and weird input before it crashes

Final Notes and Questions

Review Reflection Prompt

Use this page for anything you want to ask, rewrite in your own words, or practice again later. The strongest review habit is to turn confusing summaries back into full explanations: What is the input? What repeats? What grows? What gets ignored? What edge case could break this?

Questions / Extra Practice Notes

Made with Goodnotes

Made with Goodnotes

CS102 Week 4 Notes and Study Guide

Week 4 Memory Hooks

- Searching finds the target.
- Sorting prepares the data.
- Linear search asks every item one by one.
- Binary search only works when the data is already sorted.
- The best choice depends on the situation, not the fanciest algorithm name.

Table of Contents

Made with Goodnotes

What is inside

- Week 4 overview: searching, search targets, search spaces, comparison steps, linear search, binary search, sorting, and choosing efficient strategies.
- Lesson 1 - Why Searching Matters
- Lesson 2 - How Searching Works with Simple Examples
- Lesson 3 - Linear Search with Small and Unsorted Data
- Lesson 4 - Binary Search with Step-by-Step Collections
- Lesson 5 - Comparing Linear and Binary Search
- Lesson 6 - Challenge: Searching
- Lesson 7 - Why Sorting Is Useful
- Lesson 8 - Sorting: Different Approaches
- Lesson 9 - Bubble Sort
- Lesson 10 - Insertion Sort
- Lesson 11 - Bubble vs. Insertion
- Lesson 12 - Challenge: Sorting
- Lesson 13 - Using Python Sorting Tools in Practice
- Lesson 14 - Advanced Sorting with Python Sorting Tools
- Lesson 15 - Challenge: Advanced Sorting
- Review tools, glossary, tracing practice, common mistakes, cheat sheet, and exam reference

Week 4 at a Glance

Week 4 moves from understanding algorithms to using algorithms for finding and organizing information. The first half focuses on searching: how a program looks through data, what it means to have a target, and why checking items one at a time can become slow as collections grow. The second half connects searching to sorting, because sorted data changes what strategies become possible.

The major story is trade-offs. Linear search is simple and works on unsorted data, but it may check every item. Binary search is much faster on large collections, but only works correctly when the data is sorted. Sorting itself takes work, so programmers choose based on the problem: how often the data changes, how often it is searched, and whether readability, speed, or simplicity matters most.

Memory hook: searching finds the target; sorting prepares the data; the best strategy depends on how the data is organized.

Week 4 Focus Questions

- What exact target am I trying to find?
- What search space am I allowed to inspect?
- Is the data sorted, unsorted, or changing often?
- Am I doing one quick search or searching the same data many times?
- Would sorting first make the work easier, or would it waste time?
- Does this problem care most about simplicity, speed, readability, or setup cost?

Week 4 Big Picture

Made with Goodnotes

- Search problems ask: what target am I trying to find, and what data am I searching through?
- Linear search is the safest beginner pattern because it does not require sorted data.
- Binary search is powerful because each comparison removes about half of the remaining possibilities.
- Sorting is useful because ordered data is easier for both humans and algorithms to work with.
- Efficiency is not just fast or slow; it is about how work grows as input gets bigger.

Week 4 Master Framework

Core Thinking Skills

Skill	What it means	Why it matters
Search thinking	Identify what you are trying to find and where you are allowed to look.	Turns vague questions into a target plus a search space.
Comparison thinking	Check one candidate value against the target.	Every search depends on asking, "Is this the thing I want?"
Linear search	Check items one by one from start to finish.	Reliable for small or unsorted data, but may require many checks.
Binary search	Use the middle value to cut a sorted search space in half.	Much faster for large sorted data, but only works when order is guaranteed.
Sorting thinking	Arrange values into a useful order before searching or presenting them.	Organization can make later work easier, faster, and less chaotic.
Trade-off thinking	Compare simplicity, speed, prerequisites, and readability.	The best algorithm is the one that fits the actual situation, not just the flashiest one.

Search and Sort Decision Guide

Made with Goodnotes

Situation	Best first thought	Reason
Small list and no special order	Linear search	Simple, readable, and good enough.
Huge sorted list	Binary search	Repeated halving avoids checking everything.
Need to show items alphabetically or numerically	Sort first	Order makes the result easier for humans to use.
Data changes constantly	Be careful sorting early	Repeated sorting can waste work.
Searching the same data many times	Consider sorting or direct lookup	A little setup may save lots of repeated searching later.

Lesson 1 - Why Searching Matters

Big Idea

Searching means trying to locate a specific thing inside a group of information. In everyday life, searching might mean finding a contact, a photo, a file, or a song. In programming, searching means designing steps that help a computer find a target value inside a collection.

Full Teaching Explanation

The lesson starts with a phone contact example because it is familiar: if you need to call Zara, you could scroll through every contact, or you could use the search bar and jump much closer to the match. Both actions are searches, but they do not feel equally efficient. Scrolling checks possibilities one at a time. A search box uses a smarter path that reduces the amount of waiting.

That is the core reason searching matters in code. Programs often store large collections: contacts, emails, photos, product listings, grades, filenames, messages, usernames, or account records. If the program checks every possible item when it does not need to, the app can feel slow even when it eventually finds the correct result.

A useful way to phrase the problem is: "I know what I want, and I need to locate it in a bunch of stuff." The thing you want is the target. The bunch of stuff is the search space. The strategy is the set of steps the computer uses to move through that space.

Good searching affects user experience. If the first layer of search is slow, the whole app feels bad. If the first layer narrows results quickly, the human user can spend less time waiting and more time choosing the exact item they wanted.

Examples and Mental Models

Everyday situation	Search target	Search space	Why it matters
Find Zara in contacts	Zara	Contact list	The app should not make you scroll forever.
Find an invoice photo	Invoice screenshot	Photo library	The computer can filter quickly, but the human may still inspect results.
Find a song by title	Song name	Music library	Direct lookup feels much faster than opening random songs.
Find an email from Dr. Smith	Dr. Smith email	Inbox	A search feature saves time compared with checking messages one by one.

Out Loud Programmer Thinking

Made with Goodnotes

When I see a search problem, I should ask: What exactly am I looking for? Where am I allowed to look? Am I checking one item at a time, jumping directly to a known place, or using some other structure to narrow the results? If I can answer those questions, I can usually describe the search strategy before writing any Python.

Code Example

```
contacts = ["Alice", "Bob", "Charlie", "Zara"]

# Scroll-style search: check each contact one at a time.
for name in contacts:
    if name == "Zara":
        print("Found", name)

# Direct lookup style: jump straight to the known key.
phone_numbers = {"Zara": "555-1234"}
print(phone_numbers["Zara"])
```

The first pattern walks through the list one name at a time. The second pattern uses a key to jump straight to the structures can make searching feel very

Lesson 1 - Why Searching Matters (continued)

Terms, Definitions, and Why They Are Called That

- Search: the process of looking for a specific item. It is called search because the answer is not immediately handed to you; the program must locate it.
- Target: the value you want to find. It is called the target because the search is aiming at that value.
- Search space: the collection of possible places where the target might be. It is the space the algorithm is allowed to inspect.
- Comparison: a check that asks whether the current item matches the target.
- Direct lookup: using a known key or location to retrieve information without checking every candidate first.

Common Mistakes and Helpful Tips

- Mistake: thinking all searches work the same way. Some searches scan one item at a time; others use structure to jump closer.
- Mistake: forgetting the human layer. A computer may find results quickly, but the user still has to recognize the right result.
- Tip: describe a search in one sentence: Find target inside search space.
- Tip: fewer unnecessary checks usually means a faster app and a happier user.

Review Checkpoint

- What is the search target in a contacts example?
- What is the search space in a photo-library example?
- Why does a search box usually feel faster than manual scrolling?
- How can a data structure make searching easier?

Key Takeaways

- Searching is one of the most common tasks real programs perform.
- A good search strategy reduces unnecessary work.
- The same user goal can be solved with slow scanning or faster direct access depending on the structure.
- User experience is affected by how quickly the first useful results appear.

Notes

Made with Goodnotes

Lesson 2 - How Searching Works with Simple Examples

Big Idea

A simple search has three parts: the search space, the search target, and comparison steps. The program looks through possible values, compares each candidate to the target, and stops when it finds the target or runs out of places to look.

Full Teaching Explanation

The lesson uses the lost-key example because it shows the search pattern clearly. You check the first pocket: no key. You check the second pocket:
no key. You check the third pocket: yes, the key is there. That is a search because you are looking for one special thing inside a small group of possible locations.

In programming terms, the key is the search target. The pockets are the search space. Each pocket check is a comparison step. The comparison asks, "Is this the thing I want?" If the answer is no, the search continues. If the answer is yes, the search can stop.

This pattern matters because it is the foundation for linear search. The computer does not magically know where the item is unless the data structure gives it a direct path. Without that direct path, it has to inspect candidates one by one. That can be perfectly fine for tiny collections, but it becomes more expensive as the number of items grows.

The important habit is tracing the search in plain English before code. If you can say, "check wallet, check phone, check key," then you already understand the loop pattern: look at each item, compare it to the target, and decide whether to continue.

Search Anatomy

Part	Plain-English meaning	Example
Search target	The specific thing we want to find.	The key
Search space	The group of places or values we are searching through.	Wallet, phone, key
Comparison step	One check asking whether the current item is the target.	Is this item the key?
Stop condition	The reason the search ends.	Found the key or checked everything

Real English Trace

- Check wallet - not the key.
- Check phone - not the key.
- Check key - yes, found it.
- This trace is the human-readable version of the loop the program will later run.

Code Example

Made with Goodnotes

```
items = ["wallet", "phone", "key"]
target = "key"

for item in items:
    print("Checking", item)
    if item == target:
        print("Found it")
        break
```

items is the search space, target is the value we want, and the loop visits one item at a time. The if statement is once the target is

Lesson 2 - How Searching Works with Simple Examples (continued)

Terms, Definitions, and Why They Are Called That

- Search space: the set of items or places being checked. It is called a space because it describes the area where the answer could exist. - Target: the value being searched for. It is the thing the algorithm is trying to hit.
- Candidate: the current item being inspected. It is a candidate because it might be the target.
- Comparison step: the moment the program asks whether the candidate matches the target.
- Break: a command that exits the loop early. It is useful when continuing would waste work after the target is found.

Common Mistakes and Helpful Tips

- Mistake: treating the list itself as the target. The list is where we search; the target is what we want.
- Mistake: forgetting that the loop checks one item per cycle.
- Mistake: continuing to search after the item has already been found when the task only needs the first match.
- Tip: label your variables out loud: search space, target, current item.

Review Checkpoint

- What are the three basic parts of a search?
- What does the comparison step ask?
- Why is break useful after a target is found?
- How can you trace a search in plain English?

Key Takeaways

- A search problem needs a target, a search space, and comparison steps.
- The current item is the candidate being checked right now.
- A loop is a natural way to repeat comparison steps.
- break is useful when the target is found and no more checking is needed.

Notes

Made with Goodnotes

Lesson 3 - Linear Search with Small and Unsorted Data

Big Idea

Linear search checks items one at a time from the beginning of a collection until it finds the target or reaches the end. It is simple, reliable, and especially useful when the data is small or not sorted.

Full Teaching Explanation

Linear search is the most straightforward search algorithm. Imagine looking for a name tag in a messy box. You do not have a sorted order, an index, or a shortcut. You pick up one tag, ask whether it is the one you want, then move to the next. That is linear search.

The word linear means "in a line." The search moves through the data as if the items are lined up from first to last. It does not skip around. It does not split the list in half. It simply checks each item in order. Because of that, it works even when the data is unsorted.

That reliability is the reason linear search is often the first search algorithm taught. It is not always the fastest, but it is easy to understand and hard to misuse. If the target is near the beginning, linear search may finish quickly. If the target is at the end or missing, it may have to check every item.

The key trade-off is simplicity versus speed. For five items, linear search is usually fine. For five million items, checking one by one can become painful. This lesson is training you to recognize when simple is good enough and when a better strategy may be worth the setup.

When Linear Search Makes Sense

Situation	Is linear search reasonable?	Why
Small unsorted list	Yes	There is not enough data to justify a more complex setup.
Short grocery list	Yes	Checking one item at a time is simple and clear.
Huge sorted database	Usually no	Binary search or indexing may be much faster.
Unknown order	Yes	Linear search does not require sorting first.
Repeated searches on same data	Maybe not	A different structure may save work over time.

Code Example

Python ▷ Run ↙ Submit code

```
def linear_search(items, target):
    for index in range(len(items)):
        if items[index] == target:
            return index
    return None

result1 = linear_search([3, 7, 9], 7)
print(result1)

result2 = linear_search([4, 5, 8], 2)
print(result2)

result3 = linear_search([1, 2, 3, 4], 4)
print(result3)
```

Output

```
1
None
3
```

Made with Goodnotes

Mental Model

Picture a row of sticky notes. You start at the left, read one note, compare it to the target, and move right if it is not a match. The search continues in a straight line. That is why the pattern is called linear.

Code Example

```
def linear_search(items, target):
    for index in range(len(items)):
        if items[index] == target:
            return index
    return None

result1 = linear_search([1, 5, 7], 5)
result2 = linear_search([1, 5, 7], 8)
print(result1)
print(result2)
```

The function receives a list and a target. Each loop cycle checks one position. If the value equals the target, the finishes without finding the target, it returns

Lesson 3 - Linear Search with Small and Unsorted Data (continued)

Notes

Step-by-Step Trace

Made with Goodnotes

Check	Current value	Question asked	Result
1	1	Is 1 the target 5?	No - keep going
2	5	Is 5 the target 5?	Yes - return index 1
If searching for 8	1, then 5, then 7	Did any value match 8?	No - return None

Terms, Definitions, and Why They Are Called That

- Linear search: a search that checks items in a straight one-by-one path. It is called linear because the work follows the collection like a line.
- Unsorted data: data that is not arranged in a guaranteed order. Linear search works here because it does not depend on order.
- Return value: the result a function sends back. In this example, the return value is either an index or None.
- None: Python's way of representing no value here. It is useful when the target was not found.

Common Mistakes and Helpful Tips

- Mistake: assuming linear search only works on sorted data. It works on unsorted data because it checks every possible item if needed.
- Mistake: confusing the target with the index. The target is what you want; the index is where it was found.
- Mistake: forgetting the missing-target case. A good function should say what happens when no match exists.
- Tip: when tracing, write the current index, current value, comparison question, and result.
- Tip: linear search is often the right first answer when the data is small, unsorted, or only searched once.

Review Checkpoint

- What are the search target and search space in a search problem?
- What does a comparison step ask?
- Why does linear search work on unsorted data?
- What is the worst-case situation for linear search?
- Why might linear search be fine for a small list but frustrating for a huge list?
- What should a function return if the target is not found?

Key Takeaways

- Searching means locating a target inside a search space.
- Comparison steps are the tiny yes/no checks that power a search.
- Linear search checks items one at a time in order.
- Linear search is simple and reliable for small or unsorted collections.
- The trade-off is that it may check every item when the target is last or missing.

Big Idea

Binary search is a smarter search strategy for sorted collections. Instead of checking every item from the beginning, binary search checks the middle item, decides whether the target must be on the left or right side, and throws away the half that cannot contain the answer.

The power is not magic; it comes from one condition: the data must already be sorted. If the list is not sorted, the left-side-smaller and right-side-bigger logic breaks.

Full Teaching Explanation

Made with Goodnotes

Linear search works like looking through flash cards one by one. Binary search works like guessing a number between 1 and 100 when someone tells you too high or too low. Every guess gives information that removes a large chunk of possible answers.

The middle value is the decision point. If the middle value is the target, the search is done. If the target is smaller than the middle value, every value on the right can be ignored because sorted order tells us those values are even larger. If the target is bigger than the middle value, every value on the left can be ignored because those values are too small.

That is why binary search feels so efficient: it does not move one step at a time. It cuts the remaining search space roughly in half each round. With 64 sorted items, linear search might need 64 checks in the worst case. Binary search needs about 6 checks because 64 -> 32 -> 16 -> 8 -> 4 -> 2 -> 1.

The beginner trap is using binary search on unsorted data. If the list is scrambled, the middle value does not tell you which half can be ignored. In that situation, linear search is safer because it makes no order assumptions.

Binary Search Mental Model

Step	What binary search does	Why it helps
1. Set low and high	low starts at the first index; high starts at the last index.	These boundaries define the part of the list that is still possible.
2. Find the middle	mid = (low + high) // 2.	The middle gives one comparison that can remove half the search space.
3. Compare middle to target	If values[mid] equals target, the target is found.	The search can stop immediately.
4. Move left or right	If the target is smaller, move high left. If the target is bigger, move low right.	The impossible half is discarded.
5. Stop when empty	If low becomes greater than high, there is no valid search space left.	The target is not in the list.

Think of the search space as a shrinking hallway. The algorithm keeps closing doors until only the target is left or no rooms remain.

Code Example - Binary Search Loop

```
numbers = [1, 3, 5, 7, 9, 11, 13, 15]
target = 7
low = 0
high = len(numbers) - 1

while low <= high:
    mid = (low + high) // 2
guess = numbers[mid]

    if guess == target:
        print("Found at index", mid)
        break     elif
guess < target:
low = mid + 1     else:
high = mid - 1
```

Plain English: start with the whole sorted list, check the middle, and keep only the half where the target could still be hiding. The list itself does not change; the possible search area changes.

Made with Goodnotes

Trace Practice - Watch the Boundaries Move

Round	low	high	mid	guess	Decision
1	0	7	3	7	Target found immediately.
If target were 13	0	7	3	7	13 is larger, so move low to 4.
Next round	4	7	5	11	13 is larger, so move low to 6.
Next round	6	7	6	13	Target found.

When tracing binary search, track low, high, mid, and guess. Those four values explain the entire search.

Terms, Definitions, and Why They Are Called That

- Binary search: a search method that repeatedly divides remaining possibilities into two parts. It is called binary because each comparison choosesbetween two directions: left half or right half.
- Sorted collection: data arranged in order, such as smallest to largest or A to Z. Binary search needs this order to decide which half is impossible.
- Low: the lowest index still being considered. It starts at the beginning and moves right when the target must be larger.
- High: the highest index still being considered. It starts at the end and moves left when the target must be smaller.
- Mid: the middle index between low and high. It is the spot binary search checks first each round.
- Search space: the part of the data where the target could still be. In binary search, the search space shrinks fast.

Binary Search Practice Lab

Out Loud Programmer Thinking

I am looking for a target inside an ordered list. I do not need to check every value because sorted order lets me eliminate values that cannot possibly work.

First I check the middle. If the middle is too small, I move right. If the middle is too large, I move left. If the boundaries cross, the target is not there.

Before I choose binary search, I must ask: is this data sorted? If not, the smart-looking algorithm becomes a confidently wrong algorithm, which is rude but very on brand for bugs.

Notes

Made with Goodnotes

Practice prompt	How to think it through	Answer pattern
Sorted list has 64 items. What is the maximum number of checks?	Keep halving: 64 -> 32 -> 16 -> 8 -> 4 -> 2 -> 1.	About 6 checks.
The list is [12, 3, 18, 7, 1]. Should you use binary search?	Ask the sorted question first. This list is scrambled.	No. Use linear search unless the list is sorted first.
Target is smaller than the middle value.	Sorted order tells us everything to the right is even bigger.	Move high left. Search the left half.
Target is bigger than the middle value.	Sorted order tells us everything to the left is too small.	Move low right. Search the right half.

Common Mistakes and Helpful Tips

- Mistake: using binary search on unsorted data. If the list is scrambled, the middle value cannot tell you which side to ignore.
- Mistake: moving low or high the wrong direction. If guess is too small, move low right. If guess is too large, move high left.
- Mistake: forgetting that indexes start at 0. The middle value and middle index are related but not the same thing.
- Tip: write low, high, mid, and guess in a small table when solving quiz questions.
- Memory hook: binary search does not search harder; it searches smarter by deleting half the possibilities each time.

Choosing the Right Search

Situation	Choose	Why
A tiny unsorted grocery list	Linear search	Simple and reliable; no sorting required.
A dictionary or phonebook sorted A to Z	Binary search	Order lets you jump to the middle and discard half.
A constantly changing list	Probably linear first	Sorting over and over may waste more work than it saves.
A huge sorted database field	Binary search or indexed lookup	Large sorted data rewards faster search strategies.

Made with Goodnotes

A quiz says sorted list and faster search	Binary search	Those words are clues that halving is allowed.

Big Idea

Linear search and binary search both try to find a target, but they make different promises. Linear search promises flexibility: it works even when the data is small, random, or unsorted. Binary search promises speed: it can remove half the remaining choices each step, but only when the data is sorted.

The real skill is not memorizing which one is better. The real skill is choosing the right search method for the situation.

Full Teaching Explanation

Linear search is the safe beginner pattern because it asks the simplest possible question over and over: is this the target? It starts at the beginning and keeps checking until it finds the answer or runs out of data. This is easy to read, easy to test, and works on almost any list.

Binary search is faster when the collection is sorted because it uses order as information. If the target is smaller than the middle value, the right half cannot contain the target. If the target is larger, the left half cannot contain the target. That lets the algorithm skip huge chunks of work.

The trade-off is that binary search has a prerequisite. The data must already be sorted in the same order the algorithm expects. If sorting has already been done or the data naturally stays sorted, binary search is excellent. If the list is tiny, constantly changing, or not sorted, linear search may be simpler and perfectly reasonable.

Linear vs Binary Search Decision Table

Question	Linear Search	Binary Search
Does the data need to be sorted?	No. It can handle random or unsorted order.	Yes. Sorting/order is required.
How does it move?	Checks one item at a time from start to finish.	Checks the middle and cuts the search space in half.
Best for	Small lists, unsorted lists, quick simple checks.	Large sorted lists or repeated searches on ordered data.
Worst-case work	May check every item.	Checks only about log2(n) positions.
Beginner risk	Mostly just forgetting to stop or return.	Using it on unsorted data or moving boundaries wrong.

Step Count Comparison

Made with Goodnotes

List size	Linear search worst case	Binary search worst case	What that means
5 items	5 checks	about 3 checks	Both are fine; simplicity may matter more.
10 items	10 checks	about 4 checks	Binary starts showing an advantage.
20 items	20 checks	about 5 checks	Binary is much more efficient if sorted.
64 items	64 checks	about 6 checks	Repeated halving becomes a big win.
1,000 items	1,000 checks	about 10 checks	Binary avoids almost all of the work.

The exact binary count may vary slightly depending on implementation and target position, but the growth pattern is the important idea: linear grows one step at a time, while binary grows by halving.

Common Mistakes and Helpful Tips

- Mistake: assuming binary search is always better. It is only better when its requirements are met.
- Mistake: forgetting the first decision question: is the data sorted?
- Mistake: measuring only speed and ignoring readability. A simple linear search can be the better choice for small data.
- Tip: for quiz questions, ask two things first: is it sorted, and is it big?
- Memory hook: linear steps grow one by one; binary steps grow much slower because each step cuts the list in half.

Notes

Code Choice - Why Linear Search Fits Unsorted Data

```
nums = [12, 5, 8, 2, 30, 25]
target = 25 # This list is
not sorted.
# Binary search should NOT be used here.

for n in nums:     if
n == target:
print("Found")
break
```

Plain English: because the list is scrambled, the program should check one value at a time. Without sorted order, the middle value does not safely eliminate a half.

Big Idea

Made with Goodnotes

The challenge lesson asks you to choose a search strategy, trace what the code does, and explain why the choice is correct. The goal is to prove that you understand the rules, not just recognize vocabulary words.

For search questions, the safest process is: identify the target, identify the search space, decide whether the list is sorted, choose linear or binary search, and then trace the comparisons.

Full Teaching Explanation

This lesson acts like a mini test of everything so far. Some prompts ask which search strategy fits. Others ask what a piece of code prints. Others ask whether a statement about binary search is true. They all come back to the same foundation: searching means looking for a target inside a search space.

If the data is not sorted, linear search is usually the correct beginner answer. Linear search does not care whether the numbers are in order. It checks each item until the target is found or the list ends. This makes it reliable for messy, short, or unpredictable data.

If the data is sorted, binary search becomes possible. Binary search is usually chosen when the list is sorted and the question hints at speed, large data, or fewer checks. But the sorted condition is not decorative; it is the entire reason binary search can throw away half the values.

The code-tracing part trains careful reading. A variable like found usually starts as False. If the loop sees the target, the code changes found to True. When the loop ends, print(found) tells you whether the target appeared at least once.

Challenge Strategy Table

Prompt clue	Best thinking move	Likely answer
The list is not sorted	Do not assume halves mean anything.	Linear search
The list is sorted	Check whether the question wants faster/fewer checks.	Binary search may fit
The target might not exist	Trace until found or until search space is empty.	Return not found / False
The code uses found = False	Watch whether an if statement changes it to True.	Final print depends on the condition
The question asks for checks	Count comparisons, not just lines of code.	Use trace steps

Code Trace - Found Flag

```
nums = [2, 5, 8, 12, 25, 30]
target = 25
found = False

for n in nums:
if n == target:
found = True

print(found)
```

Plain English: start by assuming the target has not been found. Check each number. If any number equals the target, switch found to True. Because 25 appears in the list, the final output is True.

Search Challenge Checklist

Made with Goodnotes

Question to ask	Why it matters
What is the target?	You cannot search if you do not know what value you want.
What is the search space?	This tells you where the algorithm is allowed to look.
Is the data sorted?	This decides whether binary search is even allowed.
How many comparisons happen?	This is the actual work the search performs.
What stops the search?	The algorithm must know when it found the target or when no possibilities remain.

Lesson 6 Quick Recap

- Linear search is the safe choice for small or unsorted collections.
- Binary search is the fast choice for sorted collections because each check can remove half the remaining search space.
- Challenge questions often test whether you notice the sorted requirement.
- Code-tracing questions require following variable changes one step at a time.
- A strong answer explains why the strategy fits, not just which strategy was chosen.

Notes

Made with Goodnotes

Lesson 7 - Why Sorting Is Useful

Code Example - Sorting Records by One Field

```
sessions = [
    {"client": "Avery", "date": "2026-05-01"},
    {"client": "Morgan", "date": "2026-04-18"},
{"client": "Avery", "date": "2026-04-21"}
]

by_client = sorted(sessions, key=lambda s: s["client"]) print(by_client)
```

Plain English: each session is a small record. The key tells Python which part of each record to compare. Sorting by client groups related sessions together before you search through dates or files.

Big Idea

Made with Goodnotes

Sorting means arranging data into a useful order. It can be alphabetical, numerical, chronological, by client name, by date, by score, or by any other rule that helps a person or program use the information more easily.

Sorting is useful because organized data is easier to search, easier to scan, easier to compare, and easier to report. It is not only about making a list look neat. It is about reducing confusion and making later work simpler.

Full Teaching Explanation

This lesson begins with everyday examples because sorting is something you already rely on constantly. Contacts are easier to find when names are alphabetized. Books are easier to locate when arranged by title, author, or subject. Photos from a shoot are easier to deliver when grouped by client, session, date, or rating instead of dumped into one chaotic folder with filenames that look like they were named by a sleep-deprived robot.

In programming, sorting does the same kind of organizing. A list like ["Zoe", "Ava", "Mason", "Ben"] contains the right names, but it is harder to scan than ["Ava", "Ben", "Mason", "Zoe"]. The values did not change. The order changed. That order can make humans faster, and it can make some algorithms possible.

The connection to searching is important. Linear search can work on unsorted data, but it may need to check many items. Binary search only works correctly when the list is sorted. That means sorting can be a setup step: you spend some work up front to organize the data so later searching, reading, or reporting becomes easier.

Sorting is not automatically free. It takes work. If the list is tiny or changes constantly, sorting again and again may not be worth it. If the list is large, reused often, or needs to be presented clearly, sorting can pay off.

Real-World Sorting Examples

Situation	Useful sort order	Why sorting helps
Phone contacts	Alphabetical by name	You can jump near the person instead of scrolling randomly.
Horse show photos	By rider, class, date, or session	Finding deliverables and client galleries becomes faster and less chaotic.
Grades or scores	Lowest to highest or highest to lowest	It reveals rank, top scores, outliers, and patterns.
File folders	By date or project	Related work stays together and repeated searching takes less effort.
Numbers before binary search	Smallest to largest	The sorted order makes halving possible.

Code Example - Sorting Names

```
names = ["Zoe", "Ava", "Mason", "Ben"]
sorted_names = sorted(names)

print(names)
print(sorted_names)
```

Plain English: keep the original list, create a sorted version, and print both so you can see that sorted() returns a new ordered list. The original names list is still in its original order.

Sorting Decision Questions

Made with Goodnotes

Question	What it tells you
Will a human need to scan this list?	Sorting may improve readability and reduce frustration.
Will the program search this data many times?	Sorting once may make later search faster.
Does binary search need to be used?	The list must be sorted first.
Does the data change constantly?	Repeated sorting may waste work.
Is there a meaningful order?	Sorting needs a rule: alphabetic, numeric, date, rank, etc.

Terms, Definitions, and Why They Are Called That

- Sorting: arranging values into a chosen order. It is called sorting because items are grouped or ordered by a rule instead of left random.
- Sort key: the value used to decide order, such as name, date, score, or client. It is called a key because it unlocks what part of each item should becompared.
- Ascending order: smallest-to-largest or A-to-Z order. Ascending means moving upward.
- Descending order: largest-to-smallest or Z-to-A order. Descending means moving downward.
- Setup cost: work done early so later work becomes easier. Sorting can be a setup cost before faster searches or cleaner reports.

Common Mistakes and Helpful Tips

- Mistake: thinking sorting changes the data itself. Usually, the values stay the same; only their order changes.
- Mistake: sorting without knowing the goal. Alphabetical order helps names, but date order may help sessions, and score order may help rankings.
- Mistake: assuming sorting is always worth it. If the data is tiny or constantly changing, sorting may not save enough work.
- Tip: ask what order would make the next task easier: search, scan, compare, display, or report.
- Memory hook: sorting is not decoration; it is organization that makes later work easier.

Notes

Made with Goodnotes

Find-the-Smallest Strategy

Step	Selection-style thinking
1	Look through the unsorted part of the list.
2	Find the smallest remaining value.
3	Swap or place that value into the next correct position.
4	Treat that position as finished.
5	Repeat for the remaining unsorted values.

Made with Goodnotes

This is the mindset behind selection sort. It is not swapping every messy neighbor; it is choosing the next best item and locking it into place.

Lesson 8 - Sorting Different Approaches

Big Idea

There is more than one way to sort. Different sorting approaches follow different strategies, even when they produce the same final ordered list.

This lesson introduces sorting as a family of algorithms. Some approaches repeatedly compare neighbors. Some repeatedly select the smallest remaining item. Some insert each new item into the right place. The final goal is the same, but the thinking pattern differs.

Full Teaching Explanation

Imagine several people trying to organize numbered cards on a table. One person might compare neighboring cards and swap them whenever the left card is bigger than the right. Another person might scan the whole row, find the smallest card, and place it at the front. Another person might pick up cards one at a time and insert each card into the correct place among the cards already sorted.

Those are all sorting strategies. Bubble sort is the neighbor-swapper. Selection sort is the smallest-card hunter. Insertion sort is the careful card-inserter. Each one gives the computer a rule for turning disorder into order.

Python gives us sorted() and .sort(), but those are convenient tools built on algorithmic ideas. Learning simple sorting approaches helps you understand what it means for a program to compare values, move values, repeat passes, and stop when order is achieved.

The big beginner skill is recognizing the story each algorithm tells. Bubble-style thinking asks: are these two neighbors in the wrong order? Selection-style thinking asks: what is the smallest remaining item? Insertion-style thinking asks: where does this new item belong in the part I have already organized?

Sorting Approaches Compared

Approach	Core question	Mental model
Bubble-style thinking	Are these neighboring items in the wrong order?	Walk across the list swapping neighbors until large values drift right.
Selection-style thinking	What is the smallest remaining item?	Find the smallest card and place it into the next correct spot.
Insertion-style thinking	Where does this item fit among the sorted items?	Build a sorted hand of cards one item at a time.
Python built-in sort	Can the language sort this for me safely?	Use sorted() or .sort() when the goal is sorting, not practicing the algorithm.

Neighbor-Swapping Strategy

Made with Goodnotes

Step	Neighbor-swap / bubble-style thinking
1	Look at two neighboring values.
2	If the left value is bigger than the right value, swap them.
3	Move one position to the right and check the next pair.
4	After one full pass, the largest value has moved toward the end.
5	Repeat passes until no more swaps are needed or the list is ordered.

Code Example - Built-In Sort vs Algorithm Thinking

```
numbers = [7, 1, 4, 2]

# Built-in sorting for real projects
ordered = sorted(numbers)
print(ordered)

# Algorithm practice asks a different question:
# "How could we build the sorting steps ourselves?"
```

Plain English: in normal Python work, sorted() is usually the clean choice. But this lesson is about the thinking underneath sorting: repeated comparisons and movements.

Notes

Common Mistakes and Helpful Tips

- Mistake: thinking all sorting algorithms work the same way. They may produce the same result while using very different steps.
- Mistake: mixing up bubble and selection. Bubble swaps neighbors; selection hunts for the smallest remaining item.
- Mistake: focusing only on the final sorted list. Sorting questions usually care about the process, not only the answer.
- Tip: describe the algorithm as a story: swap neighbors, find the smallest, or insert into place.
- Memory hook: same destination, different roads.

Made with Goodnotes

How to Tell Sorting Approaches Apart

Question	Bubble-style answer	Selection-style answer	Insertion-style answer
What does it focus on?	Neighbor pairs	Smallest remaining item	Correct position for the current item
What movement does it use?	Repeated swaps	Choose and place/swap	Shift values and insert
Why beginners like it	Very visual and easy to trace	Clear goal each round	Feels like sorting cards by hand
Main warning	Can be slow for large lists	Still scans repeatedly	Shifting can be confusing at first

Terms, Definitions, and Why They Are Called That

- Swap: exchanging two values. It is called a swap because each value takes the other value's place.
- Pass: one trip through part of the list. It is called a pass because the algorithm passes across the data checking or moving values.
- Neighbor comparison: comparing two values next to each other. Bubble sort relies on neighbor comparisons.
- Selection: choosing one item from a group. Selection sort selects the smallest remaining item each round.
- Insertion: placing an item into the right spot. Insertion sort inserts each item into the sorted portion.

Mini Practice - Match the Sorting Story

Clue in the question	Approach it hints at	Why
Compare neighboring items and swap if wrong	Bubble-style thinking	The algorithm only looks at adjacent pairs.
Find the smallest remaining card	Selection-style thinking	Each round selects one value and locks it into place.
Insert each new value into an already sorted hand	Insertion-style thinking	The sorted portion grows one item at a time.
Use Python to get a clean ordered copy	sorted()	The goal is the final order, not practicing the manual algorithm.

When a quiz describes the behavior, translate it into the story. Neighbor swaps point to bubble sort. Smallest remaining value points to selection sort. A growing sorted hand points to insertion sort.

Lesson 9 - Bubble Sort

Code Example - One Pass of Bubble Sort

```
numbers = [5, 1, 4, 2, 8]

for i in range(len(numbers) - 1):
if numbers[i] > numbers[i + 1]:
temp = numbers[i]
numbers[i] = numbers[i + 1]
numbers[i + 1] = temp

print("After one pass:", numbers)
```

Plain English: walk across neighboring pairs. If the left number is bigger, swap it with the number on the right. This is one pass, not the full sort.

Big Idea

Made with Goodnotes

Bubble sort is a sorting algorithm that repeatedly compares neighboring items and swaps them when they are in the wrong order. After each pass, larger values drift toward the end of the list, like bubbles rising through water.

Bubble sort is not famous because it is the fastest. It is useful for learning because its logic is easy to see: compare neighbors, swap if needed, keep repeating until the list is sorted.

Full Teaching Explanation

Bubble sort starts with a simple local question: are these two neighboring values in the correct order? If the left value is bigger than the right value, they are out of order for ascending sorting, so the algorithm swaps them. Then it moves to the next pair and asks the same question again.

One pass does not necessarily sort the whole list. One pass moves the largest unsorted value toward the right side. For example, in [5, 1, 4, 2, 8], the 5 swaps past 1, then past 4, then past 2. Large values gradually drift toward their final positions.

The word bubble comes from that movement. Large values gradually move to the end through repeated swaps, similar to bubbles rising upward. The algorithm does not know the final position immediately. It discovers order by comparing adjacent pairs over and over.

Bubble sort is easy to trace, but it can do a lot of repeated work. With nested loops, it often behaves like O(n^2): as the list gets longer, the number of pair comparisons grows quickly. That is why bubble sort is mostly a learning tool, not the sort you would normally choose for a large production dataset.

Bubble Sort Step Pattern

Step	What happens	Plain-English meaning
1	Start at the beginning of the list.	Look at the first pair of neighbors.
2	Compare the left and right neighbor.	Ask whether they are in the wrong order.
3	Swap if the left value is larger.	Move the bigger value to the right.
4	Move to the next pair.	Keep checking neighbors across the list.
5	Repeat passes.	Keep bubbling large values toward the end until sorted.

Trace Example - One Bubble Sort Pass

Comparison	List after comparison	Why
5 and 1	[1, 5, 4, 2, 8]	5 is bigger than 1, so swap.
5 and 4	[1, 4, 5, 2, 8]	5 is bigger than 4, so swap.
5 and 2	[1, 4, 2, 5, 8]	5 is bigger than 2, so swap.
5 and 8	[1, 4, 2, 5, 8]	5 is smaller than 8, so leave them.
End of pass	[1, 4, 2, 5, 8]	A large value has moved toward the right side.

Made with Goodnotes

Code Example - Repeated Passes

```
numbers = [8, 7, 1, 3, 6]

for pass_num in range(len(numbers) - 1):
for i in range(len(numbers) - 1):
if numbers[i] > numbers[i + 1]:
temp = numbers[i]             numbers[i]
= numbers[i + 1]            numbers[i +
1] = temp

print(numbers)
```

Plain English: the outer loop repeats passes. The inner loop compares neighboring values during each pass. This nested-loop shape is why bubble sort is commonly described as O(n^2) in beginner Big O terms.

Bubble Sort Concepts and Efficiency

Idea	Meaning for bubble sort
Neighbor comparison	Only two adjacent values are checked at a time.
Swap	Values trade places when they are out of order.
One pass	A walk across the list that pushes a large value toward the end.
Repeated passes	The process continues because one pass usually does not finish the whole sort.

Out Loud Programmer Thinking

I am not trying to find the smallest item directly. I am walking across the list and cleaning up neighbor pairs. If the left neighbor is bigger than the right neighbor, I swap them. If not, I leave them. After enough passes, the list becomes ordered. This is wonderfully visual and wonderfully inefficient - the algorithm equivalent of tidying a room by picking up one sock at a time.

Notes

Made with Goodnotes

O(n^2)	Nested repeated comparisons can grow quickly as list size increases.

Terms, Definitions, and Why They Are Called That

- Bubble sort: a sorting algorithm where large values bubble toward the end through repeated neighbor swaps.
- Adjacent values: values next to each other. Bubble sort only compares adjacent values at each moment.
- Swap: exchange two values so they trade positions.
- Pass: one complete walk through the list comparing neighbor pairs.
- Nested loop: a loop inside another loop. Bubble sort often uses nested loops to repeat passes and comparisons.
- O(n^2): a growth pattern where work increases roughly with n times n.

Common Mistakes and Helpful Tips

- Mistake: thinking one pass always sorts the entire list. One pass often only moves a large value closer to the end.
- Mistake: comparing every item to every other item randomly. Bubble sort compares neighbors in order.
- Mistake: forgetting the temporary variable during a manual swap.
- Mistake: calling bubble sort fast just because it is easy to understand. It is usually not efficient for large lists.
- Tip: trace bubble sort with arrows between neighbor pairs. Write the list after each swap.
- Memory hook: bubble sort = compare neighbors, swap if needed, repeat until sorted.

Made with Goodnotes

Code Example

Python Run Submit code

```python
numbers = [8, 7, 1, 3, 6]

for i in range(len(numbers) - 1):      # one pass
    print("Comparing index", i, "and", i + 1, "→", numbers)
    if numbers[i] > numbers[i + 1]:    # if left is bigger
        # swap them
        temp = numbers[i]
        numbers[i] = numbers[i + 1]
        numbers[i + 1] = temp
        print("  swapped →", numbers)

print("After one pass:", numbers)
```

Output

```
Comparing index 0 and 1 → [8, 7, 1, 3, 6]
  swapped → [7, 8, 1, 3, 6]
Comparing index 1 and 2 → [7, 8, 1, 3, 6]
  swapped → [7, 1, 8, 3, 6]
Comparing index 2 and 3 → [7, 1, 8, 3, 6]
  swapped → [7, 1, 3, 8, 6]
Comparing index 3 and 4 → [7, 1, 3, 8, 6]
  swapped → [7, 1, 3, 6, 8]
After one pass: [7, 1, 3, 6, 8]
```

Lesson 10 - Insertion Sort

Big Idea

Insertion sort builds a sorted section one item at a time. It looks at the next unsorted value, called the key, and inserts that value into the correct position inside the already-sorted left side of the list. The mental model is sorting cards in your hand: each new card gets compared against the cards you already placed, then slides into the right spot.

Full Teaching Explanation

Made with Goodnotes

Insertion sort is one of the friendliest sorting algorithms because it matches something people already do without thinking. If you pick up playing cards one at a time, you do not usually throw the whole hand into chaos every time a new card appears. You keep the left side sorted and place each new card where it belongs.

The algorithm divides the list into two regions. The left region is treated as sorted. The right region is still unsorted. At the beginning, the first item by itself is considered sorted because one item is automatically in order. Then the algorithm takes the next value, stores it as the key, and compares it backward through the sorted region.

If the previous value is larger than the key, that previous value shifts one spot to the right. This creates space. The algorithm keeps shifting until it either reaches the beginning of the list or finds a value that is smaller than or equal to the key. Then the key is inserted into the empty spot. That is why the algorithm is called insertion sort: the main action is inserting each next value into the correct place.

Insertion sort is not the fastest general-purpose algorithm for huge random data. In the worst case, it can still require many comparisons and shifts. But it is excellent for learning because it teaches sorted-region thinking, backward comparison, shifting values, and the difference between swapping and inserting.

Card-Hand Mental Model

Part of the analogy	Sorting meaning	Why it matters
Cards already in your hand	The sorted left side	This portion is already in correct order.
New card you pick up	The key value	This is the value being inserted.
Sliding bigger cards right	Shifting larger elements	This makes room for the key.
Putting the card in place	Insertion step	The key lands where the order stays correct.

Code Example - Insertion Sort in Python

```
numbers = [8, 7, 1, 3, 6]

for i in range(1, len(numbers)):
key = numbers[i]     j = i - 1

    while j >= 0 and numbers[j] > key:
numbers[j + 1] = numbers[j]         j
-= 1      numbers[j + 1] = key

print(numbers)
```

Plain English: start with the second item. Treat everything to its left as sorted. Move left while values are bigger than the key, shift those bigger values right, then place the key into the open spot.

Line-by-Line Explanation

- **for i in range(1, len(numbers))**: starts at index 1 because index 0 is already a sorted section of one item.
- **key = numbers[i]**: saves the value we are trying to insert. This matters because shifting values will overwrite positions.
- **j = i - 1**: begins comparison immediately to the left of the key, inside the sorted section.
- **while j >= 0 and numbers[j] > key**: keeps moving left only while there is still a valid index and the value is too large to stay before the key.
- **numbers[j + 1] = numbers[j]**: shifts the larger value one space to the right. This is not a swap; it is making room.
- **j -= 1**: moves the comparison one step farther left.
- **numbers[j + 1] = key**: places the saved key in the open location after the shifting stops.

Trace Example - First Few Insertions

Made with Goodnotes

Pass	Key	What gets shifted?	List after insertion
Start	-	-	[8, 7, 1, 3, 6]
i = 1	7	8 shifts right	[7, 8, 1, 3, 6]
i = 2	1	8 shifts, then 7 shifts	[1, 7, 8, 3, 6]
i = 3	3	8 shifts, then 7 shifts	[1, 3, 7, 8, 6]
i = 4	6	8 shifts, then 7 shifts	[1, 3, 6, 7, 8]

Terms, Definitions, and Why They Are Called That

- Insertion sort: a sorting algorithm that inserts each next value into its correct place within an already sorted section. It is called insertion sort because thekey action is inserting, not repeatedly swapping every neighbor pair.
- Key: the value currently being placed. It is called the key because it is the value we hold onto while we decide where it belongs.
- Sorted section: the part of the list that has already been organized. In insertion sort, this usually grows from left to right.
- Shift: moving a value one position to the right. This creates the opening where the key can be inserted.
- In-place sort: a sort that rearranges the original list instead of building a completely separate sorted list.

Common Mistakes and Helpful Tips

- Mistake: calling every movement a swap. In insertion sort, larger values usually shift right while the key waits.
- Mistake: forgetting to save the key before shifting. If you do not save it, you can overwrite the value you meant to insert.
- Mistake: starting the loop at 0. The algorithm usually starts at index 1 because the first item alone is already sorted.
- Mistake: inserting at j instead of j + 1. The loop stops one step before the correct opening.
- Tip: say it out loud: hold the key, shift bigger values right, drop the key into the opening.

Made with Goodnotes

Notes

Code Example

Python Run Submit code

```python
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1

        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -=1
        arr[j + 1] = key

    return arr

print(insertion_sort([1, 2, 4, 3, 5]))
```

Output

```
[1, 2, 3, 4, 5]
```

Lesson 11 - Bubble Sort vs Insertion Sort

Big Idea

Bubble sort and insertion sort are both beginner-friendly sorting algorithms, and both can be slow on large random lists. The difference is how they move values. Bubble sort repeatedly compares neighbors and swaps them. Insertion sort builds a sorted left side and inserts each new value where it belongs.

Full Teaching Explanation

This lesson matters because two algorithms can have the same broad goal - sorting a list - while using completely different strategies. Bubble sort is neighbor-focused. It walks across the list asking, "Are these two neighbors in the wrong order?" If yes, it swaps them. After one full pass, a large value tends to bubble toward the end.

Insertion sort is placement-focused. It asks, "Where does this next value belong inside the part I already sorted?" Instead of swapping adjacent pairs again and again, it holds a key value and shifts larger values to make space. Once the opening is found, the key is inserted.

Both can take O(n^2) time in the worst case because they may require repeated comparisons as the list grows. But insertion sort often feels smarter on small or nearly sorted data because it can do very little work when values are already close to where they belong. Bubble sort is easier to visualize but often less efficient in practice.

The point is not to memorize one as "good" and one as "bad." The point is to recognize the shape of the strategy. Bubble sort is repeated neighbor cleanup. Insertion sort is sorted-hand building. Once you can see those shapes, code tracing becomes much less chaotic.

Side-by-Side Decision Table

Made with Goodnotes

Question	Bubble sort answer	Insertion sort answer
Main action	Compare neighboring values and swap if needed.	Insert the key into the sorted left side.
Mental model	Bubbles rising to the end of the list.	Sorting cards in your hand.
Movement style	Many neighbor swaps.	Shifts larger values right, then places key.
Best beginner clue	Nested loops with adjacent comparison like numbers[i] > numbers[i + 1].	A key variable, j moving backward, and shifting values.
Efficiency idea	Usually O(n^2), simple but not great for large data.	Also O(n^2) worst case, but often better for nearly sorted data.

Code Shape Comparison

```
# Bubble sort clue: neighbor comparison and swap
if numbers[j] > numbers[j + 1]:     temp =
numbers[j]     numbers[j] = numbers[j + 1]
numbers[j + 1] = temp

# Insertion sort clue: save key, shift larger values right
key = numbers[i] while j >= 0 and
numbers[j] > key:     numbers[j +
1] = numbers[j]
    j -= 1 numbers[j
+ 1] = key
```

Plain English: bubble sort keeps trading neighboring values. Insertion sort holds one value aside, slides bigger values over, and inserts the held value into the open spot.

How to Tell Which Algorithm You Are Looking At

Code clue	Usually means	Why
numbers[i] > numbers[i + 1]	Bubble sort	The algorithm is comparing immediate neighbors.
temp = numbers[i]	Bubble sort swap	A temporary variable often helps trade two values.
key = numbers[i]	Insertion sort	The algorithm saves the value it wants to place.
j -= 1 inside a while loop	Insertion sort	The comparison walks backward through the sorted section.
numbers[j + 1] = numbers[j]	Insertion sort shift	A larger value slides right to make room for the key.

Trace Thinking: Same List, Different Strategy

For the list [8, 7, 1, 3, 6], bubble sort starts by comparing 8 and 7, swapping them, then comparing 8 and 1, swapping again, and so on. The large value 8 gets pushed to the right through repeated neighbor swaps.

Insertion sort starts with 7 as the key. It compares 7 against 8, shifts 8 right, and inserts 7 at the front. Then it uses 1 as the next key, shifts 8 and 7 right, and inserts 1 at the front. Instead of pushing one large value through neighbors, it keeps inserting the next card into a sorted hand.

When a quiz asks what happens after one pass, be careful. "One pass" means different things depending on the algorithm. In bubble sort, one pass usually means one left-to-right neighbor sweep. In insertion sort, one pass usually means placing one key into the sorted section.

Common Mistakes and Helpful Tips

Made with Goodnotes

- Mistake: assuming both algorithms do the same work because both sort. Same goal, different movement pattern.
- Mistake: treating insertion sort as bubble sort with different names. Insertion sort shifts and inserts; bubble sort swaps neighbors.
- Mistake: saying O(n^2) means the algorithms are identical. Big O describes growth, not the exact behavior or readability.
- Tip: find the variable names. key usually screams insertion sort. adjacent indexes usually scream bubble sort.
- Memory hook: bubble sort cleans neighbor pairs; insertion sort builds a sorted hand.

Key Takeaways

- Bubble sort moves values through repeated neighbor swaps.
- Insertion sort moves the key into the sorted section by shifting larger values.
- Both can be O(n^2), especially on large random input.
- Insertion sort can be efficient on small or nearly sorted lists.
- Algorithm comparison should include readability, amount of movement, prerequisites, and growth rate.

Notes

Lesson 12 - Challenge: Sorting

Big Idea

Sorting challenges test whether you can recognize the sorting pattern, trace the list step by step, and explain why the algorithm changes the list the way it does. The goal is not just to get the final sorted list. The goal is to understand the work that happened along the way.

Full Teaching Explanation

Challenge lessons are where the course asks you to stop watching the algorithm and start driving it. You may be given a list and asked what happens after one pass, which algorithm is being used, which values are compared, or whether the list is sorted correctly. The safest strategy is to slow down and identify the pattern before touching the answer choices.

If the challenge shows neighboring values being compared and swapped, think bubble sort. If the challenge shows a key value being held while larger values shift to the right, think insertion sort. If the challenge asks whether a list must be sorted before searching, connect back to binary search: binary search only works when the data is sorted.

The most common trap is jumping straight to the final sorted list when the question only asks for one pass or one step. A full sort and one pass are not the same thing. Another trap is assuming all sorting algorithms move values the same way. Bubble sort, insertion sort, and other approaches may all produce sorted output, but the intermediate steps look different.

A good challenge answer explains the reason, not only the result. "The list becomes [1, 3, 6, 7, 8]" is useful, but "the key moved left after larger values shifted right" shows real understanding. That is the difference between guessing and tracing.

Sorting Challenge Checklist

Made with Goodnotes

Question to ask	What it tells you
Is the list supposed to be fully sorted or only changed after one pass?	Prevents doing too much work.
Are neighbors being compared?	Likely bubble sort or another adjacent-swap method.
Is there a key value being held aside?	Likely insertion sort.
Are larger values shifting right?	Insertion sort is making room for the key.
Does the question mention binary search?	Confirm the data is sorted before using it.
What exactly is the output asking for?	Final list, one pass, number of checks, or algorithm name.

Practice Trace - Bubble Sort One Pass

List: [5, 1, 4, 2, 8]

Goal: Show the list after one left-to-right bubble pass.

Comparison	Action	List after action
5 and 1	Swap because 5 > 1	[1, 5, 4, 2, 8]
5 and 4	Swap because 5 > 4	[1, 4, 5, 2, 8]
5 and 2	Swap because 5 > 2	[1, 4, 2, 5, 8]
5 and 8	No swap because 5 < 8	[1, 4, 2, 5, 8]

After one bubble pass, the largest value seen during the pass has moved rightward. The list is not fully sorted yet, and that is the whole point of the challenge trap.

Practice Trace - Insertion Sort One Step

List before step: [2, 4, 7, 3, 8]

Assume the sorted section is [2, 4, 7] and the next key is 3.

Step	Action	List state / meaning
Save key	key = 3	Hold 3 aside before shifting.
Compare 7 > 3	Shift 7 right	[2, 4, 7, 7, 8] temporarily
Compare 4 > 3	Shift 4 right	[2, 4, 4, 7, 8] temporarily
Compare 2 > 3	False, stop shifting	The key belongs after 2.
Insert key	Place 3 at index 1	[2, 3, 4, 7, 8]

Challenge Question Types

Made with Goodnotes

Challenge type	What to do first	Common trap
Identify the algorithm	Look for neighbor swaps vs key/shift behavior.	Only looking at the final sorted result.
Predict one pass	Trace exactly one outer-loop pass.	Accidentally fully sorting the list.
Count comparisons	Count only the comparisons the algorithm actually makes.	Counting every possible pair randomly.
Spot the bug	Check indexes, loop bounds, saved key, and insertion location.	Assuming the syntax-looking part is correct.
Choose search after sorting	Ask whether binary search is now allowed.	Forgetting binary search requires sorted data.

Code Debugging Checklist

- For bubble sort, check that neighboring values are compared: item i with item i + 1.
- For bubble sort, check that the swap does not overwrite one value before saving it in temp.
- For insertion sort, check that the key is saved before shifting begins.
- For insertion sort, check that the while loop moves backward and stops when the correct spot is found.
- For insertion sort, check that the key is inserted at j + 1, not j.
- For any sorting challenge, write the list after each visible change. Your eyes are dramatic and will absolutely lie to you.

Terms, Common Mistakes, and Helpful Tips

- Trace: manually following an algorithm step by step. It is called a trace because you are tracking the path the data takes.
- Pass: one major sweep or cycle through part of the algorithm. The meaning depends on the algorithm, so define it before answering.
- Comparison: checking two values to decide order. Sorting is built out of many small comparisons.
- Swap: trading two values. Bubble sort uses swaps heavily.
- Shift: moving a value to make room. Insertion sort uses shifting heavily.
- Tip: write the question goal at the top of your scratch work: final list, one pass, one step, algorithm name, or number of checks.

Notes

Lesson 13 - Using Python Sorting Tools in Practice

Big Idea

Python gives you built-in sorting tools so you do not always have to write bubble sort or insertion sort by hand. The big practical tools are list.sort(), sorted(), reverse=True, and key=. These tools let you sort numbers, strings, tuples, and records in a clean, readable way.

This lesson connects the earlier manual sorting algorithms to real Python habits. Bubble sort and insertion sort teach how sorting works underneath. Python sorting tools are what you usually reach for when you need a polished program to arrange data efficiently.

Full Teaching Explanation

Made with Goodnotes

The earlier sorting lessons focused on how a list can be rearranged step by step. That matters because it builds the mental model: comparisons decide order, and values move until the collection has a useful arrangement. In real Python code, though, you usually do not want to manually write those steps every time. Python already has reliable sorting tools built in.

The first tool is list.sort(). This changes the original list in place. In place means the list object itself is rearranged. If scores = [3, 9, 1, 7] and you call scores.sort(), the variable scores now refers to [1, 3, 7, 9]. Nothing new has to be assigned because the existing list was modified.

The second tool is sorted(). This creates a new sorted list and leaves the original alone. That difference is small but important. If you need to keep the original order for later, sorted() is safer. If you want to permanently reorder the list you already have, sort() is direct and clean.

The third tool is reverse=True. By default, Python sorts from low to high or A to Z. reverse=True flips the result so numbers go high to low and strings go Z to A. The fourth tool, key=, tells Python what part of each item should be used for sorting. This is how you sort pairs by score, dictionaries by a field, or names by length instead of alphabetical order.

Python Sorting Tool Comparison

Tool	What it does	Best use
list.sort()	Reorders the original list in place.	Use when you want the existing list changed.
sorted(list)	Returns a new sorted list without changing the original.	Use when you need both original and sorted versions.
reverse=True	Flips the sorted order from ascending to descending.	Use for high-to-low scores, newest-to-oldest dates, or Z-to-A text.
key=	Tells Python what part of each item to sort by.	Use for tuples, dictionaries, objects, lengths, names, or custom rules.
lambda	A tiny unnamed function often used with key=.	Use when the sorting rule is simple and does not need a separate function.

Code Example - sort(), sorted(), and reverse=True

```
scores = [3, 9, 1, 7]

scores.sort()
print(scores)          # [1, 3, 7, 9]

original = [3, 9, 1, 7] new_scores =
sorted(original) print(original)
# [3, 9, 1, 7] print(new_scores)
# [1, 3, 7, 9]

scores.sort(reverse=True)
print(scores)          # [9, 7, 3, 1]
```

Plain English: sort() changes the list you already have. sorted() gives you a new list. reverse=True means descending order instead of ascending order.

Out Loud Programmer Thinking

Before I choose a sorting tool, I ask two questions: Do I want to change this exact list, or do I need a separate sorted copy? Then I ask what Python should compare for each item. If the answer is the whole item, the basic sort is enough. If the answer is one field or one position inside each item, I need key=.

That habit keeps the code from becoming random incantation soup. The line of code should match the sentence: sort this data by this piece, in this direction.

Lesson 13 - Using Python Sorting Tools in Practice (continued)

Code Example - Sorting Pairs with key=lambda

Made with Goodnotes

```
players = [("Alex", 80), ("Bob", 50), ("Cara", 90)]

players.sort(key=lambda pair: pair[1])
print(players) # [("Bob", 50), ("Alex", 80),
("Cara", 90)]

players.sort(key=lambda pair: pair[1], reverse=True)
print(players) # [("Cara", 90), ("Alex", 80),
("Bob", 50)]
```

Plain English: pair[1] means sort by the second value inside each pair. In this example, that second value is the score. Adding reverse=True puts the highest scores first.

Terms, Definitions, and Why They Are Called That

- In-place: changing the existing object instead of making a separate copy. list.sort() sorts in place because the original list is rearranged.
- Return value: the value a function gives back. sorted() returns a new list; list.sort() returns None because its main job is modifying the list itself.
- Ascending order: smallest to largest or A to Z. It is called ascending because the values climb upward.
- Descending order: largest to smallest or Z to A. It is called descending because the values move downward.
- Key function: a function that extracts the value Python should compare. key=lambda pair: pair[1] says, "Compare by the second item."- Lambda: a small unnamed function. It is useful when the sorting rule is short enough to write right where it is used.

Common Mistakes and Helpful Tips

- Mistake: expecting scores.sort() to give back a new list. It does not. It changes scores directly and returns None.
- Mistake: using sorted() when you meant to permanently change the original list. sorted() gives a new list, so assign the result if you need it.
- Mistake: sorting pairs without a key and wondering why Python sorts by the first item. If you need the second item, use key=lambda pair: pair[1].
- Mistake: forgetting reverse=True when the question asks for largest first, highest score, most recent date, or descending order.
- Tip: before writing the sort line, say out loud: what exactly am I comparing? The whole item, the first value, the second value, a dictionary field, or acalculated value?

Mini Practice

Question	Best answer / thinking
You need scores from highest to lowest.	scores.sort(reverse=True) or sorted(scores, reverse=True).
You need a new sorted copy but must keep the old order.	Use sorted(original).
You have pairs like (name, score) and need score order.	Use key=lambda pair: pair[1].
You have names and need shortest to longest.	Use key=len or key=lambda name: len(name).

Notes

Made with Goodnotes

Made with Goodnotes

Code Example

Python Run Submit code

Lesson 14 - Advanced Sorting with Python Sorting Tools

Big Idea

Advanced sorting means sorting more realistic data: dictionaries, records, repeated values, ties, and multiple priorities. Instead of only sorting a list of numbers, you may sort players by score, sessions by client, photos by date, or tasks by deadline and priority.

The important thinking shift is this: the item being sorted may be complex, but the sort still needs one comparison rule. The key function is how you tell Python which part of the item matters first.

Full Teaching Explanation

A beginner sorting example usually looks like [4, 1, 8, 2]. Real programs are rarely that tidy. A photography workflow might have sessions with client names, dates, galleries, and delivery status. A game leaderboard might have player names and scores. A course gradebook might have students, assignments, and averages. The data is not a single number anymore; each item is a record.

Python handles this with key functions. If each item is a dictionary, the key function can pull out one field. For example, key=lambda player: player["score"] tells Python to compare the score field. The dictionaries stay intact, but the sort order is decided by that selected value.

Advanced sorting also includes ties. If two players both have a score of 200, sorting only by score does not fully explain which one should appear first. A multi-key sort uses a tuple as the key, such as (-score, name). Python compares the first part first. If the first parts tie, it uses the second part. That is how you can sort by highest score and then alphabetically by name.

The most useful question is: what is the primary sorting rule, and what is the backup rule if there is a tie? Once you can answer that, the Python code becomes much less mysterious.

Advanced Sorting Decision Guide

Situation	Sorting rule	Python idea
Players on a leaderboard	Highest score first, then name A-Z.	key=lambda p: (-p["score"], p["name"])
Client sessions	Client name A-Z, then date.	key=lambda s: (s["client"], s["date"])
Names by length	Shortest name first.	key=len or key=lambda n: len(n)
Tasks by priority	High priority first, then due date.	Use a key tuple with priority order and date.
Photos by timestamp	Oldest first or newest first.	Use key=date field and reverse=True when newest first.

Code Example - Sort Dictionaries by Score and Name

```
players = [
    {"name": "Alex",  "score": 120},
{"name": "Brook", "score": 200},
    {"name": "Casey", "score": 200},
    {"name": "Drew",  "score": 150}
]

players.sort(key=lambda p: (-p["score"], p["name"]))
print(players)
```

Plain English: sort by negative score first so larger scores come earlier. If two scores tie, sort those tied players by name from A to Z.

Why Not Just Use reverse=True?

reverse=True reverses the entire final order. That is fine when every sorting priority should go backward. But if the problem says highest score first and names A-Z for ties, reversing the whole sort would also reverse the names. Using -score keeps score descending while leaving the name tie-breaker ascending.

Translation: reverse=True is great for one direction. Negative numeric keys are often better when only the number part should go high-to-low.

Made with Goodnotes

Lesson 14 - Advanced Sorting with Python Sorting Tools (continued)

Line-by-Line Thinking

Code piece	Plain-English meaning	Why it matters
p	One player dictionary at a time.	The lambda receives each item from the list.
p["score"]	The score field inside that player record.	This is the primary value we care about.
-p["score"]	A negative score.	Turns high scores into smaller comparison values, so high scores appear first in normal ascending sort.
p["name"]	The player name.	Used as the backup rule when scores tie.
(-p["score"], p["name"])	A tuple key with two priorities.	Python compares score first, then name only if needed.

Code Example - Sort Sessions by Client, Then Date

```
sessions = [
    {"client": "Rivera", "date": "2026-07-01"},
{"client": "Adams",  "date": "2026-06-15"},
{"client": "Rivera", "date": "2026-06-21"},
{"client": "Bennett", "date": "2026-06-30"}
]

sessions.sort(key=lambda s: (s["client"], s["date"]))
print(sessions)
```

Plain English: group sessions by client name first. If the same client appears more than once, put that client's sessions in date order.

Terms, Definitions, and Why They Are Called That

- Record: one complete item with multiple pieces of information, such as a player with a name and score. Records act like rows in a table.
- Field: one named piece of information inside a record, such as score, client, date, or name. It is the specific part you may sort by.
- Primary key: the first sorting priority. In a leaderboard, score is usually the primary key.
- Secondary key: the backup sorting priority used when the primary key ties. In a leaderboard, name may be the secondary key.
- Tuple key: a key made of multiple values, like (-score, name). Python compares tuple items from left to right.
- Tie-breaker: the rule used when two items match on the main sorting value.

Common Mistakes and Helpful Tips

- Mistake: trying to sort dictionaries without telling Python which field matters. Dictionaries are complex records, so use key= to choose a field.
- Mistake: using reverse=True when only one part of a multi-key sort should be reversed. For high score then A-Z name, use negative score instead ofreverse=True on the whole tuple.
- Mistake: forgetting that tuple keys compare left to right. The first value is the main rule; later values only matter when earlier values tie.
- Tip: write the sorting sentence before code: "Sort by score high to low, then name A-Z." Then convert each phrase into the key tuple.
- Tip: for advanced sorting, sketch one example tie. If two items have the same main value, what should happen next?

Notes

Made with Goodnotes

Code Example

Python

Run

Submit code

```python
players = [
    {"name": "Alex", "score": 120},
    {"name": "Brooke", "score": 200},
    {"name": "Casey", "score": 200},
    {"name": "Drew", "score": 150},
]

players.sort(key=lambda p: (-p["score"], p["name"]))

print(players)
```

Output

```
[{'name': 'Brooke', 'score': 200}, {'name': 'Casey',
'score': 200}, {'name': 'Drew', 'score': 150}, {'name':
'Alex', 'score': 120}]
```

Python

Run

Submit code

```python
players = [
    {"name": "Alex", "score": 120},
    {"name": "Brooke", "score": 200},
    {"name": "Casey", "score": 200},
    {"name": "Drew", "score": 150},
]

players.sort(key=lambda p: p["name"])                # 1) by name
players.sort(key=lambda p: p["score"], reverse=True)  # 2) by score
print(players)
```

Output

```
[{'name': 'Brooke', 'score': 200}, {'name': 'Casey', 'score': 200}, {'name': 'Drew',
'score': 150}, {'name': 'Alex', 'score': 120}]
```

Python

Run

Submit code

```python
books = [
    {"title": "A", "price": 12.99, "in_stock": True},
    {"title": "B", "price": 5.99, "in_stock": False},
    {"title": "C", "price": 7.50, "in_stock": True},
    {"title": "D", "price": 20.00, "in_stock": False},
]

books.sort(key=lambda b: (not b ["in_stock"], b["price"]))
print(books)
```

Output

```
[{'title': 'C', 'price': 7.5, 'in_stock': True}, {'title': 'A',
'price': 12.99, 'in_stock': True}, {'title': 'B', 'price': 5.99,
'in_stock': False}, {'title': 'D', 'price': 20.0, 'in_stock': False}]
```

Made with Goodnotes

Lesson 15 - Challenge: Advanced Sorting

Big Idea

The advanced sorting challenge asks you to choose and trace the right Python sorting tool for the situation. The answer is usually less about memorizing one line of code and more about reading the problem carefully: What data do I have? What field matters? Should the order be ascending or descending? What happens when values tie?

This is where Week 4 pulls together searching, sorting, efficiency, and Python syntax. You are proving that you can move from a story problem to a clean sorting rule.

Full Teaching Explanation

Advanced sorting problems can look intimidating because the data often has brackets, parentheses, dictionaries, strings, numbers, and lambda functions all piled together like a tiny syntax lasagna. The way through is to separate the data from the rule.

First, identify the shape of the data. A simple list like [7, 2, 9] can be sorted directly. A list of tuples like [("Alex", 80), ("Bob", 90)] needs you to decide whether the first value or second value matters. A list of dictionaries needs a field name, such as "score" or "client". Once you know the shape, the key function becomes a map from item to comparison value.

Second, identify the requested order. Words like smallest, earliest, A-Z, and increasing usually mean ascending order. Words like largest, highest, newest, Z-A, and decreasing usually mean descending order. For one sorting value, reverse=True is often enough. For multiple sorting values with mixed directions, a tuple key may be cleaner.

Third, check for ties. Tie rules are where many quiz mistakes happen. If the problem says sort by score descending, then by name ascending, you cannot just reverse the whole sort unless you are okay with names also going backward. A key like (-score, name) handles that mixed direction correctly.

Challenge Reading Checklist

Question to ask	What it tells you
What is each item?	Number, string, tuple, dictionary, or object.
What part should be compared?	Whole item, index 0, index 1, a dictionary field, length, date, or score.
Ascending or descending?	Whether you need normal order, reverse=True, a negative number, or another transformation.
Are there ties?	Whether you need a second or third sorting priority.
Should the original list change?	Use sort() for in-place change or sorted() for a new list.
What output is being asked for?	Final sorted list, one chosen line of code, printed output, or explanation.

Challenge Example - Choose the Correct Key

```
players = [
    {"name": "Maya", "score": 95},
{"name": "Noah", "score": 88},
    {"name": "Zoe",  "score": 95},
    {"name": "Ari",  "score": 95}
]

players.sort(key=lambda p: (-p["score"], p["name"])) print(players)
```

Plain English: score is the main rule, and high scores should come first. The negative score handles that. When scores tie, name decides the order: Ari, Maya, then Zoe.

Lesson 15 - Challenge: Advanced Sorting (continued)

Made with Goodnotes

Trace the Output

Original item	Key value	Why it lands there
{"name": "Maya", "score": 95}	(-95, "Maya")	Tied score group; M comes after A.
{"name": "Noah", "score": 88}	(-88, "Noah")	Lower score, so it comes after the 95 group.
{"name": "Zoe", "score": 95}	(-95, "Zoe")	Tied score group; Z comes last alphabetically.
{"name": "Ari", "score": 95}	(-95, "Ari")	Tied score group; A comes first alphabetically.

Final order: Ari, Maya, Zoe, Noah. The key values explain the order better than staring at the dictionaries and hoping the answer vibes its way into existence.

Common Advanced Sorting Challenge Types

Challenge type	How to handle it	Trap to avoid
Sort tuples by second value	Use key=lambda item: item[1].	Accidentally sorting by the first value.
Sort dictionaries by one field	Use key=lambda row: row["field"].	Forgetting the field name must be chosen.
Sort high to low	Use reverse=True for simple cases.	Using reverse=True when only one part of a tuple should reverse.
Sort by score descending, then name ascending	Use key=lambda p: (-p["score"], p["name"]).	Reversing both score and name.
Keep original list unchanged	Use sorted(data) and assign the new list.	Using data.sort() and losing the old order.
Explain output	Write the key value beside each item.	Trying to mentally sort complex records without tracing.

Week 4 Connection - Searching After Sorting

Sorting and searching often work together. If data is small or unsorted and you only search once, linear search may be perfectly reasonable. If data is large and you will search it repeatedly, sorting first can make faster search strategies possible.

The important trade-off is setup cost versus repeated benefit. Sorting takes work upfront. Binary search benefits from that organization later. That is the same theme the whole week has been building: algorithms are not just about getting an answer; they are about choosing a strategy that fits the situation.

Final Checklist for Advanced Sorting Questions

- Circle the data shape: list, tuple, dictionary, nested list, or record.
- Underline the sorting rule: score, name, date, length, second item, or another field.
- Mark the order words: highest, lowest, newest, oldest, A-Z, Z-A, increasing, or decreasing.
- Look for tie-breakers. If the problem says "then by," you probably need a tuple key.
- Decide whether the original list should change. If yes, use sort(). If no, use sorted().
- For mixed directions, avoid blindly using reverse=True. Consider negative numbers for descending numeric fields.
- Trace the key value beside each item when the output feels confusing.

Made with Goodnotes

Notes

Week 4 Review Tools - Searching and Sorting

Big Picture Review

Week 4 ties together two huge programming habits: finding information and arranging information. Searching asks, "Where is the thing I want?" Sorting asks, "What useful order should this data be in?" The skills work together because the way data is organized changes how easily it can be searched.

The main search contrast is linear search versus binary search. Linear search is the steady, reliable option: check one item at a time. Binary search is the fast option when the list is sorted: look in the middle, throw away half the possibilities, and repeat. The main sorting contrast is manual algorithm thinking versus Python tool thinking. Bubble sort and insertion sort teach the movement logic. Python sort(), sorted(), key=, lambda, and reverse=True are the practical tools you will usually use in real code.

Week 4 Concept Map

Concept area	Main idea	Refresher sentence
Search target	The exact value you want to find.	If you cannot name the target, you cannot design the search clearly.
Search space	The collection or area you are allowed to search.	The search space is the pile, list, folder, or dataset where the target may live.
Linear search	Check each item one by one.	Best for small, unsorted, simple data.
Binary search	Use the middle of a sorted list to cut possibilities in half.	Fast, but only valid when sorted order is guaranteed.
Sorting	Arrange data into a useful order.	Sorting can make output clearer and searching faster.
Python sorting tools	Use sort(), sorted(), key=, lambda, and reverse=True.	Use tools when the goal is real work, not practicing the algorithm by hand.

Memory Hooks

- Linear search = walk down the line and ask, "Is this it?"
- Binary search = sorted list only, then check the middle and cut the problem in half.
- Bubble sort = neighbors swap until large values bubble toward the end.
- Insertion sort = take one item and slide it left into the sorted part.
- sort() changes the original list; sorted() makes a new sorted list.
- key= answers the question, "What part of each item should Python compare?"

Made with Goodnotes

Notes

Search Review - Targets, Spaces, and Comparisons

Full Teaching Review

Every search problem has three basic pieces: a target, a search space, and comparison steps. The target is the thing you want. The search space is where you are allowed to look. A comparison step is the moment the program asks, "Is this current item the target?"

This vocabulary matters because it keeps search problems from feeling like vague scavenger hunts. If the problem says to find 9 in nums = [3, 7, 9, 2], the target is 9, the search space is the list nums, and each comparison is one check against an item in the list. The same logic applies to contacts, emails, photos, files, or customer records.

When tracing search code, write each comparison in order. Do not jump straight to the answer. Search questions often ask what happens step by step, what gets printed, what index is returned, or how many comparisons happen before the target is found or the search fails.

Search Vocabulary Table

Term	Definition	Why the name makes sense
Search target	The value being searched for.	It is the target because the whole search is aimed at finding it.
Search space	The collection being searched.	It is the allowed space where the target may be located.
Comparison	One check between the current item and the target.	The program compares candidate value versus target value.
Found case	The search reaches the target.	The algorithm can stop because the job is complete.
Not found case	The search finishes without a match.	The algorithm must report failure instead of pretending it found something.
Index	The position where a target is found.	The index answers "where" inside the list.

Linear Search Trace Pattern

```
nums = [3, 7, 9, 2]
target = 9

for i in range(len(nums)):     if
nums[i] == target:
print("Found at index", i)
break
```

Plain English: start at the beginning, check each item against the target, and stop as soon as the target is found.

Trace It Out Loud

- Check index 0: nums[0] is 3. Is 3 equal to 9? No.
- Check index 1: nums[1] is 7. Is 7 equal to 9? No.
- Check index 2: nums[2] is 9. Is 9 equal to 9? Yes.
- Print "Found at index 2" and stop the loop.

Linear Search vs Binary Search Decision Guide

Made with Goodnotes

Full Teaching Review

Linear search is the safest search pattern because it does not need the data to be sorted. It simply checks candidates one by one. That makes it readable and reliable, especially when the list is small, unsorted, changing constantly, or only searched once.

Binary search is faster when the data is sorted because every comparison removes about half of the remaining search space. The danger is that binary search depends completely on order. If the list is not sorted, the middle-value decision becomes meaningless and the algorithm can eliminate the side where the target actually lives.

The best decision is not "binary is always better." The best decision is "what does this situation allow?" If the data is small or unsorted, linear search is often the right first thought. If the data is large and sorted, binary search becomes the stronger choice.

Search Algorithm Comparison

Question	Linear search	Binary search
Does data need to be sorted?	No. It works on sorted or unsorted data.	Yes. Sorted order is required.
How does it search?	Checks each item one at a time.	Checks the middle, then keeps only the half where the target could be.
Best for	Small lists, unsorted lists, one-time searches, simple code.	Large sorted lists, repeated searches, data where order is guaranteed.
Worst case	May check every item.	May keep halving until the search space is empty.
Main risk	Can be slow for large data.	Wrong answer risk if the data is not sorted.
Memory hook	Walk down the line.	Middle, half, repeat.

Binary Search Checklist

- Is the data sorted? If no, do not use binary search yet.
- What is the target?
- What are the low and high boundaries?
- What is the middle index?
- Is the middle value equal to the target, too small, or too large?
- Which half can be safely ignored?

Binary Search Boundary Pattern

```
low = 0
high = len(nums) - 1

while low <= high:     mid
= (low + high) // 2

    if nums[mid] == target:
        return mid     elif
nums[mid] < target:
        low = mid + 1
else:         high =
mid - 1
```

Plain English: keep a left boundary and right boundary. Check the middle. Move one boundary inward after each comparison.

Sorting Review - Why Sorting Helps

Full Teaching Review

Made with Goodnotes

Sorting is the act of arranging values into a useful order. That order might be alphabetical, numerical, chronological, highest-to-lowest, lowest-to-highest, newest-to-oldest, or grouped by a field like client name. Sorting does not just make data look nicer. It can make later work easier.

Sorted data is easier for humans because patterns become visible. A contact list sorted A-Z is easier to scan than names in random order. Photos sorted by session or date are easier to deliver. Scores sorted highest-to-lowest make rankings obvious. Sorted data can also help algorithms. Binary search, for example, only works because sorted order lets the algorithm know which side of the list to eliminate.

The trade-off is that sorting takes work. If the data changes constantly or is only needed once, sorting may not be worth the setup. If the same data will be searched, displayed, filtered, or compared many times, sorting can save frustration later. The Week 4 habit is to ask whether order is useful enough to justify the work.

Sorting Decision Questions

Question	Why it matters	Likely decision
Will humans read this output?	Humans understand ordered lists faster.	Sort if order improves readability.
Will the data be searched many times?	Repeated searching may benefit from sorted organization.	Consider sorting or using a faster lookup structure.
Does binary search need to be used?	Binary search requires sorted data.	Sort first or use already sorted data.
Does the data change constantly?	Repeated sorting can waste work.	Be careful; sorting every tiny update may be inefficient.
Is the list tiny?	The performance difference may not matter.	Use the simplest readable option.
Is the sort rule clear?	Unclear rules produce confusing output.	Define what field or value controls order first.

Sorting Mental Models

- Alphabetical contact list: sorting helps humans find names faster.
- Leaderboard: sorting makes rank visible immediately.
- Photo sessions: sorting by client/date makes delivery and retrieval less chaotic.
- Deck of cards: sorting creates structure from randomness.
- Search preparation: sorting can unlock binary search when the sorted order is valid.

Notes

Manual Sorting Algorithms - Bubble and Insertion

Full Teaching Review

Made with Goodnotes

Bubble sort and insertion sort are not usually the fastest production choices, but they are excellent teaching algorithms because they show how sorting can happen through repeated comparisons and movement. The point is not to worship bubble sort like it is a tiny algorithm celebrity. The point is to understand the movement pattern.

Bubble sort compares neighbors. If two neighboring values are in the wrong order, it swaps them. After one full pass, a large value has usually moved toward the end, like a bubble rising. Repeated passes keep pushing values into place. Bubble sort is easy to trace, but it repeats a lot of work.

Insertion sort grows a sorted section on the left. It takes the next unsorted item and inserts it into the correct spot among the already sorted values. This feels like sorting cards in your hand. You pick up a card, slide it left past larger cards, and place it where it belongs.

Bubble vs Insertion Sort

Feature	Bubble sort	Insertion sort
Core movement	Swap neighboring values that are out of order.	Take one item and slide it left into the sorted part.
Mental model	Bigger values bubble toward the end.	Sorting playing cards in your hand.
What it compares	Adjacent neighbors.	Current item against previous sorted items.
Good teaching purpose	Shows repeated neighbor comparisons and swaps.	Shows growing a sorted section.
Main weakness	Can repeat many passes and waste work.	Can still be slow if many values must shift.
Common beginner trap	Forgetting that one pass may not fully sort the list.	Losing the saved current value while shifting items.

Bubble Sort Shape

```
for pass_num in range(len(nums) - 1):     for i in
range(len(nums) - 1):         if nums[i] > nums[i +
1]:             nums[i], nums[i + 1] = nums[i + 1],
nums[i]
```

Plain English: repeat passes through the list. On each pass, compare neighbors and swap if the left value is larger than the right value.

Insertion Sort Shape

```
for i in range(1,
len(nums)):     current =
nums[i]     j = i - 1

    while j >= 0 and nums[j] >
current:         nums[j + 1] = nums[j]
j -= 1

    nums[j + 1] = current
```

Plain English: save the current value, slide larger sorted values to the right, then insert the saved value into the open spot.

Manual Sorting Quick Checks

- Bubble sort question? Look for neighboring values and swap decisions.
- Insertion sort question? Look for current value, sorted-left section, shifting, and final insertion.
- One pass is not always a full sort. Pay attention to whether the question asks for one pass or the final result.
- If the list changes after each operation, write the list state. Your eyes are not a runtime environment, tragically.

Python Sorting Tools Review

Full Teaching Review

Made with Goodnotes

Manual algorithms explain how sorting works. Python sorting tools are what you use when you need practical, readable code. The two biggest tools are list.sort() and sorted(). Both arrange data, but they do not behave the same way.

list.sort() changes the original list in place. That means the existing list is rearranged. sorted() returns a new sorted list and leaves the original list alone. This distinction appears constantly in quiz questions because beginners often expect sort() to return a new list. It does not. If you write result = scores.sort(), result becomes None, because sort() modifies scores directly.

The key= argument tells Python what to compare. With simple numbers, the whole item is the comparison value. With complex items like tuples or dictionaries, you often need to point Python to the score, name, date, length, or another field. lambda is a quick way to write that extraction rule in place.

Python Sorting Tool Table

Tool	What it does	Common use
list.sort()	Sorts the original list in place and returns None.	When you want to permanently rearrange the list.
sorted(data)	Creates and returns a new sorted list.	When you want a sorted copy and still need the original order.
reverse=True	Sorts in descending order for the whole key.	Highest score first, newest date first, Z-A text.
key=len	Sorts strings or collections by length.	Shortest to longest names, words, or lists.
key=lambda item: item[1]	Sorts by a selected part of each item.	Sort pairs by score, price, date, or second field.
key tuple	Sorts by multiple priorities.	Sort by score first, then name if scores tie.

sort() vs sorted() Check

```
scores = [80, 95, 70]

changed = scores.sort()
print(scores)    # [70, 80, 95]
print(changed)   # None

original = [80, 95, 70] copy =
sorted(original)
print(original)  # [80, 95,
70] print(copy)      # [70,
80, 95]
```

Plain English: sort() changes the list and returns None. sorted() returns a new list and leaves the old one alone.

Common Sorting Tool Mistakes

- Mistake: assigning result = scores.sort() and expecting result to be the sorted list.
- Mistake: using reverse=True when only one part of a multi-part key should go backward.
- Mistake: forgetting that key= chooses what to compare, not what to print.
- Mistake: sorting dictionaries without pointing to a specific field.
- Tip: say the rule in English first: sort players by score descending, then name ascending.

Trace Practice - Search and Sort

How to Use This Page

Tracing means acting like the computer. Do not summarize. Do not skip. Write the changing values after each important step. For search, track the current item, comparison, boundaries, and result. For sorting, track list state after swaps, shifts, or passes.

The goal is not speed at first. The goal is correctness. Once you can trace slowly, the patterns become familiar and faster. This is where the quiz goblins lose their power. Dramatic, but true.

Practice Set

Made with Goodnotes

Prompt	Trace focus	Answer / expected thinking
Linear search: find 4 in [8, 2, 4, 9].	Comparison order.	Check 8 no, 2 no, 4 yes. Found at index 2.
Linear search: find 7 in [1, 3, 5].	Not found case.	Check every item. No match, so return -1, False, or "not found" depending on code.
Binary search: find 11 in [3, 5, 7, 11, 15].	Middle and boundaries.	Middle 7 too small, search right half. Next middle 11 found.
Binary search on [9, 1, 5, 3].	Prerequisite check.	Invalid choice unless sorted first. Binary search requires sorted data.
Bubble one pass on [4, 2, 3].	Neighbor swaps.	Compare 4 and 2 swap -> [2,4,3]. Compare 4 and 3 swap -> [2,3,4].
Insertion sort first insert in [5, 2, 4].	Shifting.	Save 2. Shift 5 right. Insert 2 at front -> [2,5,4].

Trace Checklist

- For linear search: write each item checked and whether it matches.
- For binary search: write low, high, mid, middle value, and which boundary changes.
- For bubble sort: write the list after every swap or after each full pass, depending on the question.
- For insertion sort: write the current value, shifted values, and where current gets inserted.
- For Python sort tools: identify whether the original list changes or a new list is created.

Notes

Weekly Review Question Breakdown

How to Read Week 4 Questions

Most Week 4 review questions are not trying to trick you with brand-new ideas. They usually test one of five things: recognizing the search type, checking the sorted-data requirement, tracing loop behavior, choosing the correct sorting tool, or understanding what key= is doing.

The quickest way to slow down correctly is to mark the clue words. Words like unsorted, small, one by one, every item, middle, sorted, half, highest, lowest, alphabetical, in place, copy, field, and tie usually point directly to the concept being tested.

Question Clue Decoder

Made with Goodnotes

Clue in the question	Likely concept	What to do
"Check each item" or "one by one"	Linear search	Trace candidates from start to finish.
"Sorted list" and "middle"	Binary search	Track low, high, mid, and boundary changes.
"Unsorted list"	Binary search warning	Do not choose binary search unless sorting happens first.
"Neighbors" and "swap"	Bubble sort	Compare adjacent values and move large values right.
"Insert into sorted part"	Insertion sort	Track current value and shifting.
"Change original list"	list.sort()	Use sort(); remember it returns None.
"Keep original order"	sorted()	Use sorted() and assign the new list.
"Sort by score/name/date"	key=	Extract the field Python should compare.
"Then by"	Tuple key	Use multiple priorities in order.

Answer Strategy

- Step 1: Identify whether the problem is asking about searching or sorting.
- Step 2: Circle the condition words: sorted, unsorted, small, huge, repeated, first, highest, by score, by name.
- Step 3: Decide whether you need an algorithm idea or a Python tool.
- Step 4: Trace only the variables that matter. Do not rewrite the whole program unless needed.
- Step 5: Check whether the answer should be an index, a Boolean, a modified list, a new list, or printed output.

Mini Self-Test

Question	Answer
What search works on unsorted data?	Linear search.
What search needs sorted data?	Binary search.
What sorting algorithm swaps neighbors?	Bubble sort.
What sorting algorithm inserts into an already sorted section?	Insertion sort.
What Python tool leaves the original list unchanged?	sorted().
What argument tells Python what field to sort by?	key=.

Common Exam Traps and Debug Thinking

Trap Review

Week 4 has a few traps that show up because the concepts sound similar. Searching and sorting are related, but they are not the same. Linear and binary search both find targets, but they have different requirements. sort() and sorted() both arrange data, but one changes the original list and one creates a new list.

When you feel stuck, pause and ask what the code is trying to protect: the target, the order, the boundaries, the original list, or the selected sorting field. That usually reveals the mistake.

Mistake Fix Table

Made with Goodnotes

Mistake	Why it is wrong	Fix
Using binary search on an unsorted list.	The middle comparison only makes sense when order is guaranteed.	Sort first or use linear search.
Forgetting to update low or high.	The loop may repeat forever or search the wrong range.	After each comparison, move the correct boundary.
Using = instead of == in a comparison.	= assigns; == compares.	Use == when asking whether two values match.
Expecting sort() to return a sorted list.	sort() changes the original list and returns None.	Use scores.sort() alone or use sorted(scores).
Using reverse=True for mixed direction sorting.	It reverses the whole result, including tie-breakers.	Use a tuple key like (-score, name).
Not saving the current value in insertion sort.	Shifting can overwrite the value you are trying to insert.	Store current before the while loop.
Thinking one bubble pass always fully sorts.	One pass may only move one large value into place.	Continue passes until the list is sorted or no swaps happen.

Debug Questions to Ask Yourself

- What exact target am I searching for?
- Is the search space sorted or unsorted?
- What line performs the comparison?
- What changes after each loop: an index, a boundary, a list order, or a sorted copy?
- Am I supposed to return an index, print a message, return True/False, or return a sorted list?
- Does this function mutate the original list or create a new one?

Notes

Week 4 Cheat Sheet

Searching Cheat Sheet

Pattern	Use when	Core logic
Linear search	Small or unsorted data; simple one-time search.	for each item: compare item to target.
Binary search	Large sorted data.	check middle; remove half; repeat.
Found target	Current item equals target.	Return index, True, or found message.
Not found	Search finishes without a match.	Return -1, False, None, or not found message.
Search trace	Quiz asks what happens step by step.	Write each comparison and state change.

Sorting Cheat Sheet

Made with Goodnotes

Pattern/tool	Use when	Core logic
Bubble sort	Practicing neighbor comparison and swaps.	Compare adjacent values; swap if wrong order.
Insertion sort	Practicing sorted-section growth.	Take current item; slide bigger values; insert current.
list.sort()	Original list should change.	Mutates list in place; returns None.
sorted(data)	Need a new sorted copy.	Returns a new list; original stays unchanged.
reverse=True	Whole result should be descending.	Flips sort order.
key=	Need to sort by a field or custom rule.	Extracts the comparison value for each item.
tuple key	Need tie-breakers.	Sorts by first value, then second, then third.

Most Useful Python Sorting Patterns

```
nums.sort()
sorted_nums = sorted(nums)
nums.sort(reverse=True)

names.sort(key=len)
players.sort(key=lambda p: p["score"])
players.sort(key=lambda p: (-p["score"], p["name"]))
```

Plain English: choose sort() or sorted() first, then add reverse= or key= only when the problem asks for direction or a specific field.

One-Minute Review

- Linear search is flexible but may check every item.
- Binary search is fast but requires sorted data.
- Sorting creates order, but sorting itself costs work.
- Bubble sort swaps neighbors; insertion sort inserts into a sorted section.
- Python sorting tools are practical; manual sorting algorithms are mostly for understanding and tracing.- key= is the sorting rule. reverse=True is the direction.

Notes

Final Week 4 Memory Sheet

What You Should Be Able to Explain

- Explain what a search target is and what a search space is.
- Trace a basic linear search and identify the comparison step.
- Explain why binary search only works on sorted data.
- Trace low, high, and mid through a simple binary search.
- Explain why sorting can make later searching or human reading easier.
- Describe the movement pattern of bubble sort and insertion sort.
- Choose between sort() and sorted() based on whether the original list should change.
- Use key= to sort by a selected field or value.
- Use a tuple key when the problem has tie-breakers.
- Recognize the common traps before they eat your quiz score for breakfast.

Made with Goodnotes

Final Plain-English Summary

Searching is about finding a target inside a search space. Linear search checks one item at a time and works almost anywhere. Binary search checks the middle and cuts the search space in half, but it only works when the data is sorted.

Sorting is about arranging data into a useful order. Bubble sort and insertion sort teach how values move through comparison, swapping, and shifting. Python sorting tools let you apply those ideas cleanly in real programs with sort(), sorted(), reverse=True, key=, lambda, and tuple keys.

The deepest Week 4 skill is decision-making. Do not just ask, "What algorithm do I know?" Ask, "What does this data look like? What does the problem require? Does order matter? Will I search once or many times? Should the original list change? What field controls the order?" That is programmer thinking, not just syntax survival.

Last-Minute Memory Hooks

- Target + search space + comparison = search thinking.
- Unsorted? Linear is safe. Sorted and large? Binary becomes powerful.
- Binary search without sorting is like using a map after someone shuffled the streets. Bold choice. Bad choice, but bold.
- Bubble sort swaps neighbors. Insertion sort slides values into place.
- sort() mutates. sorted() creates. key= compares. reverse=True flips.
- When confused, trace slowly. The computer is not guessing, and neither should we.

Notes

Made with Goodnotes